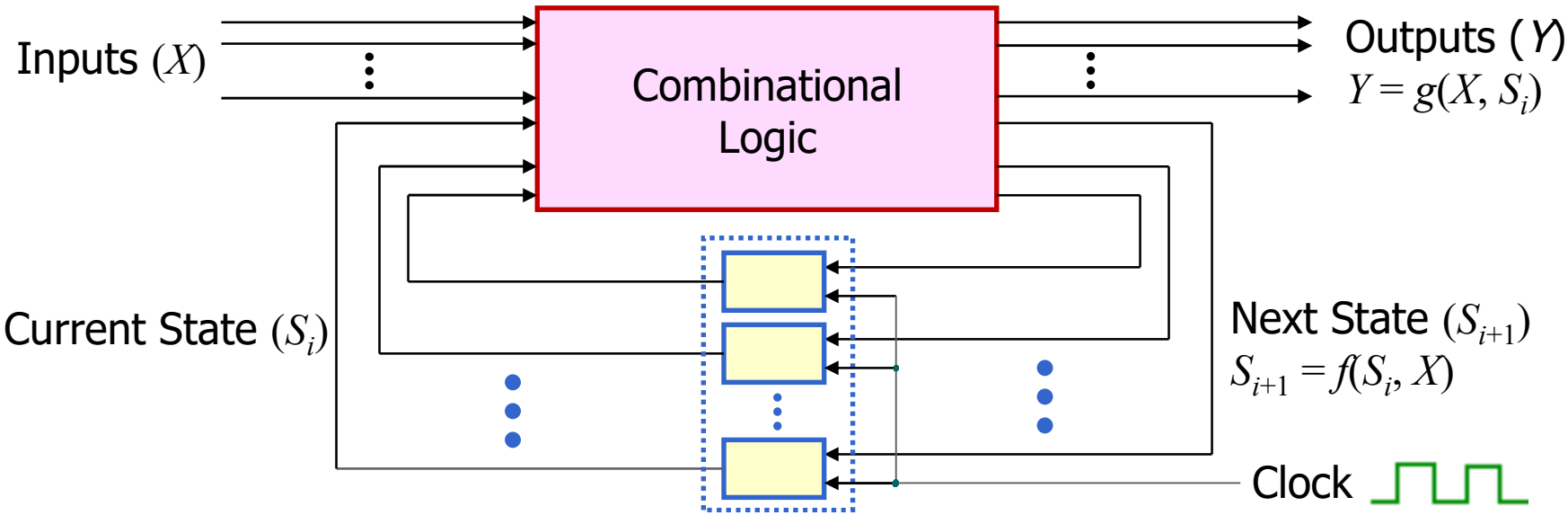
Topic : Sequential Circuits

- Latches & flip-flops
- Synchronous state machine analysis
- Synchronous state machine design
- Timing parameters
- Counters
- Shift registers
- Memory

Clocked Sequential Circuits

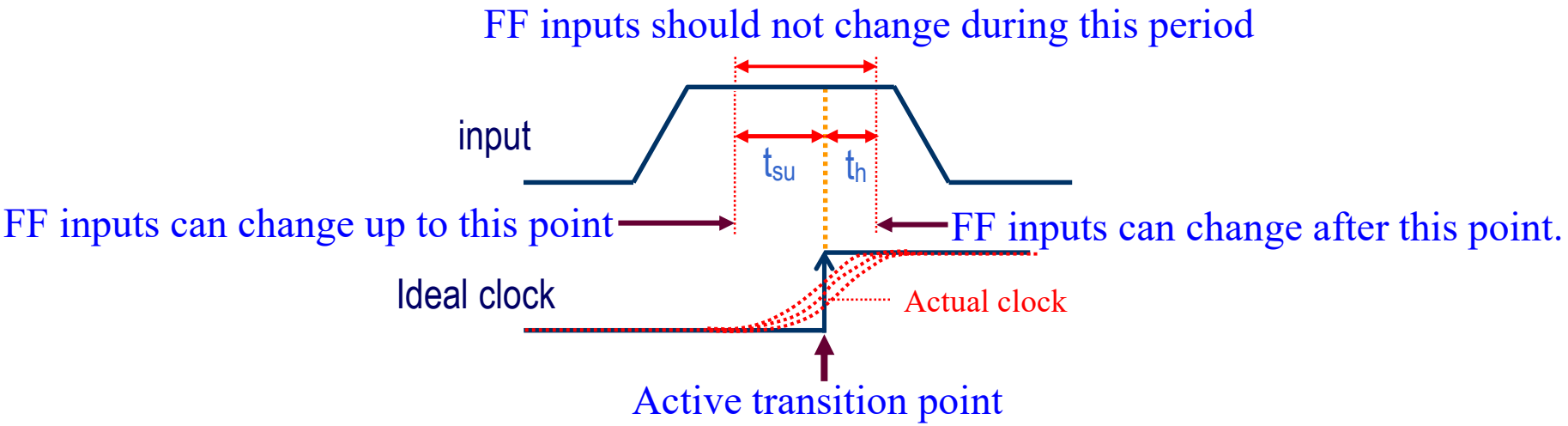
- **States**: determined by possible values in sequential storage elements.
- **Transitions**: change of state.
- **Clock**: controls when state can change by controlling storage elements.



Timing in Clocked Sequential Circuits

- All digital devices have propagation delays (min, max).
- The clock signal synchronizes the operation of flip flops and latches to prevent race conditions and simplify timing analysis.
- In sequential logic, besides propagation delays through gates and wires, changes relative to clocking events of flip flops need to be considered.
- Important timing specifications of sequential devices include
  - ✓ Maximum propagation delay.
  - ✓ Minimum propagation delay.
  - ✓ Setup time.
  - ✓ Hold time.
- At what speeds will a device function?
  - ✓ What is the maximum clock rate?

Setup/Hold Time



- An input to a flip flop is guaranteed to be validly recognized only if:
  - ✓ it is stable before the clocking event for a minimum time interval t_{setup}, and
  - ✓ it is stable after the clocking event for a minimum time interval t_{hold}.

Setup/Hold Time

Setup time : Time required for INPUT DATA to settle BEFORE the triggering edge of clock. It is the minimum time between when the inputs become active until the input is effectively triggered.

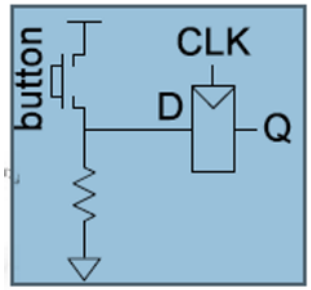
Ignoring this specification, unpredictable behaviors occur:

1. Missed data or ignored actions.
2. Possible partial transient outputs.

Hold time : Time required for DATA to remain stable AFTER the triggering edge of clock. If specification is ignored, unpredictable behavior will result.

- It is dangerous to allow input signals to change very close to the sampling event.
- If setup or hold time constraints are not satisfied, the input may be interpreted as a 1, a 0 or some unrecognizable value between 0 and 1 (metastable value).

Setup/Hold Time



CLK

t_{pd}

If input changes within the window, it is considered a violation

Case 1: D

Q

Data conflict

Long Delay

Case 2: D

Q

Terrible data conflict

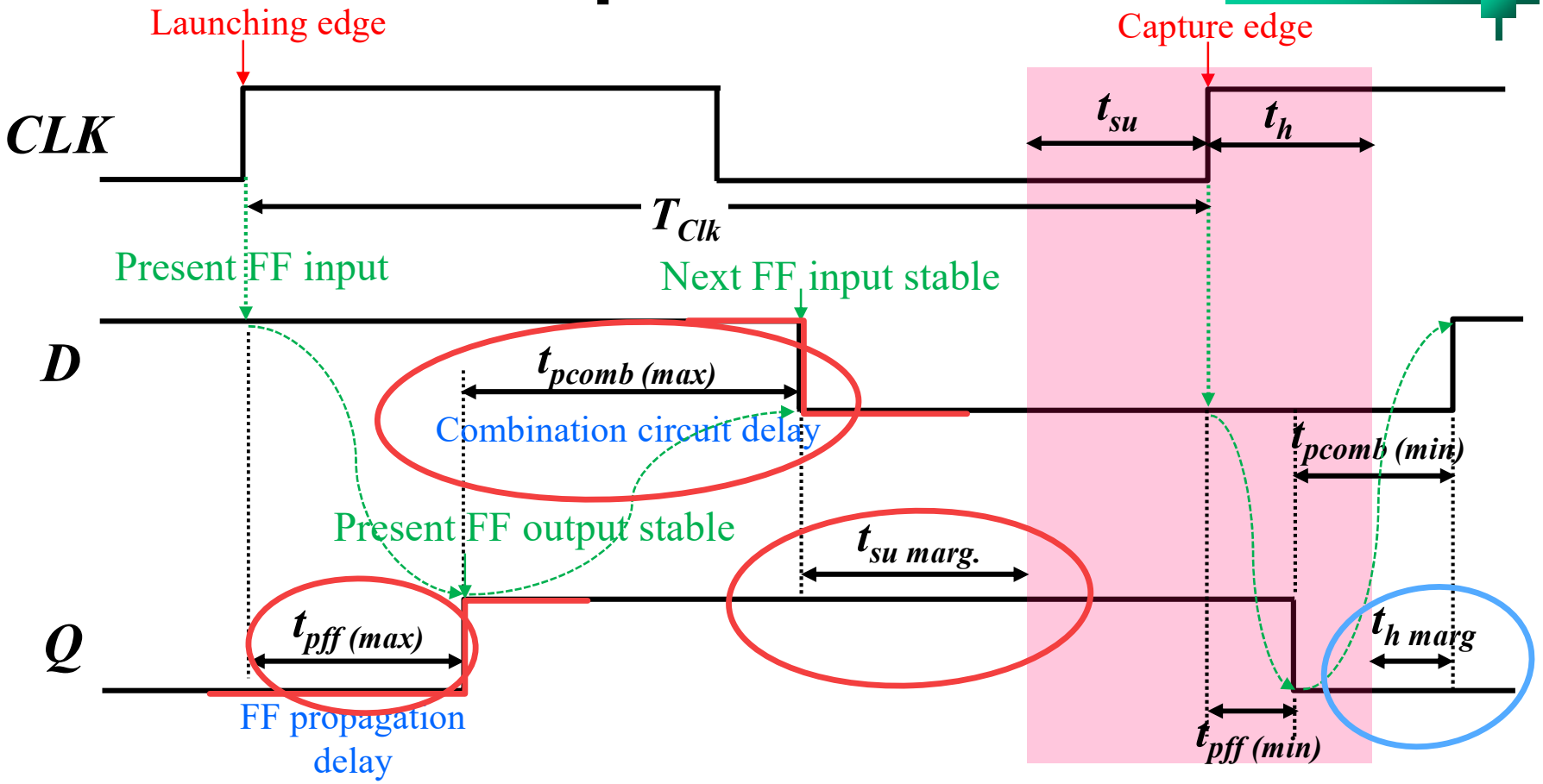
Case 3: D

Q

$t_a = t_{su} + t_h$

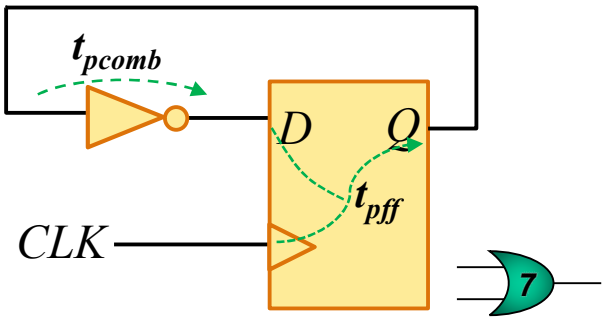
Metastability

Setup/Hold Time

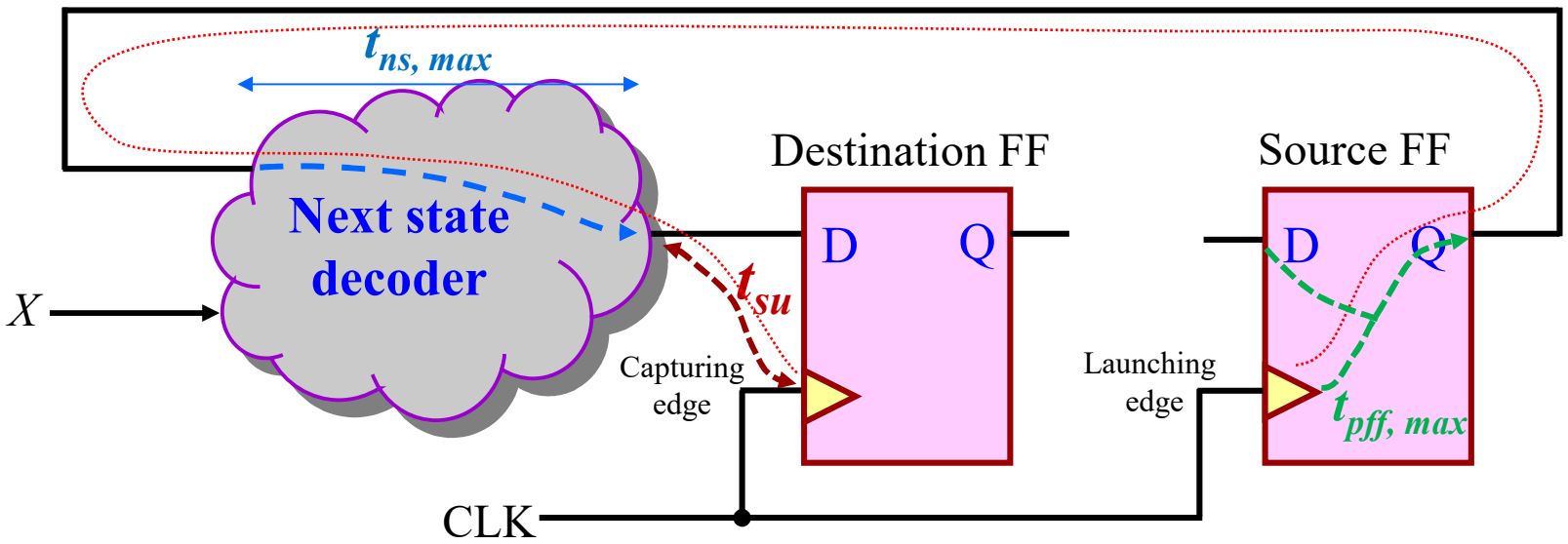


$$T_{Clk} = t_{pff\,(\max)} + t_{pcomb(\max)} + t_{su\ marg} + t_{su}$$

$$t_h + t_{h\ marg} = t_{pff\,(\min)} + t_{pcomb\,(\min)}$$



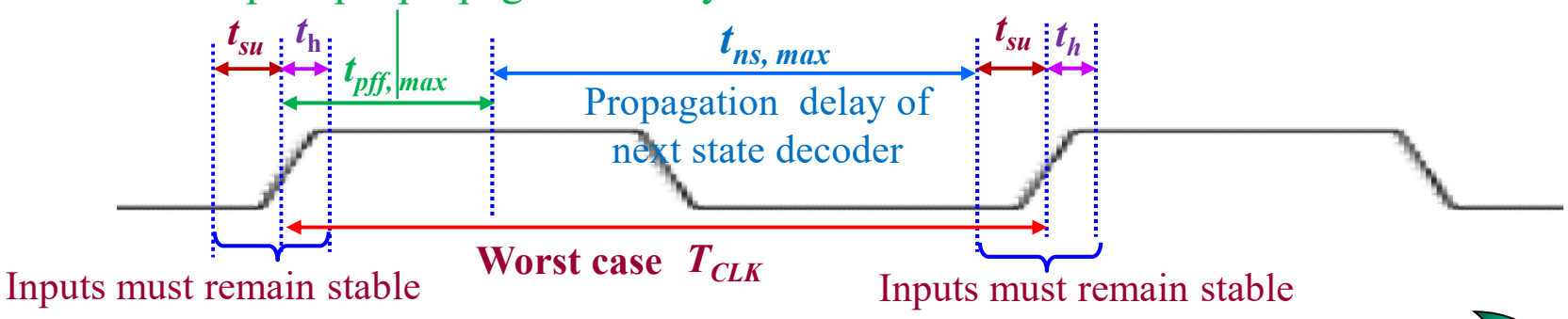
Max. Clock Freq. (Min. Clock Period)



$$T_{CLK} \geq t_{pff,max} + t_{ns,max} + t_{su}$$

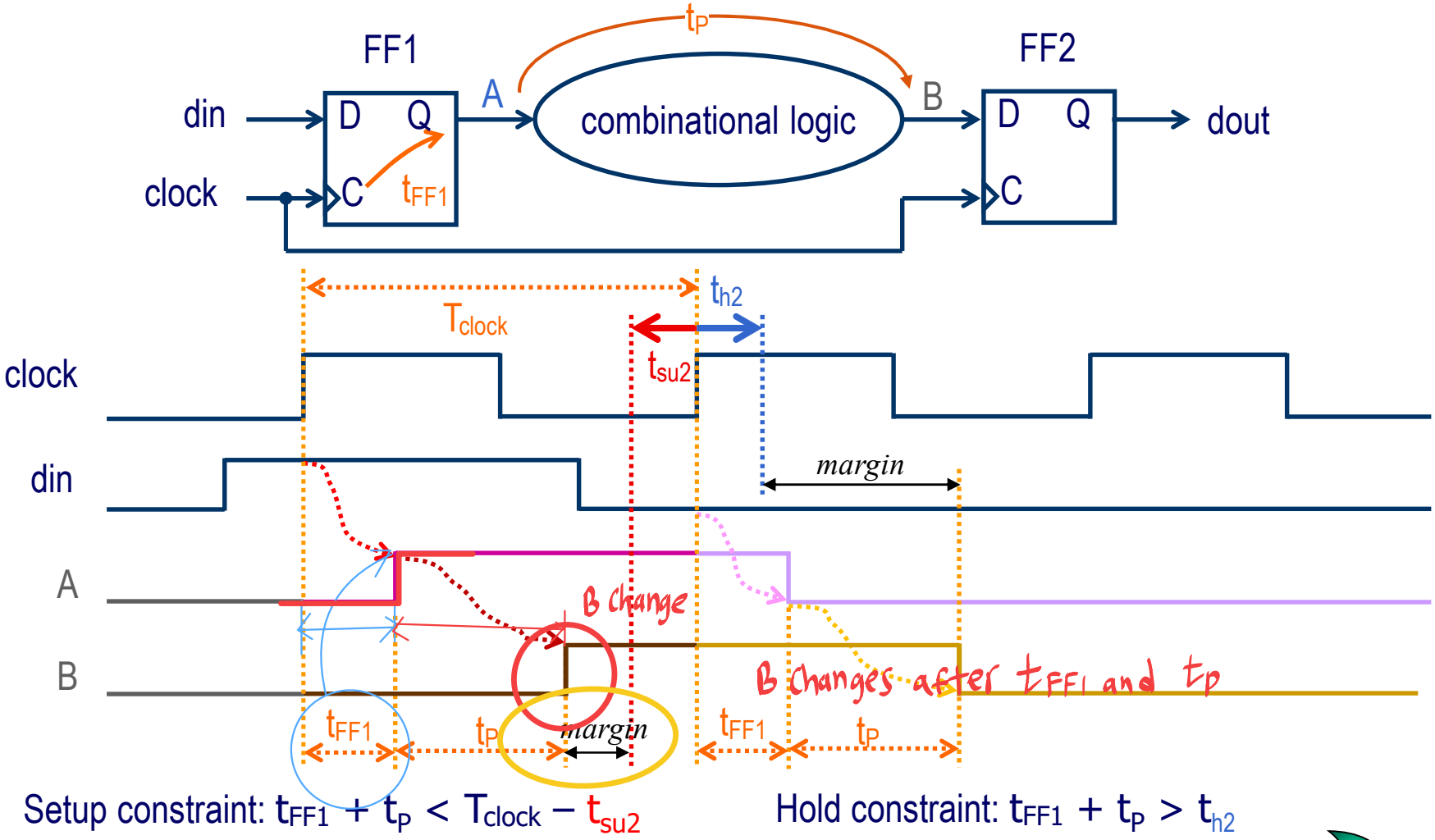
$$f_{CLK,\max} = \frac{1}{T_{CLK,\min}}$$

Flip-flop's propagation delay



Timing constraints in clocked sequential circuits

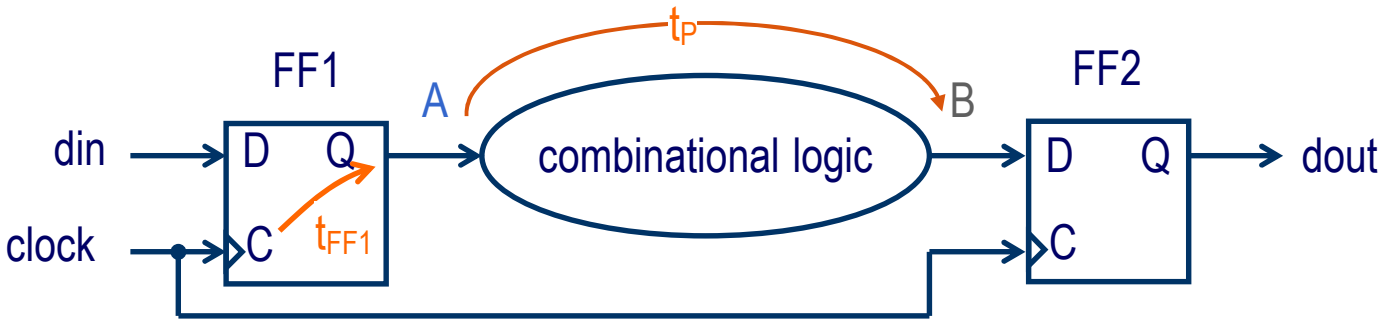
Let's assume that din is applied in a way that satisfies setup and hold for FF1, and let's examine what will happen at FF2.



Setup constraint: $t_{FF1} + t_P < T_{clock} - t_{su2}$

Hold constraint: $t_{FF1} + t_P > t_{h2}$

Max/Min delays

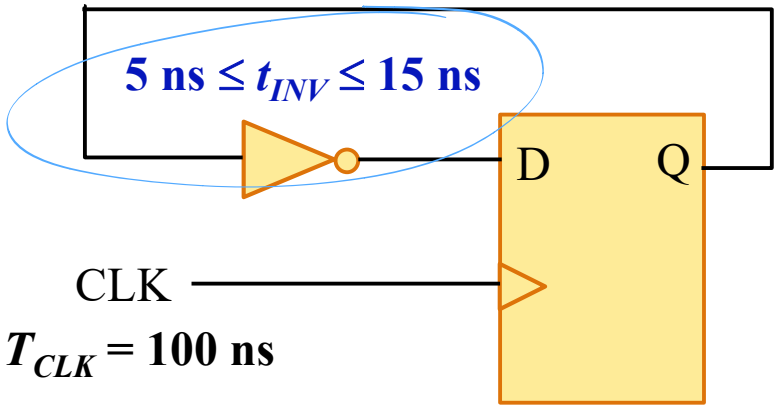


Setup constraint	$t_{FF1} + t_P < T_{clock} - T_{su2}$

Hold constraint	$t_{FF1} + t_P > T_{h2}$

- Unfortunately, delays through gates are not constant. Delays change with:
  - √ Supply Voltage, Temperature, and Manufacturing Process (VTP)
- **Setup constraint** is more difficult to satisfy when delays are **max**
- **Hold constraint** is more difficult to satisfy when delays are **min**

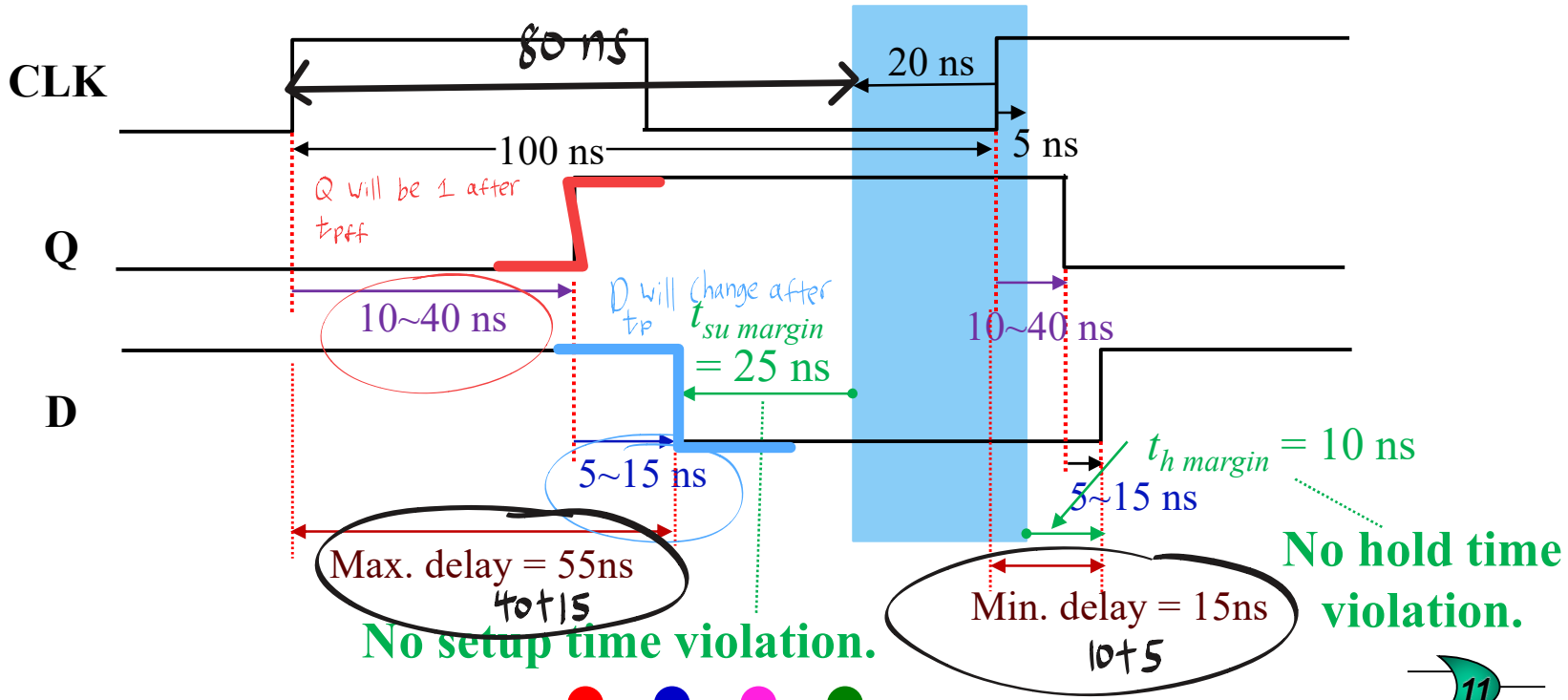
Any Setup or Hold Time Violation?



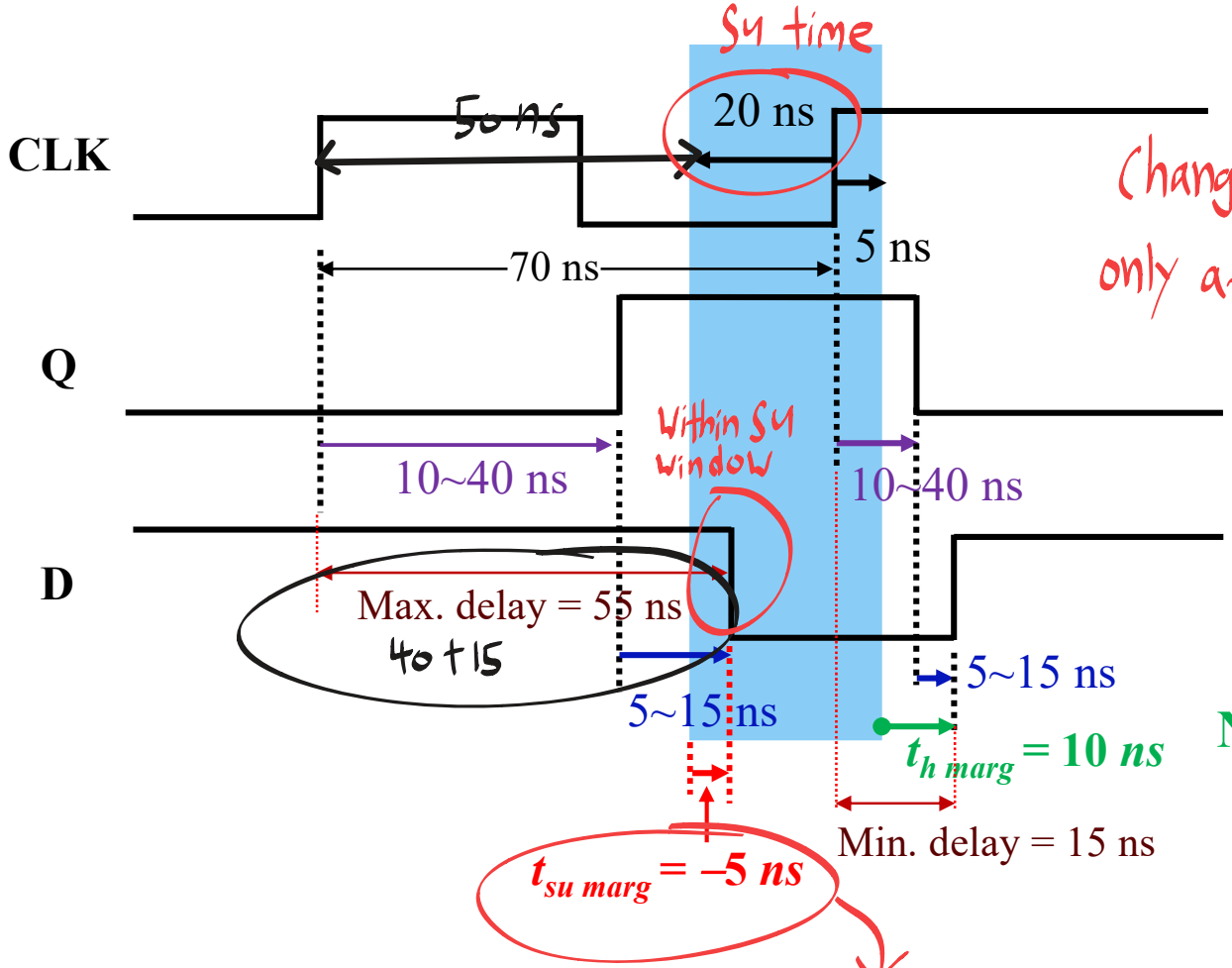
t_{su} = 20 ns

t_h = 5ns

10 ns ≤ t_{pff} ≤ 40 ns



What if T_{CLK} = 70ns?



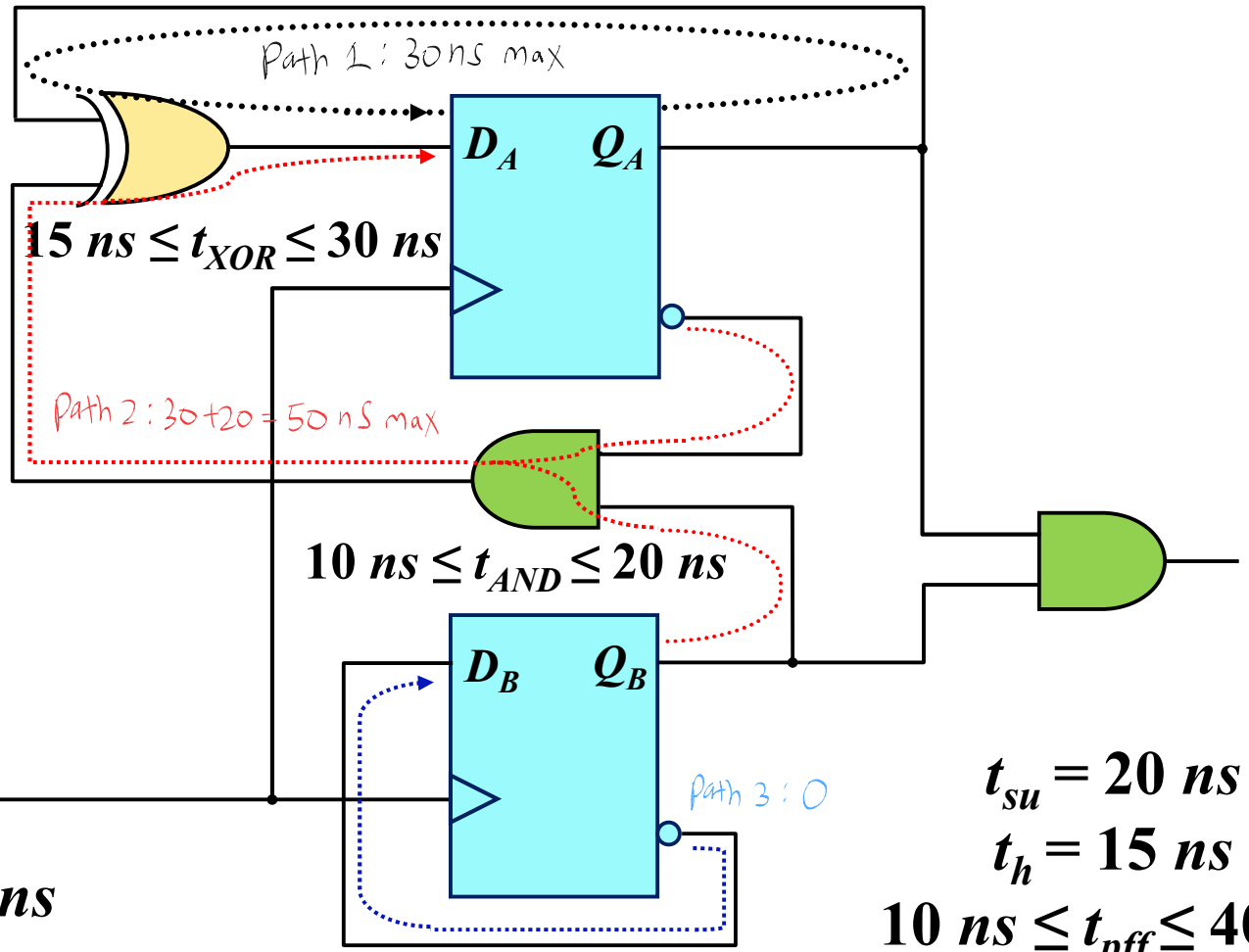
Changing clock period only affects setup time

No hold time violation.

Any Setup or Hold Time Violation?

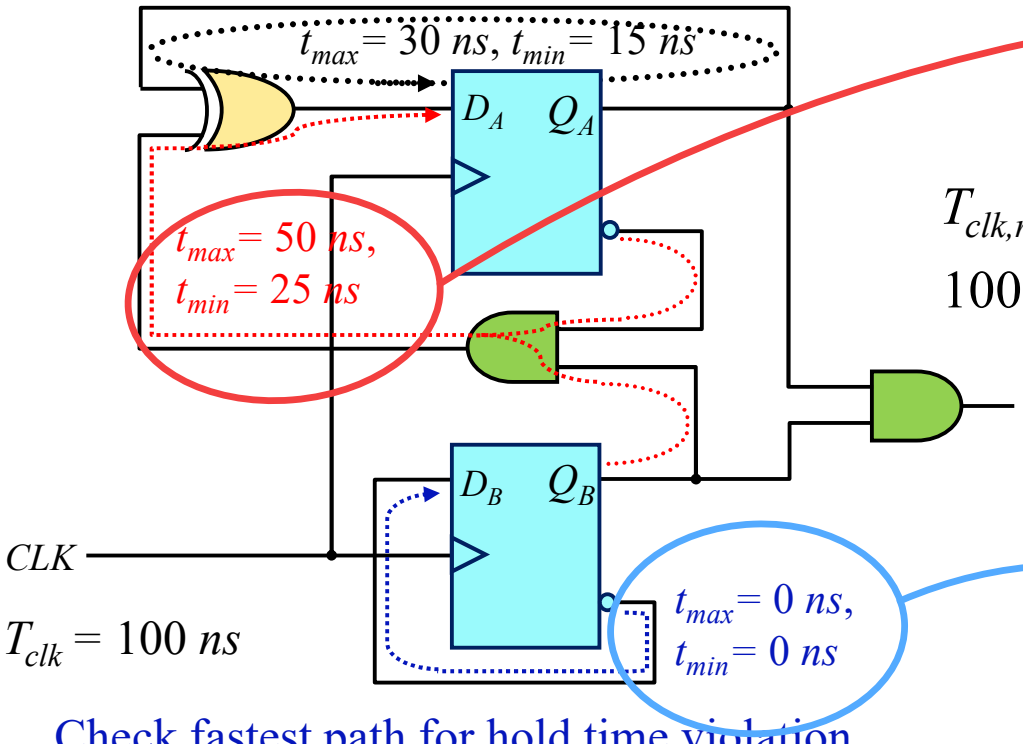
Setup
- find which signal path gives MAX delay

hold time
- which signal path gives MIN delay



Any Setup or Hold Time Violation?

Check slowest path for setup time violation



Setup time (slowest path)

$$t_{pcomb,\,max} = t_{XOR,\,max} + t_{AND,\,max}$$
$$= (30 + 20)\ ns = 50\ ns$$

$$T_{clk,min} = t_{pff,max} + t_{pcomb,\,max} + t_{su} + t_{su\ marg}$$

$$100\ ns = 40\ ns + 50\ ns + 20\ ns + t_{su\ marg}$$

(40 ns: flip flop; 50 ns: what we calculated; 20 ns: su time)

$$t_{su\ marg.} = -10\ ns < 0$$

Setup time violation!

Hold time (shortest path)

$$t_{pcomb,\,min} = 0\ ns$$

$$t_{pff,min} + t_{pcomb,\,min} = t_h + t_{h\ marg}$$

$$10\ ns + 0\ ns = 15\ ns + t_{h\ marg}$$

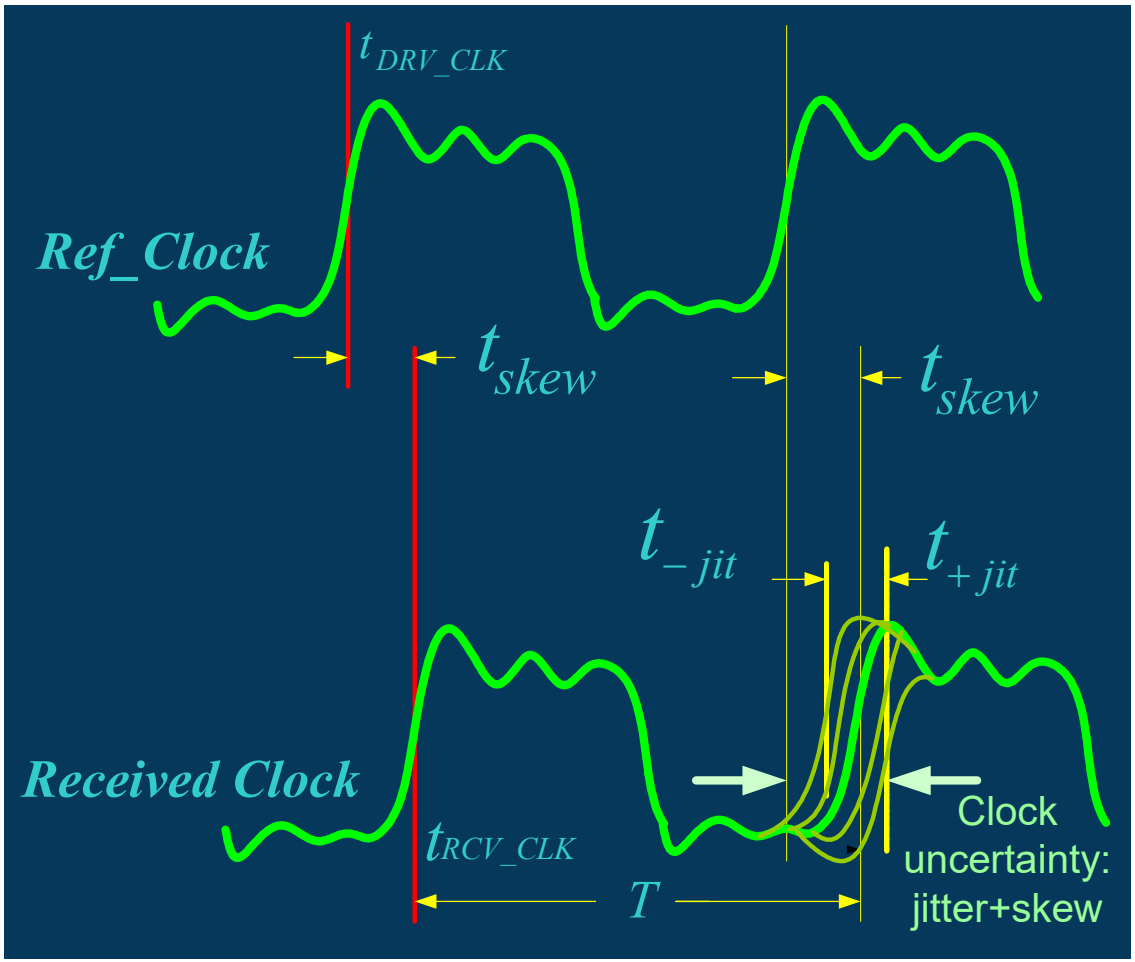
$$t_{h\ marg.} = -5\ ns < 0$$

Hold time violation!

Check fastest path for hold time violation

$$15\ ns \le t_{XOR} \le 30\ ns$$
$$10\ ns \le t_{AND} \le 20\ ns$$
$$t_{su} = 20\ ns$$
$$t_h = 15\ ns$$
$$10\ ns \le t_{pff} \le 40\ ns$$

Clock Uncertainties



Clock skew:

- The difference in the arrival time of the clock signal at different parts of a circuit.
- Caused by differences in wire lengths or clock signal paths.
- Variation is static and spatial.

Clock Jitter:

- The variation in the timing of a clock signal's edges from their ideal position.
- Causes by thermal noise, power supply noise, or signal integrity.
- Variation is random and temporal.

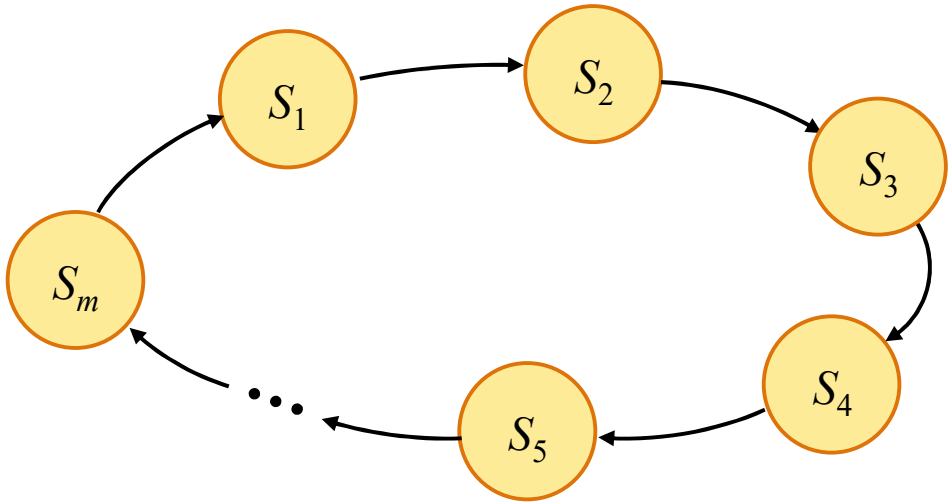
Topic : Sequential Circuits

- Latches & flip-flops
- Synchronous state machine analysis
- Synchronous state machine design
- Timing parameters
- Counters
- Shift registers
- Memory

Counters

- A counter is a clocked sequential circuit that sequences through a fixed set of patterns.
- A counter with *m*-states is called a modulo-*m*-counter, or sometimes a divide-by-*m* counter.
- The most commonly used counter type is an *n*-bit binary counter (each of the states is encoded as the corresponding *n*-bit binary integer).

General structure of a counter's state diagram



Counters

- Depending on the way they are clocked, counters can be broadly categorized as:
  1. Synchronous
  2. Asynchronous

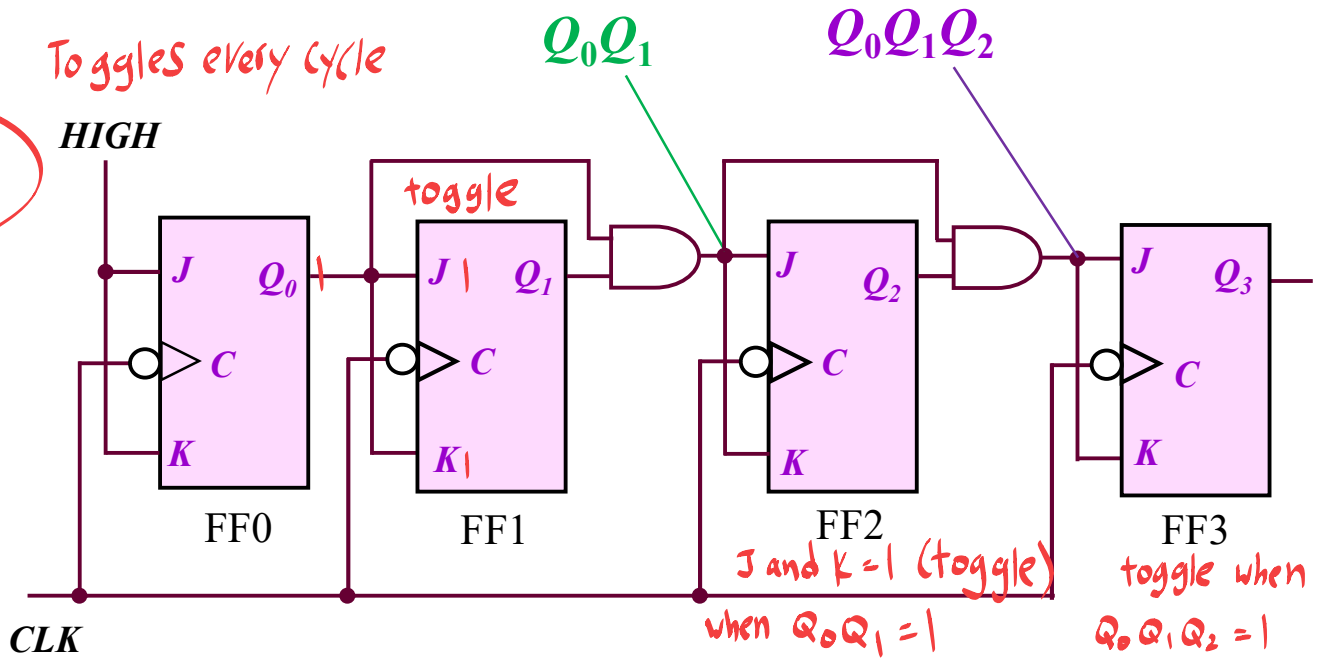
- Counters can also be further classified in terms of:
  1. Type of sequence
  2. Modulus – number of states
  3. Number of flip-flops

- Applications include:
  1. Timing units
  2. Control circuits
  3. Signal generators, etc ...

A 4-bit Synchronous Binary Counter

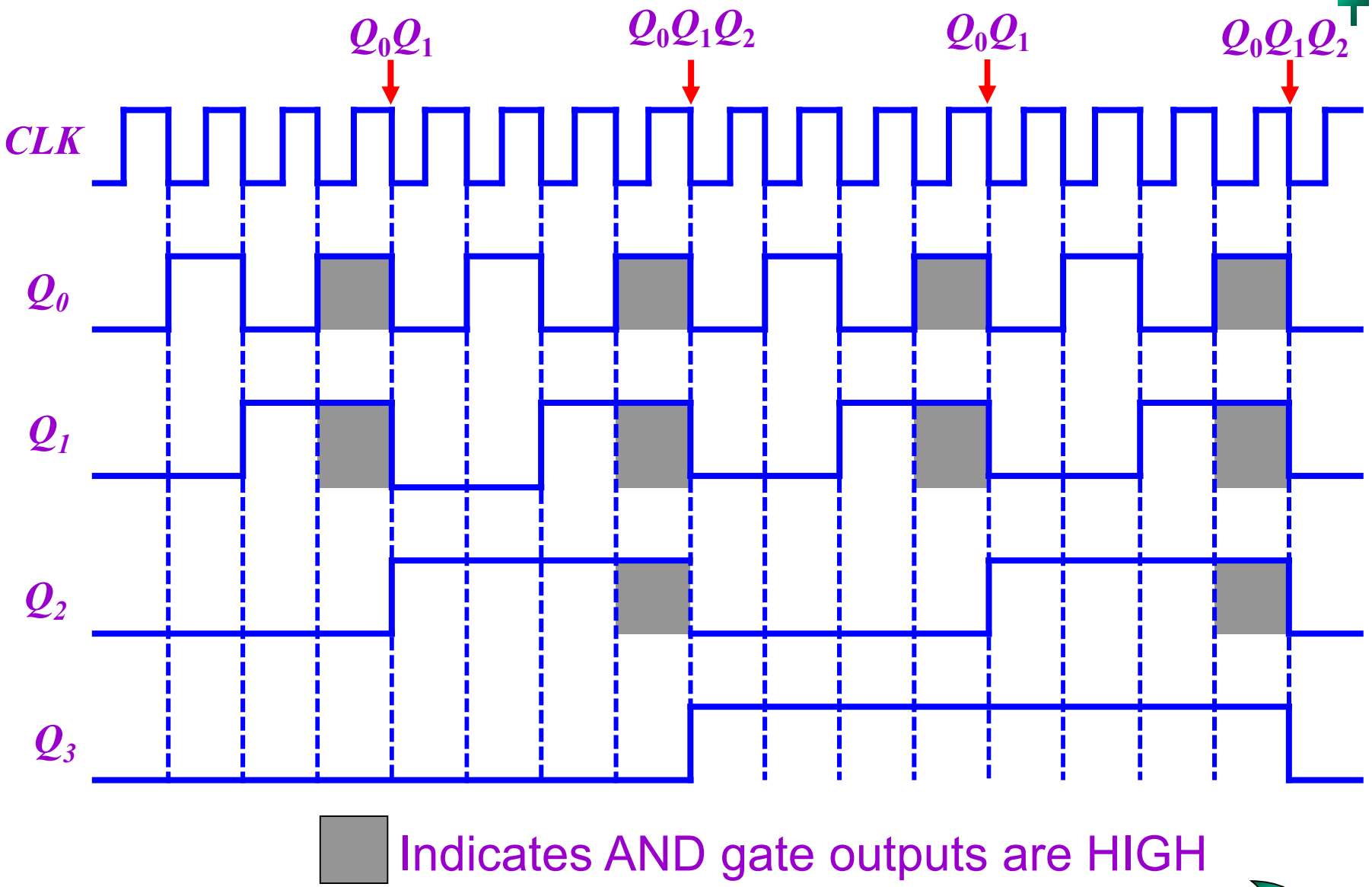
Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

Q_0 toggles every clock cycle;
Q_1 toggles when $Q_0 = 1$;
Q_2 toggles when $Q_0 = Q_1 = 1$;
Q_3 toggles when $Q_0 = Q_1 = Q_2 = 1$;

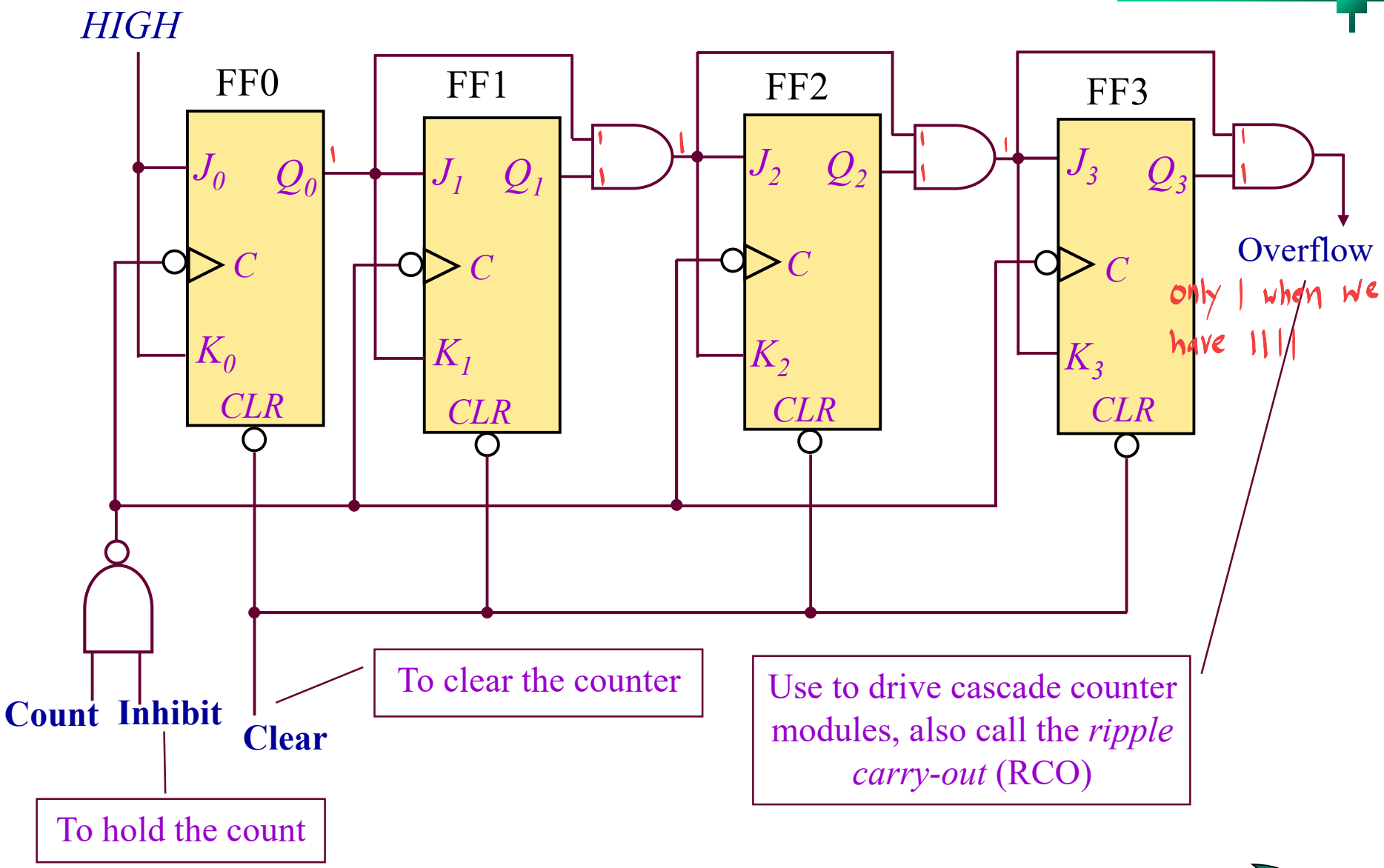


A binary counter of *n* F/Fs will sequence through the number 0, 1, 2, 3, …2^n–1, 0, 1, 2, … and so forth.

Timing Diagram of Synchronous Binary Counter



Adding Control Circuitry



Asynchronous Counters

- Also known as *ripple counters.*
- Asynchronous => the clock inputs of some or all flip-flops are triggered by transitions in other flip-flop outputs instead of a common clock signal.
- Flip-flop within the counter do not change states simultaneously, due to the propagation delay.
- By eliminating the requirement for clock synchronization, a reduced amount of circuitry can be used to implement a binary counter. However, the asynchronous counter suffers from chaining of F/F propagation delays.

3-bit Asynchronous Binary Counter

Clock pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0

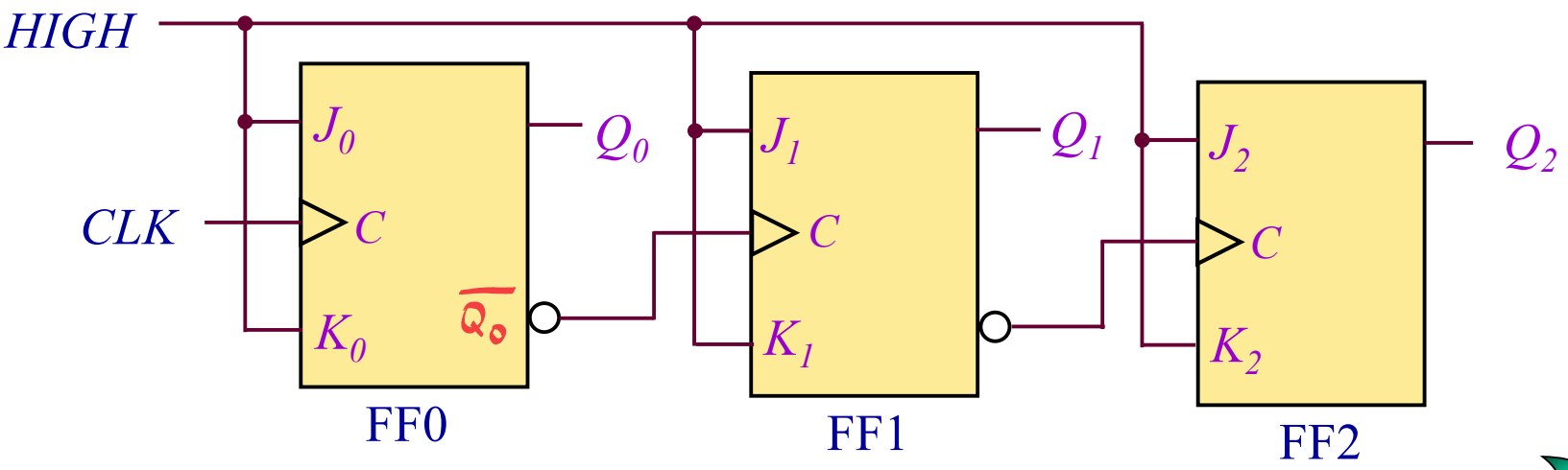
toggles at rising edge

toggle

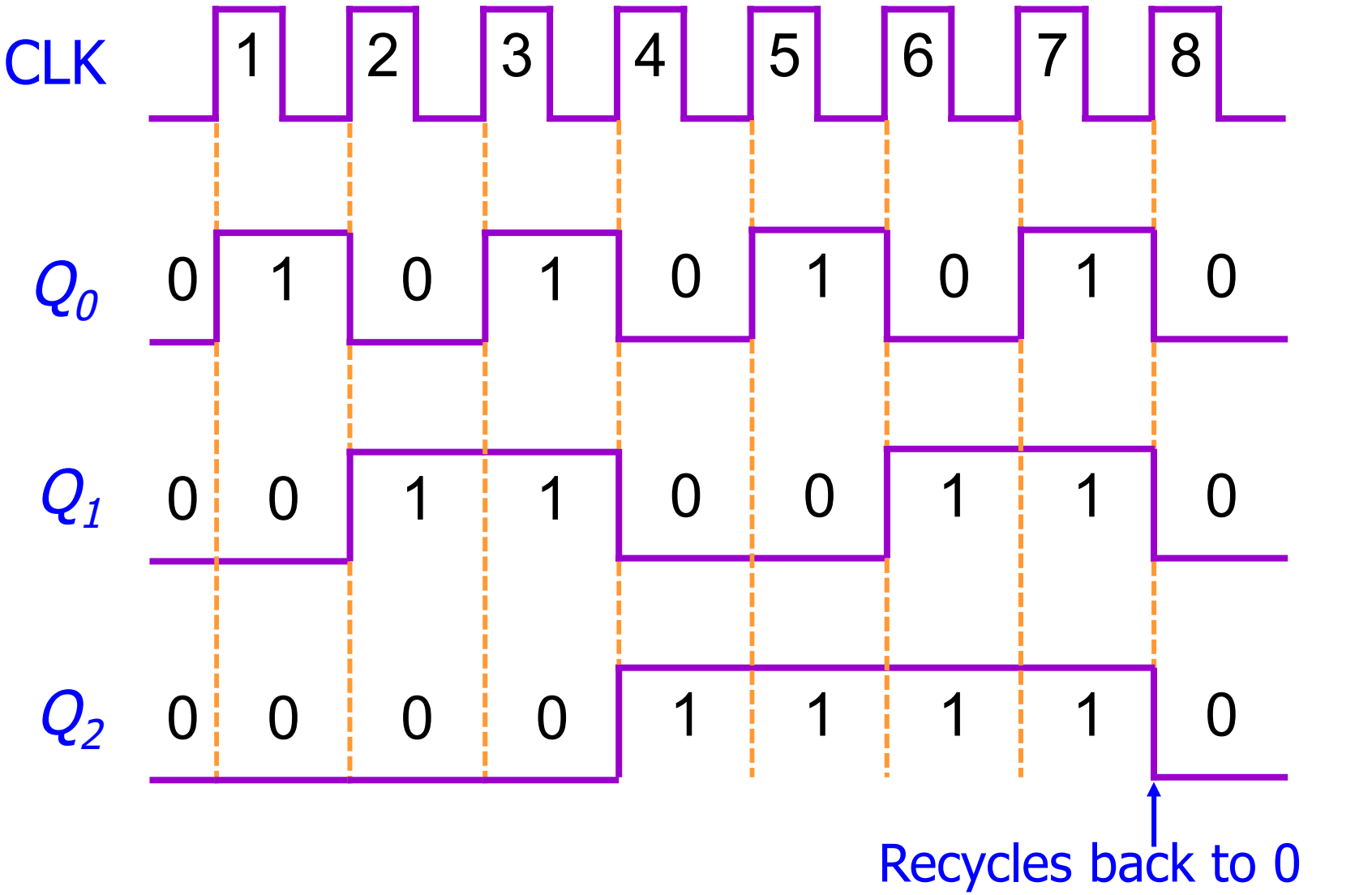
$\overline{Q_0}$

rising edge

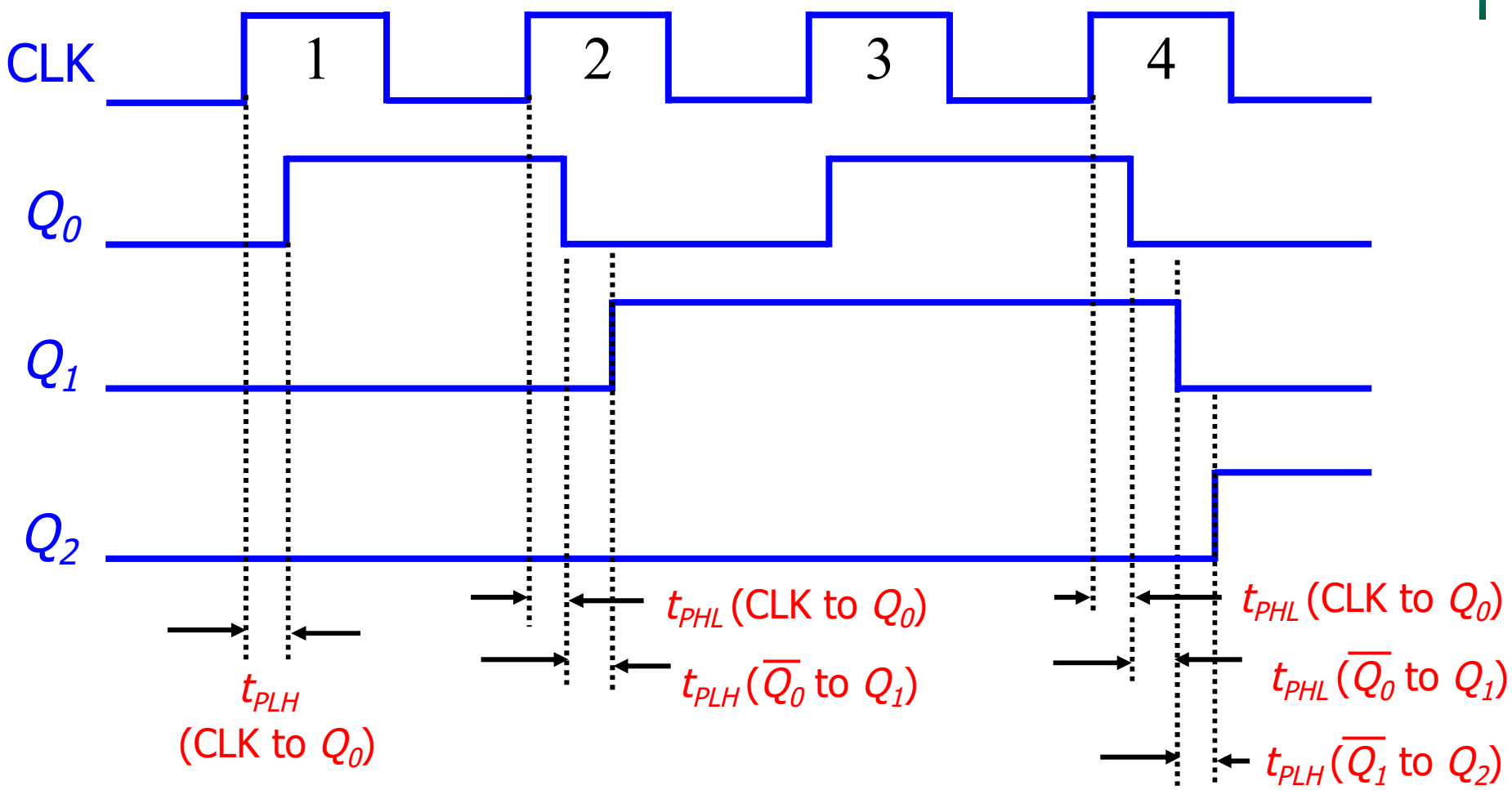
rising edge (toggle when 0 to 1)



Timing Diagram of 3-bit Binary Counter (Ideal)



Propagation Delay of Asynchronous Binary Counter

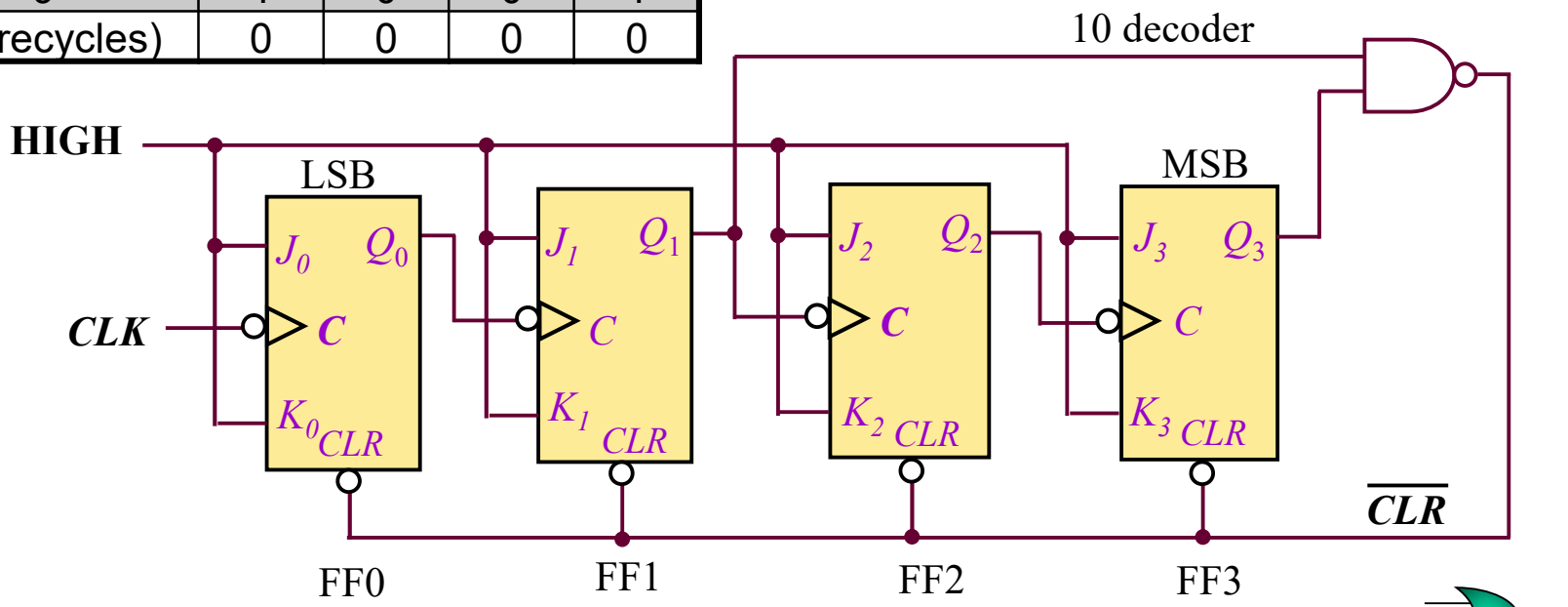


Although the transition is rapid, they can generate unwanted transient condition if counter outputs are used to drive combinational logic circuit.

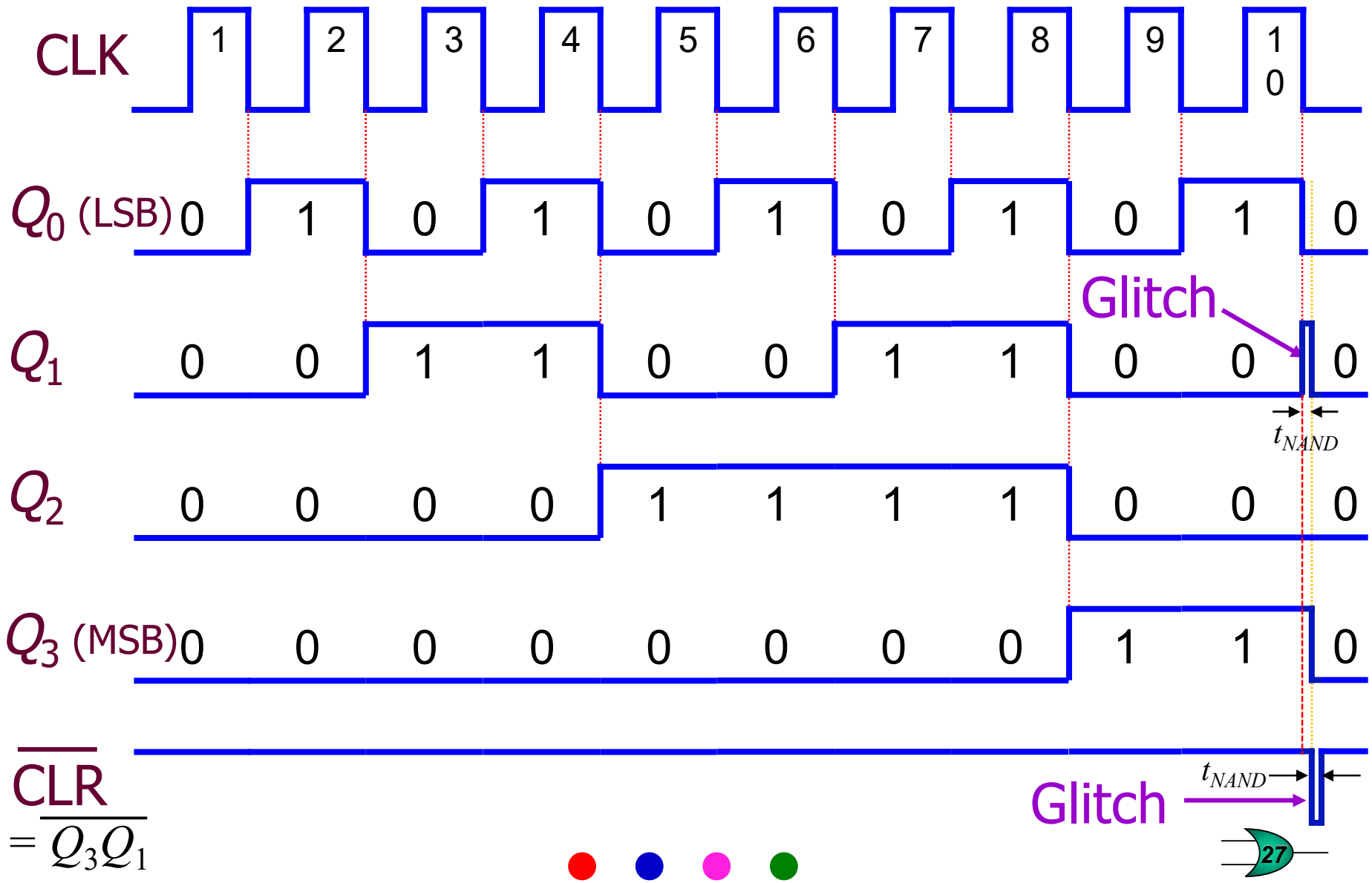
Asynchronous BCD Decade Counter

Clock Pulse	Q_3	Q_2	Q_1	Q_0
Initially	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10 (recycles)	0	0	0	0

With Asynchronous Recycling



Timing Diagram of Asynchronous BCD Counter



Synchronous BCD Decade Counter

Clock Pulse	Q_3	Q_2	Q_1	Q_0	T_3	T_2	T_1	T_0
0	0	0	0	0	0	0	0	1
1	0	0	0	1	0	0	1	1
2	0	0	1	0	0	0	0	1
3	0	0	1	1	0	1	1	1
4	0	1	0	0	0	0	0	1
5	0	1	0	1	0	0	1	1
6	0	1	1	0	0	0	0	1
7	0	1	1	1	1	1	1	1
8	1	0	0	0	0	0	0	1
9	1	0	0	1	1	0	0	1
0	0	0	0	0	0	0	0	1

T F/F implemented by J = K

$T_0 = 1 \Rightarrow J_0 = K_0 = 1$

Q_1Q_0 \ Q_3Q_2	00	01	11	10
00	0	0	X	0
01	1	1	X	0
11	1	1	X	X
10	0	0	X	X

$$J_1 = K_1 = \overline{Q}_3 Q_0$$

Q_1Q_0 \ Q_3Q_2	00	01	11	10
00	0	0	X	0
01	0	0	X	0
11	1	1	X	X
10	0	0	X	X

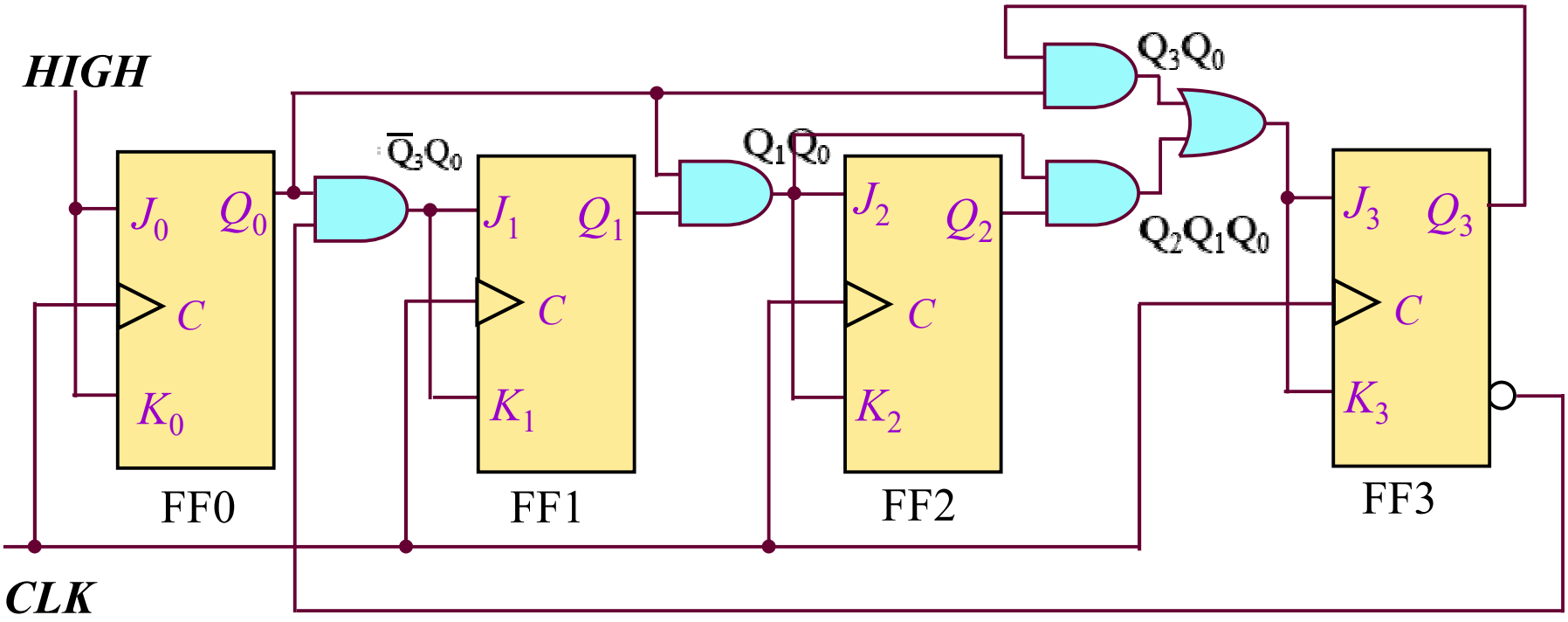
$$J_2 = K_2 = Q_1 Q_0$$

Q_1Q_0 \ Q_3Q_2	00	01	11	10
00	0	0	X	0
01	0	0	X	1
11	0	1	X	X
10	0	0	X	X

$$J_3 = K_3 = Q_3 Q_0 + Q_2 Q_1 Q_0$$

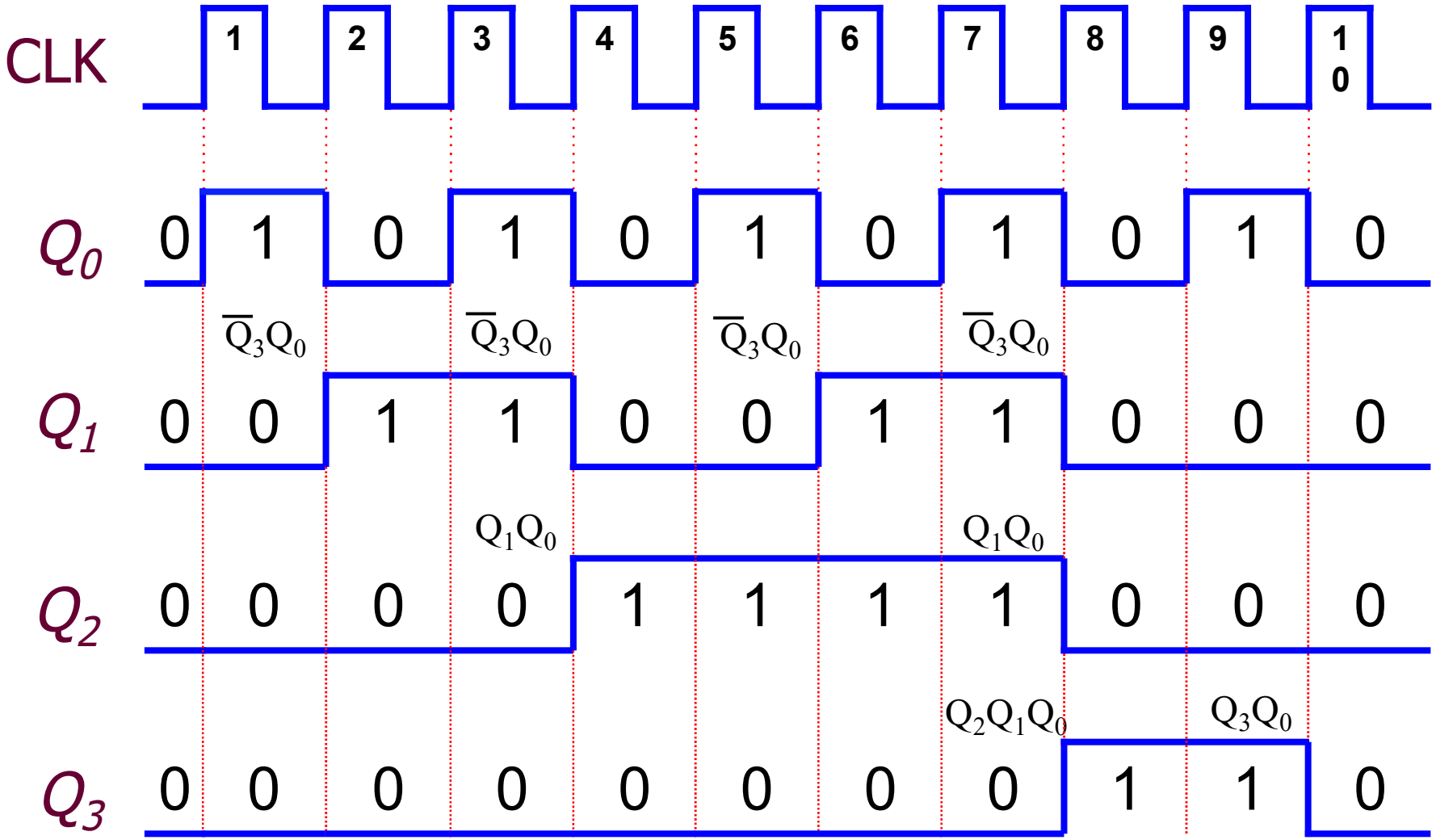
Synchronous BCD Decade Counter

$J_0 = K_0 = 1$ $J_1 = K_1 = \overline{Q}_3 Q_0$ $J_2 = K_2 = Q_1 Q_0$ $J_3 = K_3 = Q_3 Q_0 + Q_2 Q_1 Q_0$

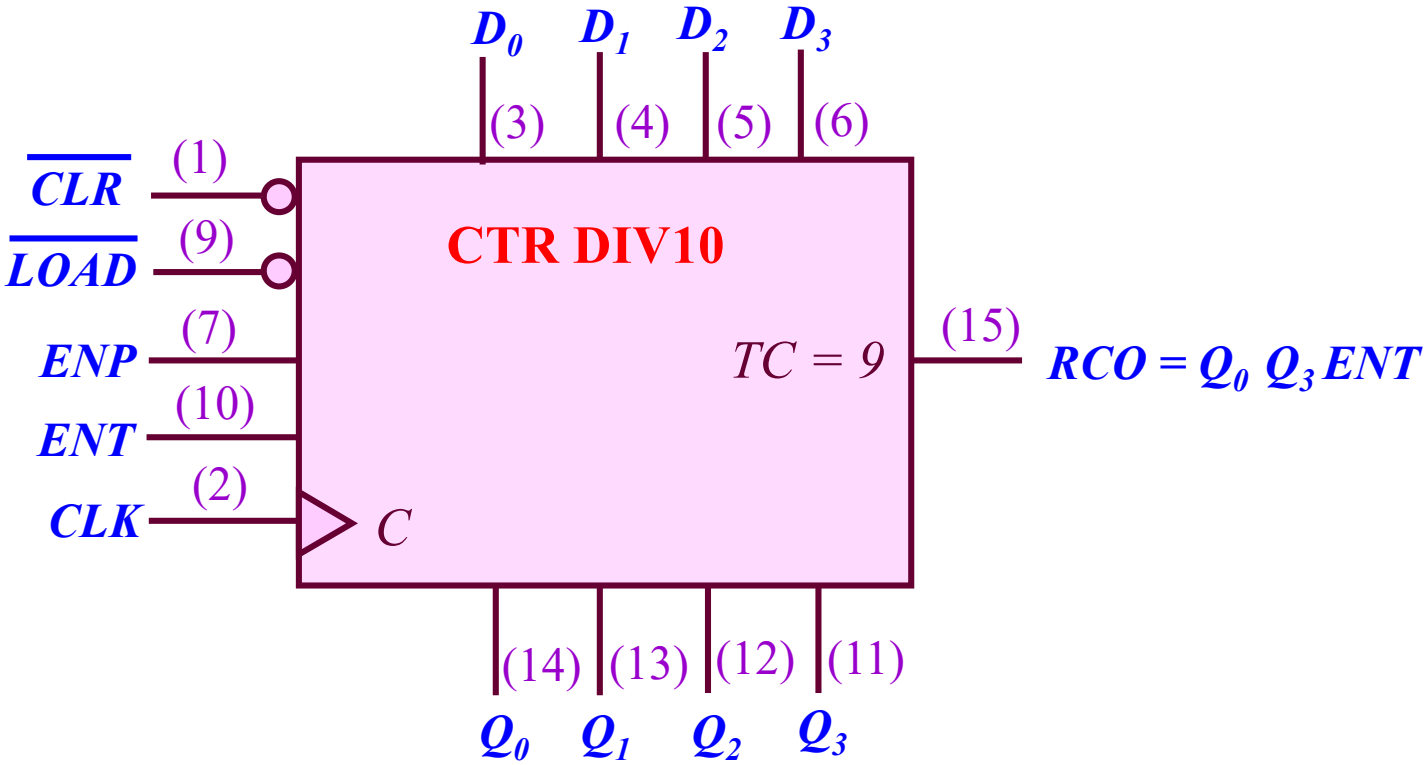


***Q_0* is the LSB**

Timing Diagram of Synchronous BCD Counter



74LS160 4-bit Synchronous BCD Counter



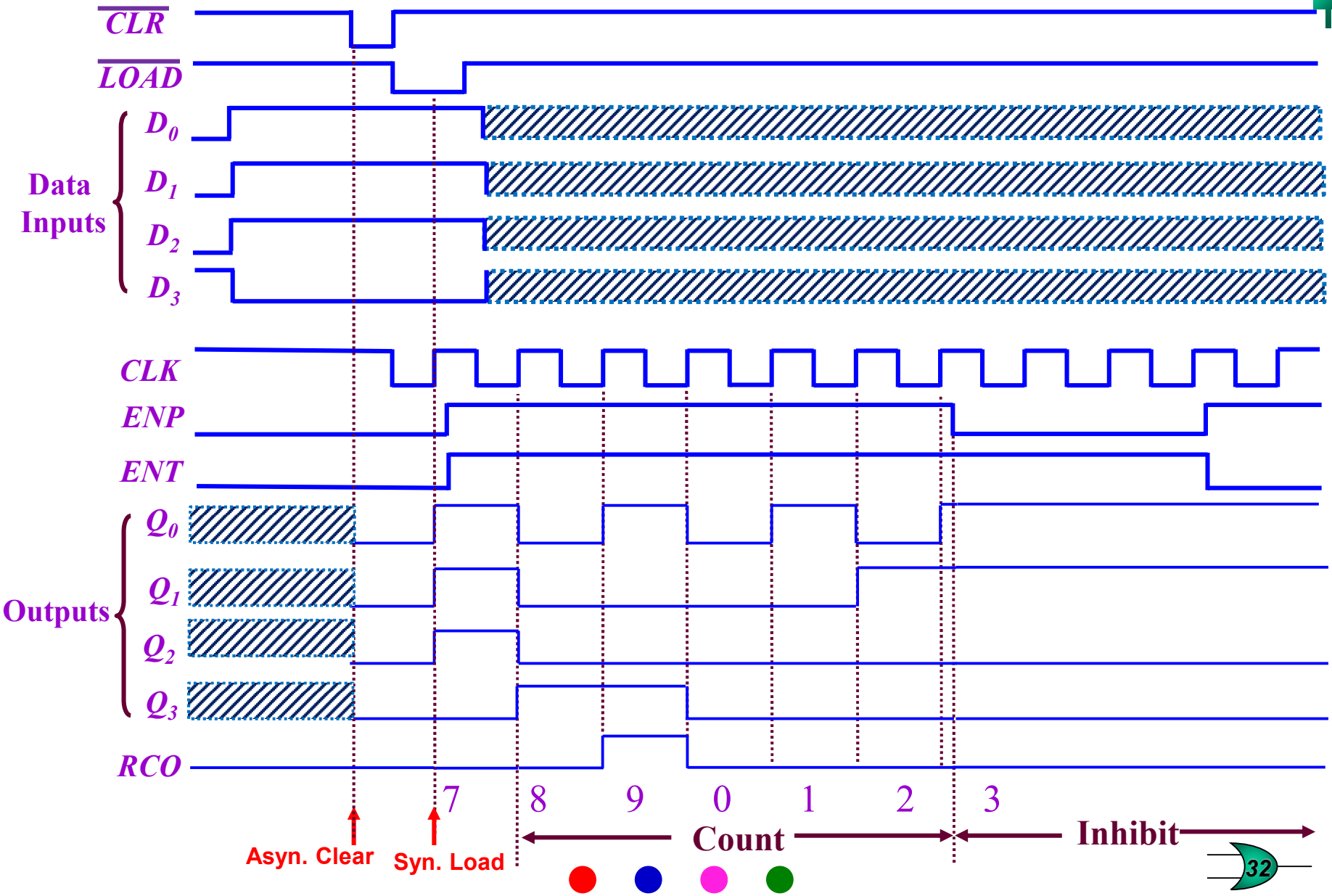
When $\overline{\text{LOAD}}$ = '0', data *D* will be **synchronously** loaded into the counter and will appear at the outputs *Q*.

When $\overline{\text{LOAD}}$ = '1', **synchronous** count mode. Both ENP (Enable Parallel) and ENT (Enable Trickle) must be '1' before the counter will cycle through its states.

Either ENP or ENT may be brought low to inhibit the counter (data-hold mode).

RCO is high during the period of time in which the counter is in state 9, its maximum state.

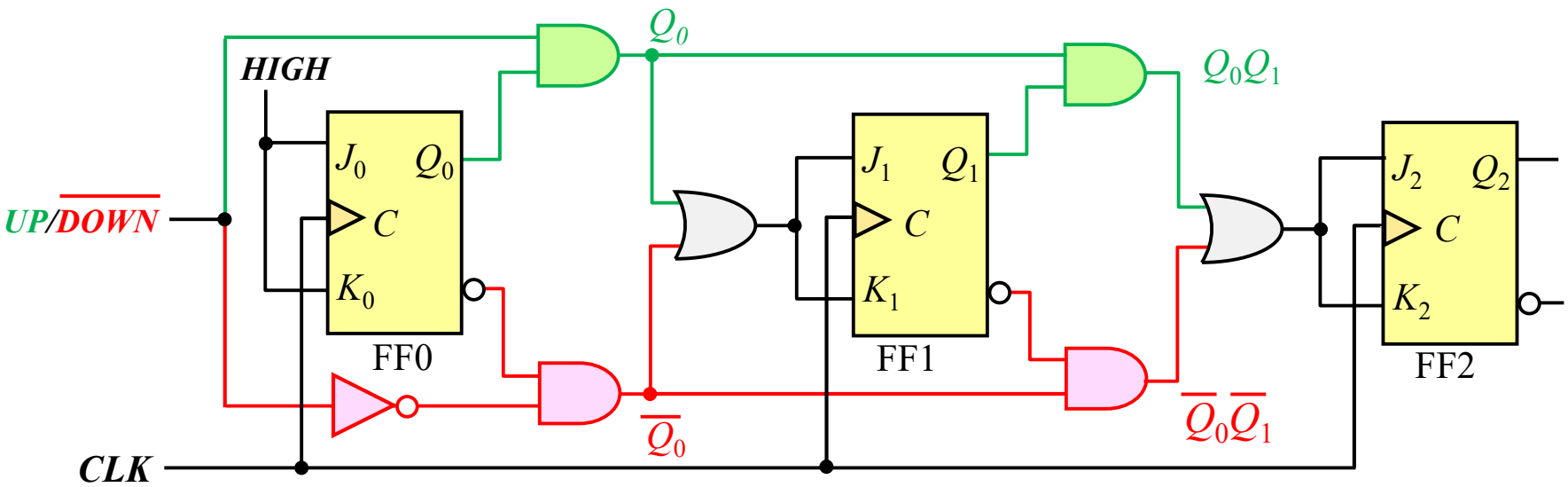
Timing Diagram of 74LS160



Synchronous Up/Down Counter

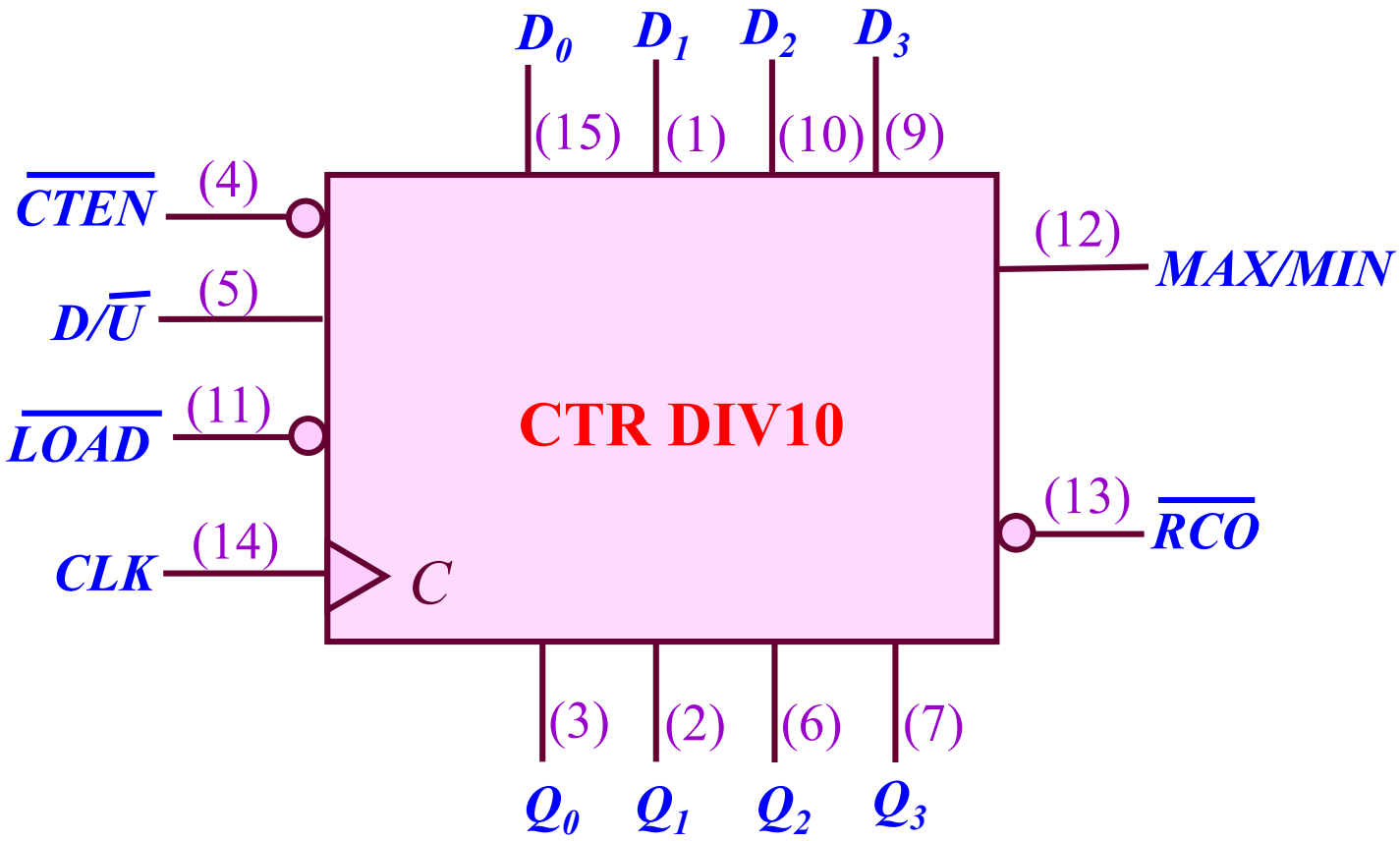
Cycle	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

Cycle	Q_2	Q_1	Q_0
0	1	1	1
1	1	1	0
2	1	0	1
3	1	0	0
4	0	1	1
5	0	1	0
6	0	0	1
7	0	0	0
8	1	1	1



For *UP* mode, the *Q* outputs of the F/Fs control J and K terminals of the F/Fs higher in the chain.

74LS190 Up/Down Syn Decade Counter

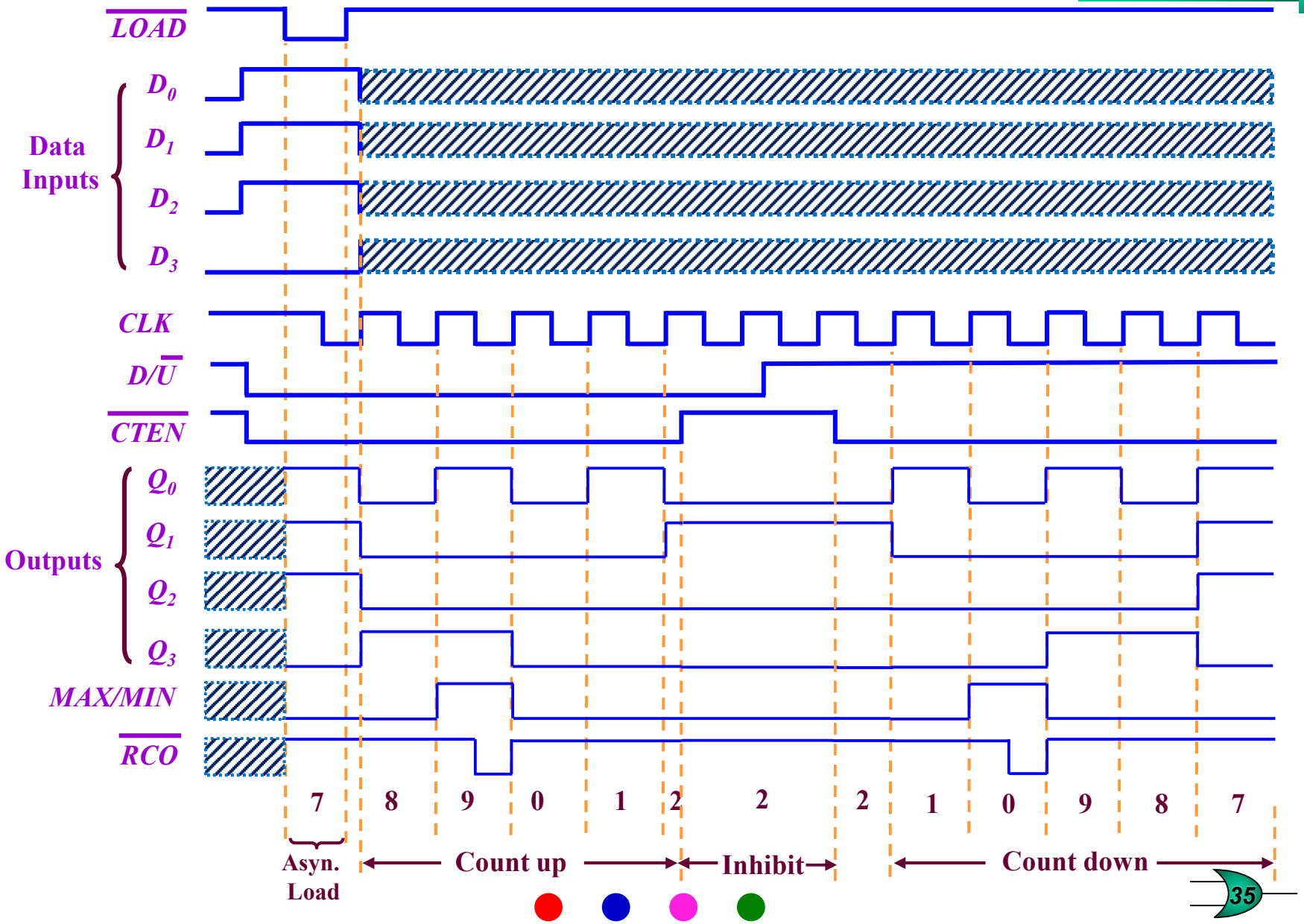


$\overline{CTEN}$: Active low count enable.

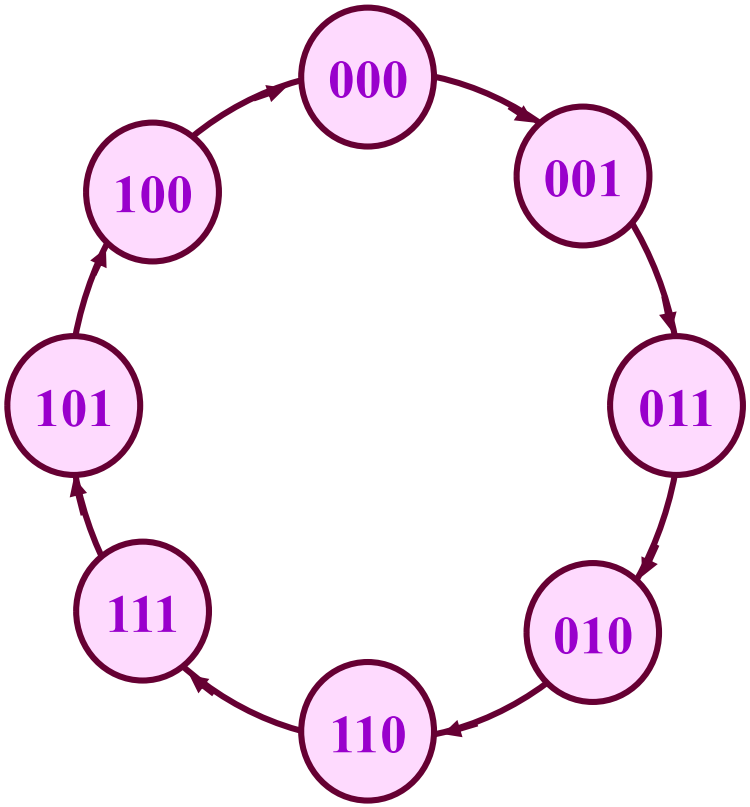
$\overline{Load}$ is asynchronous.

$\overline{RCO}$ is asserted low on falling edge of *CLK* when *MAX/MIN* is asserted high.

74LS190 Up/down Synchronous Decade Counter



Design a 3-bit Gray Code Counter



Present State			Next State			F/F Input		
Q_2	Q_1	Q_0	Q_2^*	Q_1^*	Q_0^*	J_2K_2	J_1K_1	J_0K_0
0	0	0	0	0	1	0 X	0 X	1 X
0	0	1	0	1	1	0 X	1 X	X 0
0	1	0	1	1	0	1 X	X 0	0 X
0	1	1	0	1	0	0 X	X 0	X 1
1	0	0	0	0	0	X 1	0 X	0 X
1	0	1	1	0	0	X 0	0 X	X 1
1	1	0	1	1	1	X 0	X 0	1 X
1	1	1	1	0	1	X 0	X 1	X 0

K-Maps of J and K Inputs

Q_2Q_1 \ Q_0	0	1
00	0	0
01	1	0
11	X	X
10	X	X

$Q_1\overline{Q_0}$

J_2 map

Q_2Q_1 \ Q_0	0	1
00	0	1
01	X	X
11	X	X
10	0	0

$\overline{Q_2}Q_0$

J_1 map

Q_2Q_1 \ Q_0	0	1
00	1	X
01	0	X
11	1	X
10	0	X

$\overline{Q_2}\overline{Q_1}$

Q_2Q_1

J_0 map

Q_2Q_1 \ Q_0	0	1
00	X	X
01	X	X
11	0	0
10	1	0

$\overline{Q_1}\overline{Q_0}$

K_2 map

Q_2Q_1 \ Q_0	0	1
00	X	X
01	0	0
11	0	1
10	X	X

Q_2Q_0

K_1 map

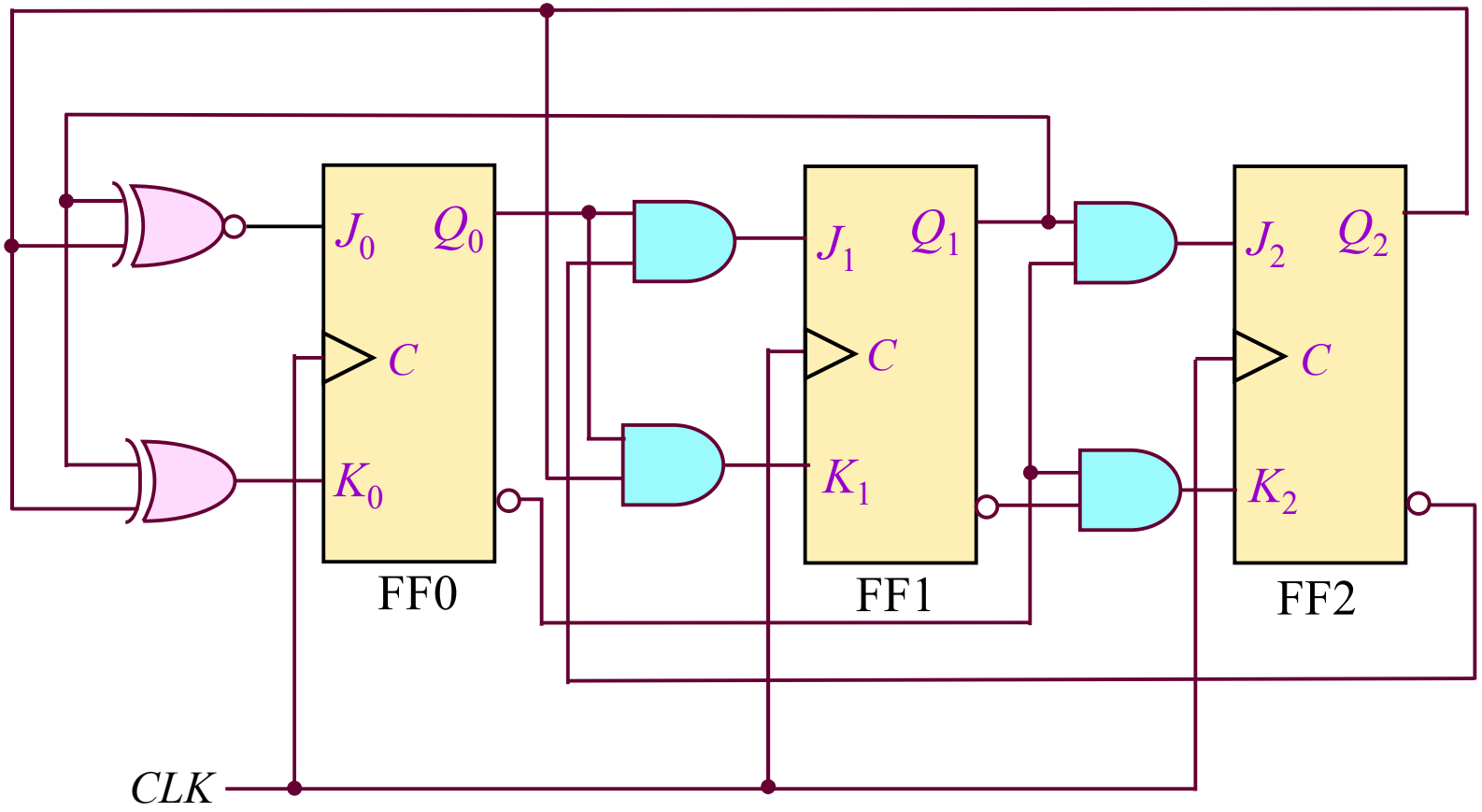
Q_2Q_1 \ Q_0	0	1
00	X	0
01	X	1
11	X	0
10	X	1

$\overline{Q_2}Q_1$

$Q_2\overline{Q_1}$

K_0 map

3-bit Gray Code Counter



$$J_0 = Q_2Q_1 + \overline{Q}_2\overline{Q}_1 = \overline{Q_2 \oplus Q_1} \qquad J_1 = \overline{Q}_2Q_0 \qquad J_2 = Q_1\overline{Q}_0$$

$$K_0 = Q_2\overline{Q}_1 + \overline{Q}_2Q_1 = Q_2 \oplus Q_1 \qquad K_1 = Q_2Q_0 \qquad K_2 = \overline{Q}_1\overline{Q}_0$$

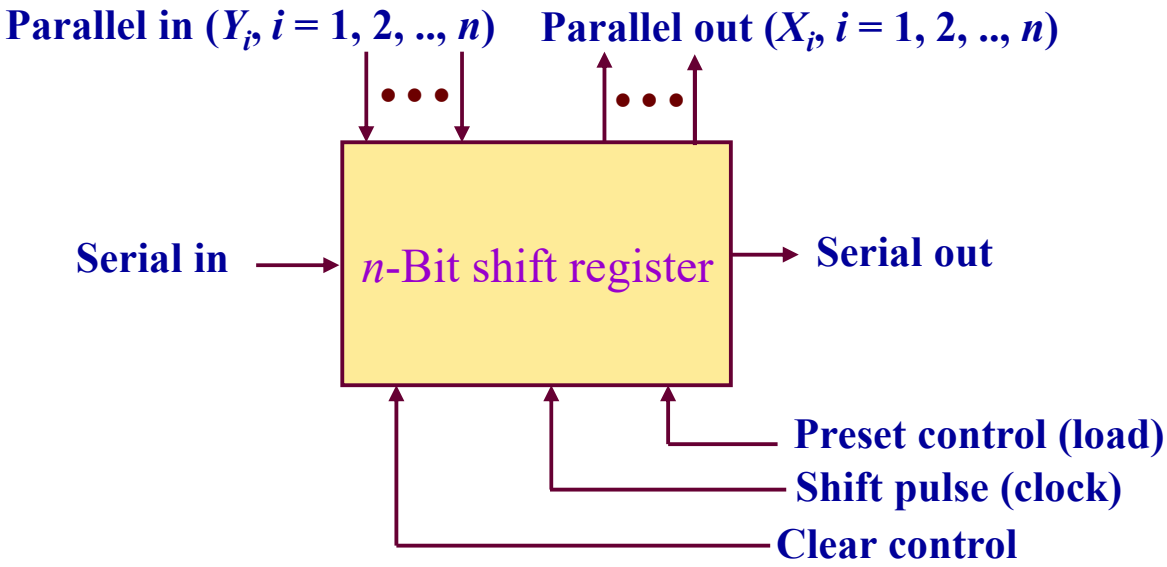
Topic : Sequential Circuits

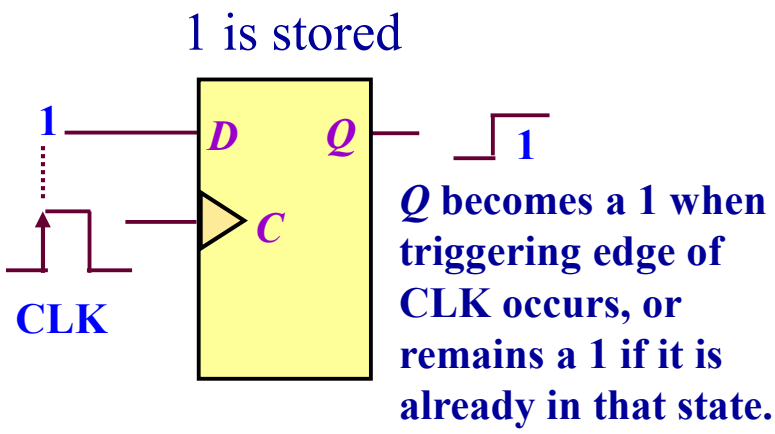
- Latches & flip-flops
- Synchronous state machine analysis
- Synchronous state machine design
- Timing parameters
- Counters
- Shift registers
- Memory

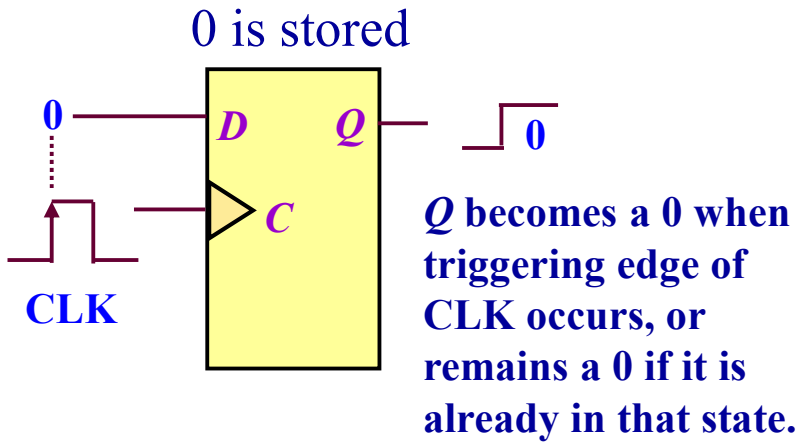
Shift Registers

- Sequential logic modules constructed from flip-flops that manipulate bit positions of binary data by shifting the data bits to the left or right.
- Unlike counters, shift registers have no specified sequence of states.
- The **storage capacity** of a shift register is the number of bits of digital data it can retain. Each flip-flop in a shift register represents one bit of storage capacity.
- Shift registers are used for storing and shifting data entered into it from an external source. The shifting capability allows movement of data from stage to stage within the register or into or out of the register upon application of clock pulses.

Universal Shift Register

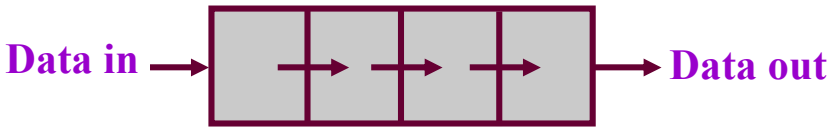




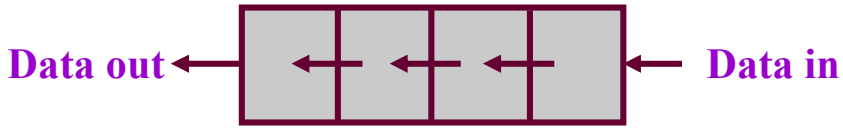


Flip-flop is used as a storage element

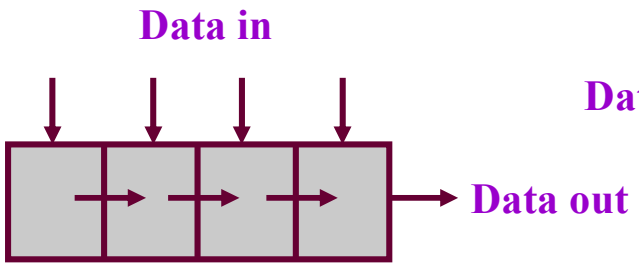
Data Movement in Shift Registers



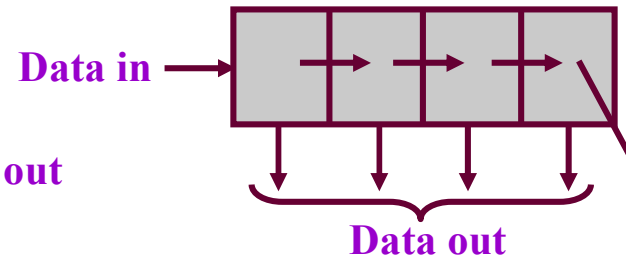
Serial in/shift right/serial out



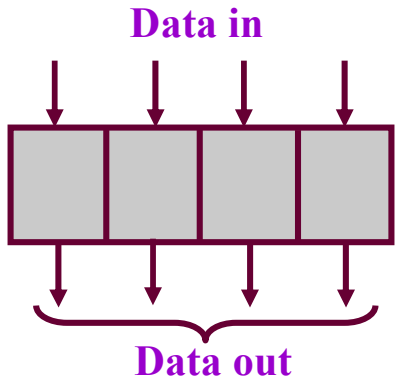
Serial in/shift left/serial out



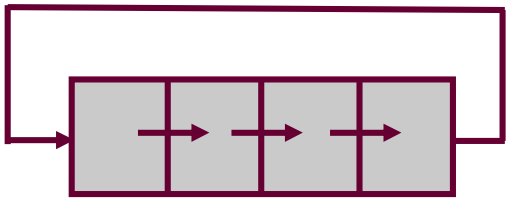
Parallel in/serial out



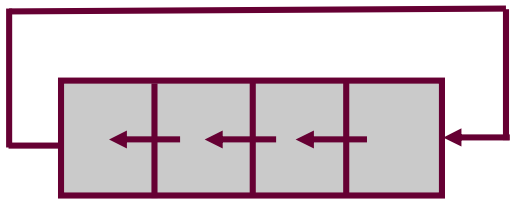
Serial in/parallel out



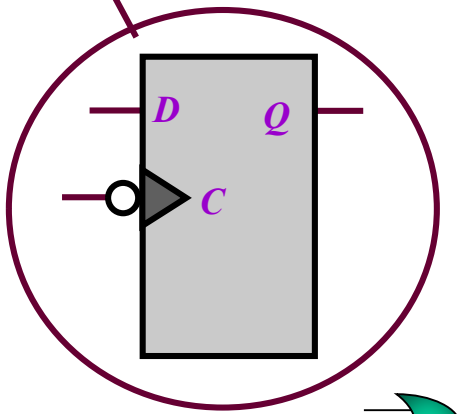
Parallel in/parallel out



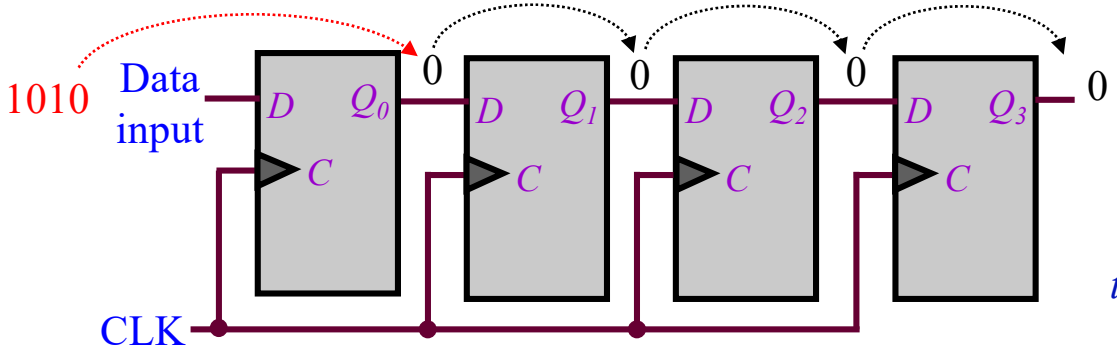
Rotate right



Rotate left

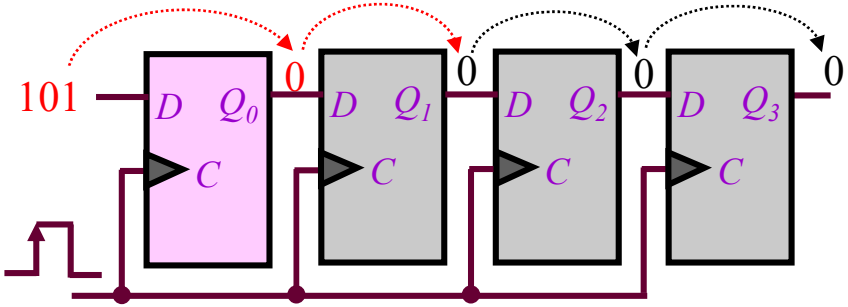


Serial-in/Serial-out With Bit Stream 1010

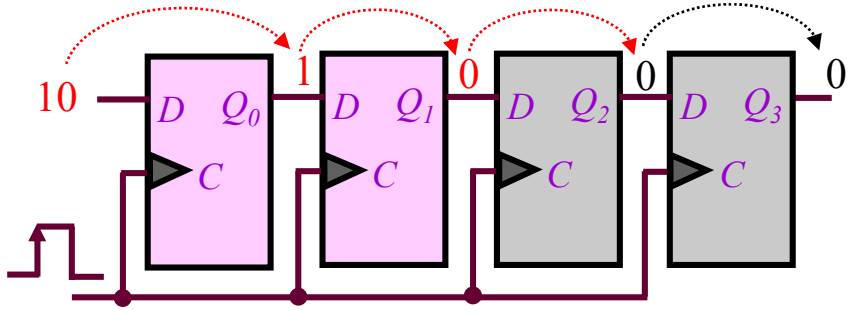


Register initially CLEAR
Example: input 1010

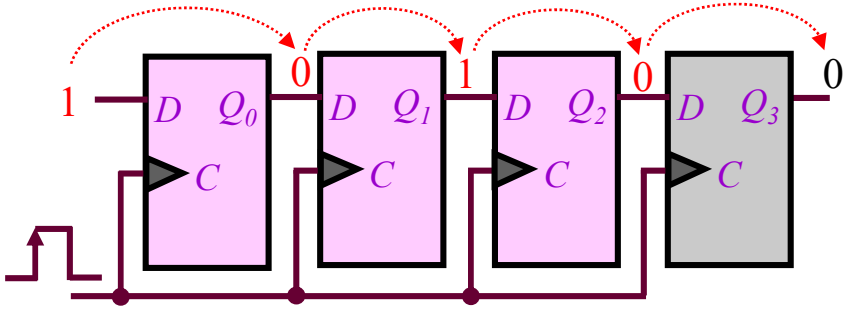
t_{pff}: Flip-flop propagation delay



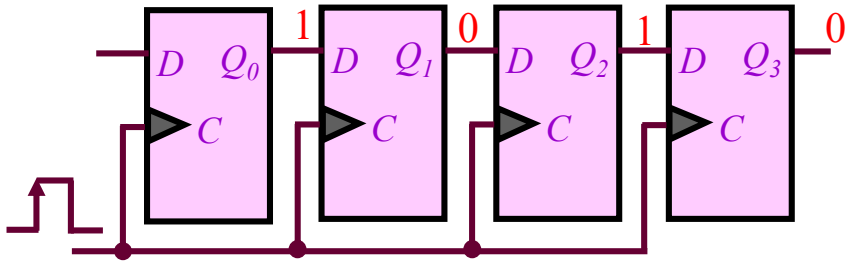
t_{pff} after CLK 1 rising edge, first bit shifted in



t_{pff} after CLK 2 rising edge, second bit shifted in

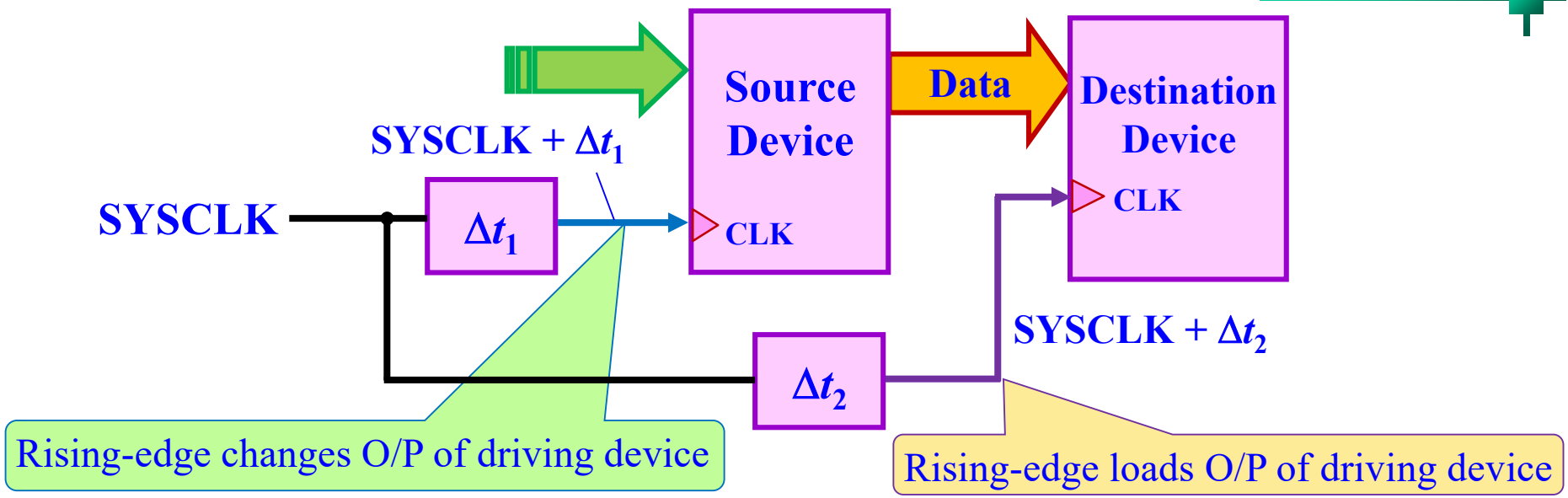


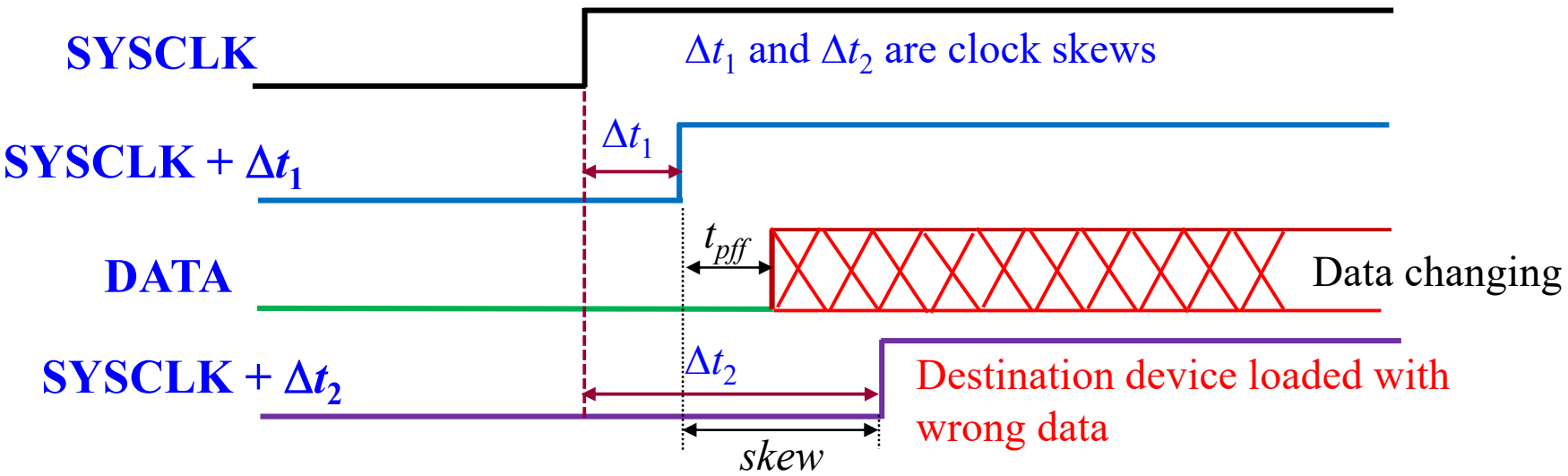
t_{pff} after CLK 3 rising edge, third bit shifted in



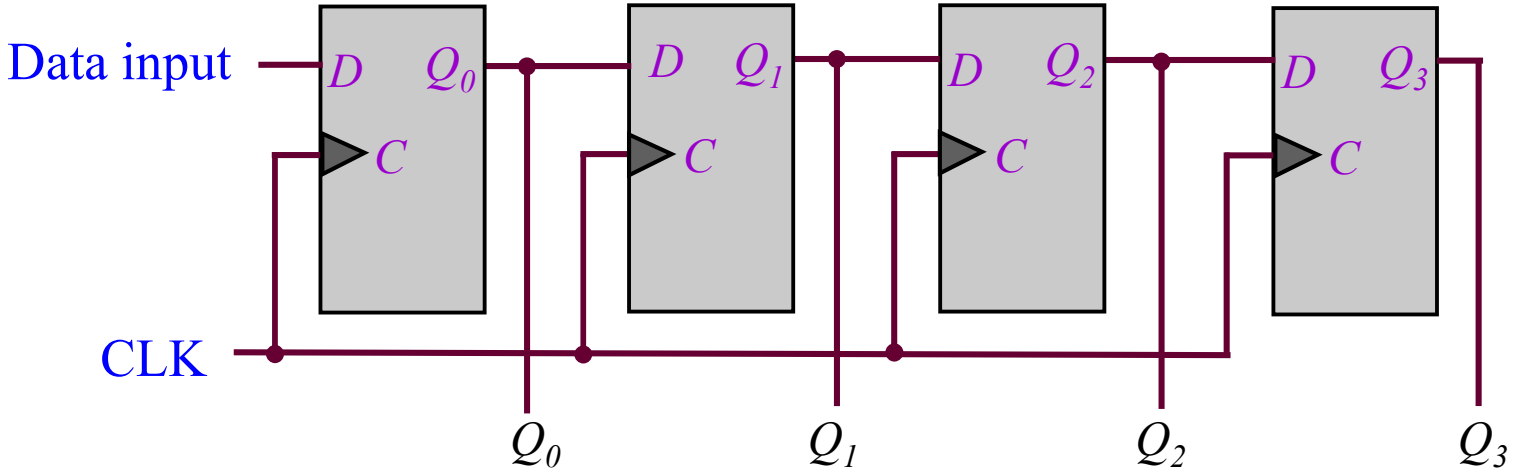
t_{pff} after CLK 4 rising edge,
the 4-bit number is completely stored in register

Effect of Clock Skew on Shift Register

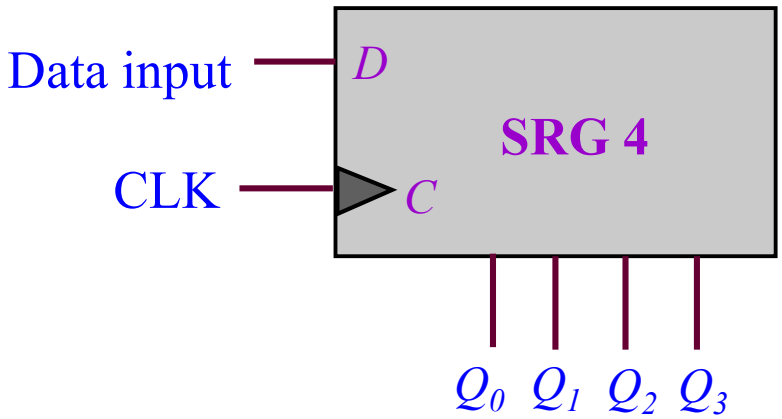




A 4-bit Serial-in/Parallel-out Shift Register

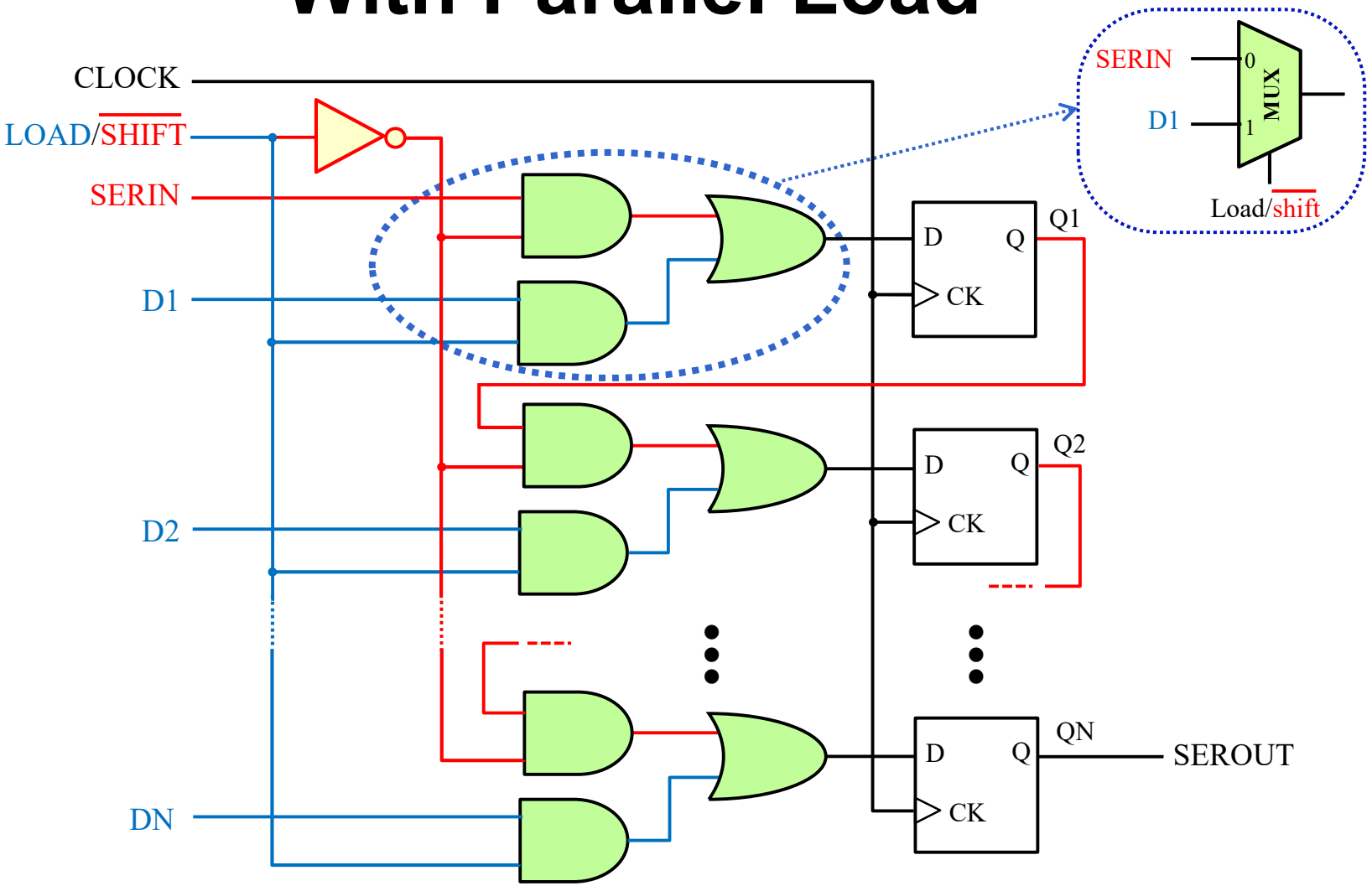


Logic Diagram

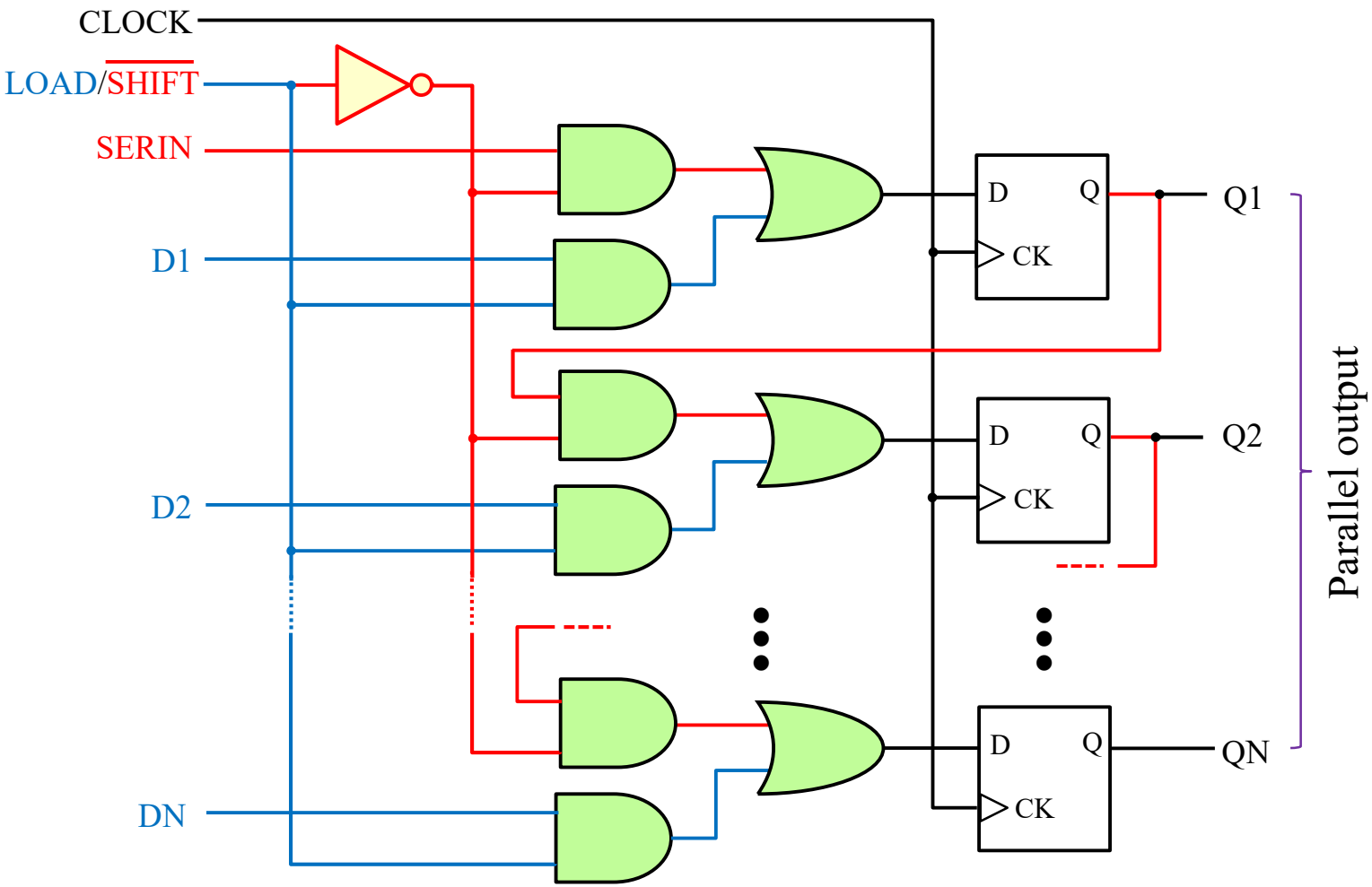


Logic Symbol

Serial-in, Serial-out Shift Register With Parallel Load



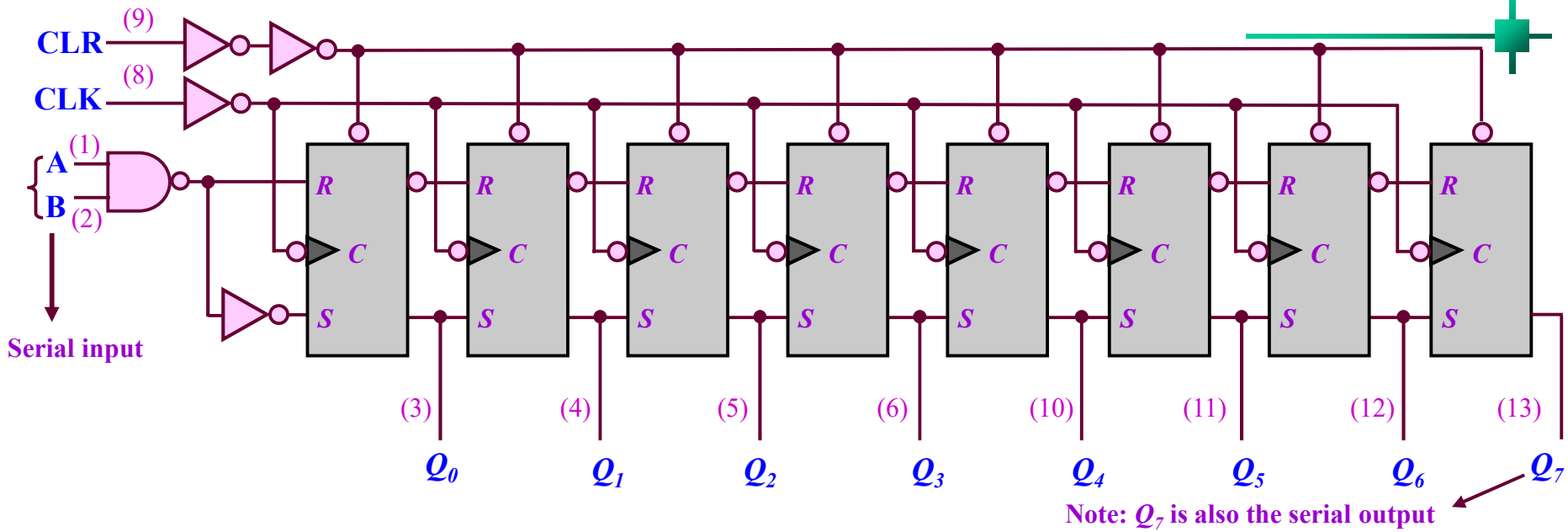
Serial-in, Parallel-out Shift Register With Parallel Load

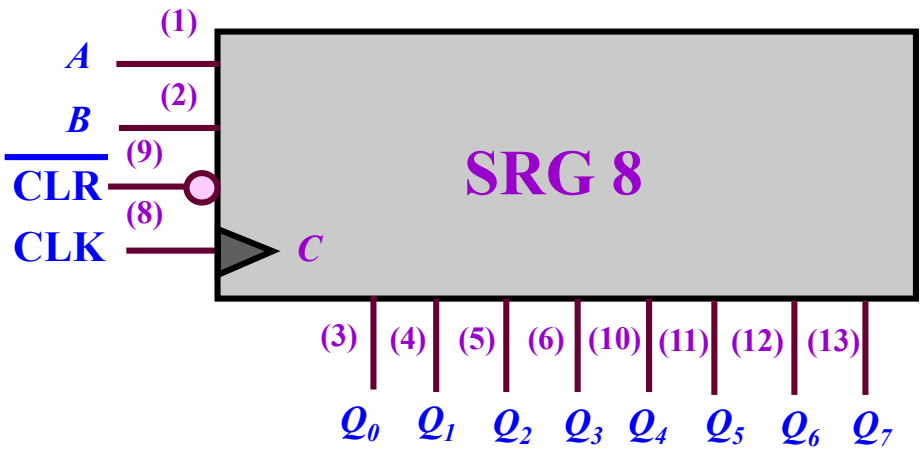


Standard Shift Register IC Modules

Device	Features
7491A	8-bit, serial in, serial out
7496	5-bit, serial in, serial out, asynchronous preset, parallel out, common clear
74164	8-bit, serial in, serial out or parallel out, common clear
74165	8-bit, serial in, serial out, asynchronous load, clock inhibit
74179	4-bit, serial in, serial out, common clear, synchronous load, parallel out, synchronous data hold
74194	4-bit, bidirectional, serial in, serial out, synchronous load, parallel out, clock inhibit, common clear

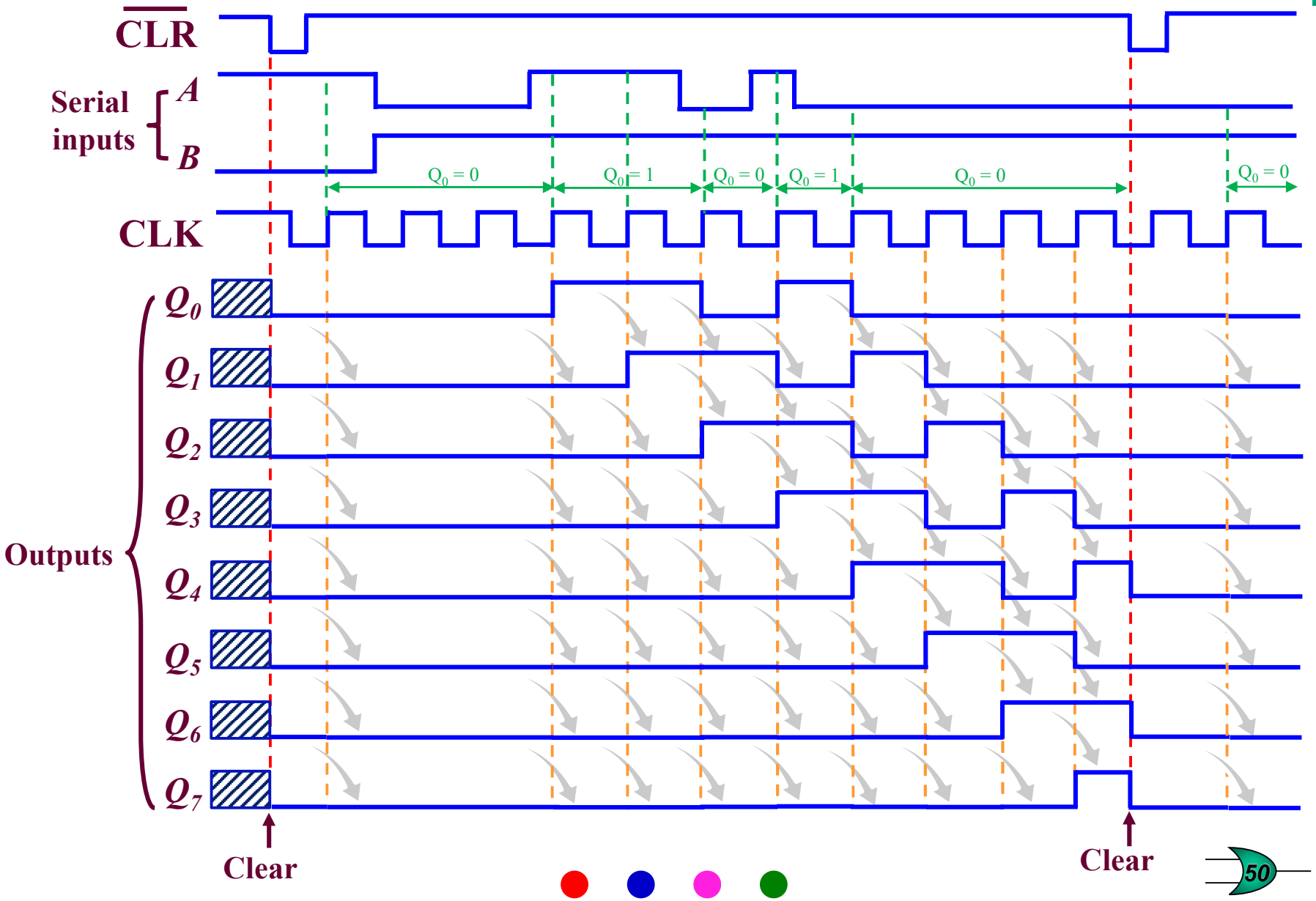
74164 8-bit Serial-in/Parallel-out





Inputs			Outputs
$\overline{CLR}$	CLK	A B	$Q_0(t)$ $Q_1(t)$ … $Q_7(t)$
L	X	X X	L L … L
H	L, H, ↓	X X	$Q_0(t\text{-})$ $Q_1(t\text{-})$ … $Q_7(t\text{-})$
H	↑	H H	H $Q_0(t\text{-})$ … $Q_6(t\text{-})$
H	↑	L X	L $Q_0(t\text{-})$ … $Q_6(t\text{-})$
H	↑	X L	L $Q_0(t\text{-})$ … $Q_6(t\text{-})$

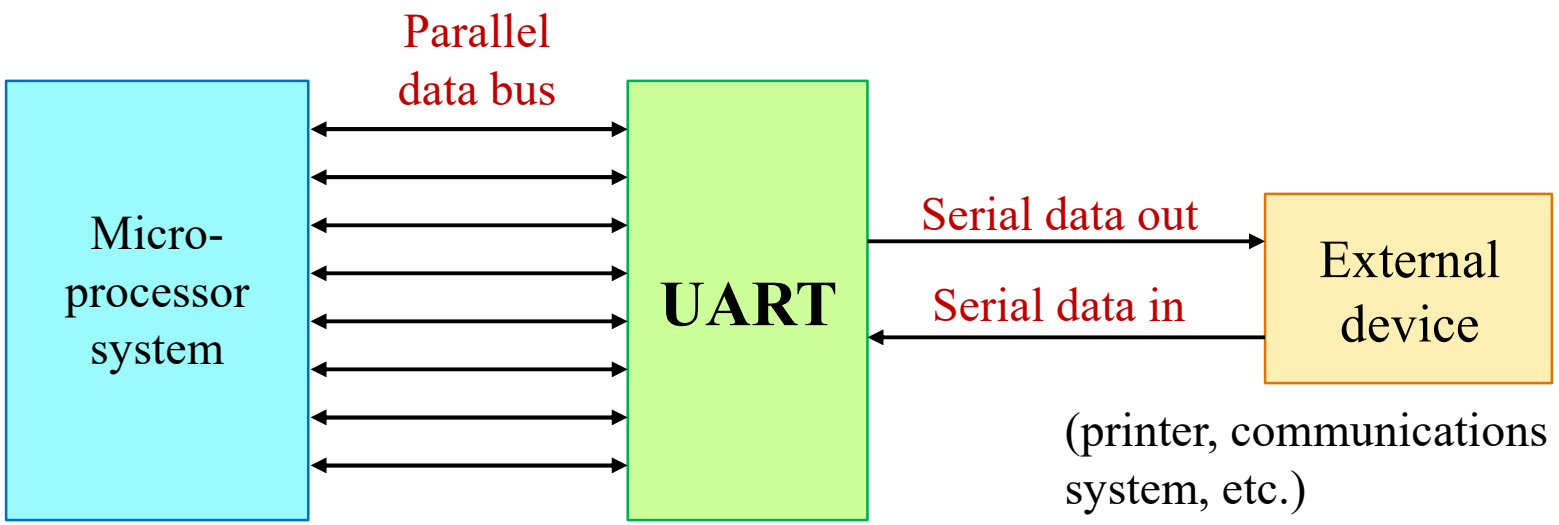
Sample Timing Diagram for 74164



Shift Register Applications

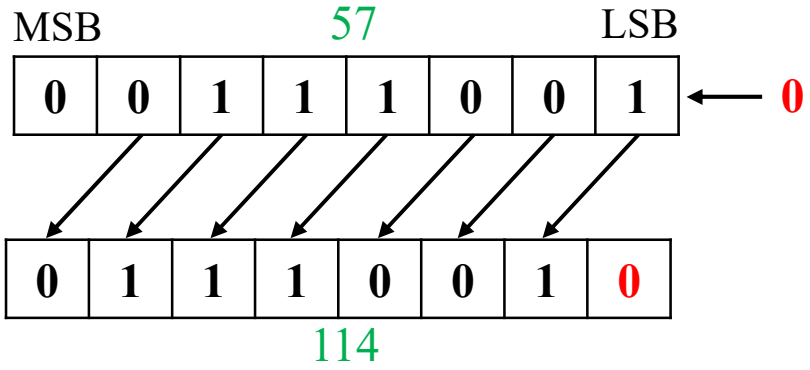
- **Parallel-to-serial conversion for serial transmission**

  Computers and microprocessors-based systems often send and receive data in a parallel format. These systems must also communicate with external devices that send and/or receive serial data. An interface device that accomplishes these conversions is the Universal Asynchronous Receiver Transmitter (UART).

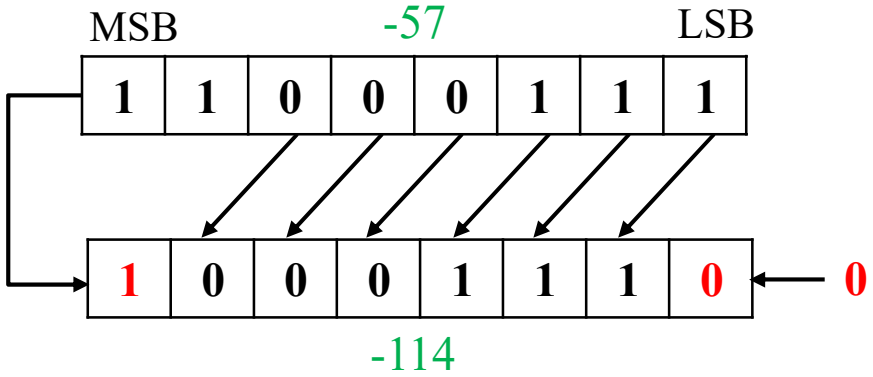


Shift Register Applications

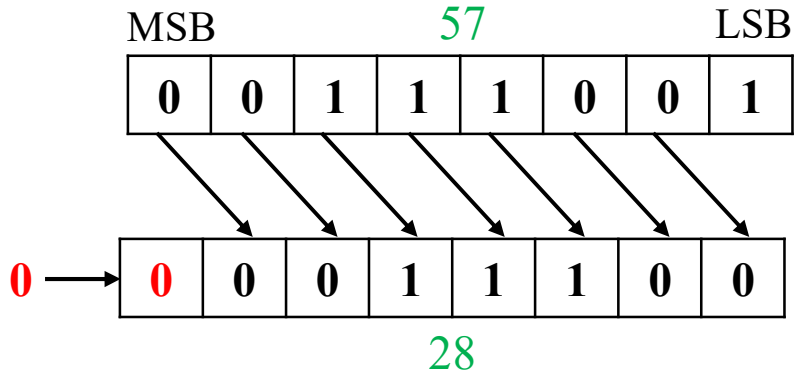
Logical left shift ⇒ multiply by an unsigned number by two



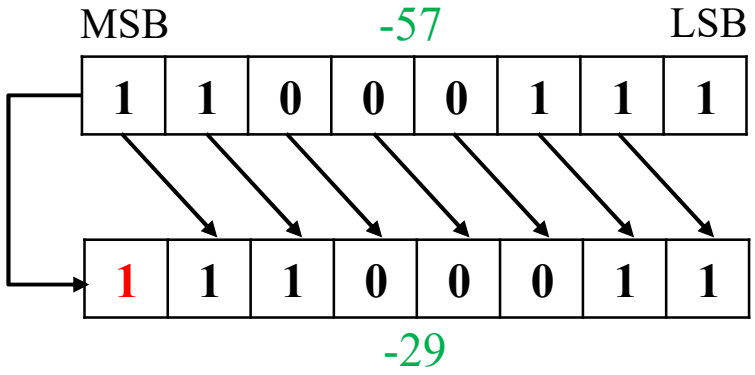
Arithmetic left shift ⇒ multiply by a signed number by two.



Logical right shift ⇒ divide an unsigned number by two with rounding.

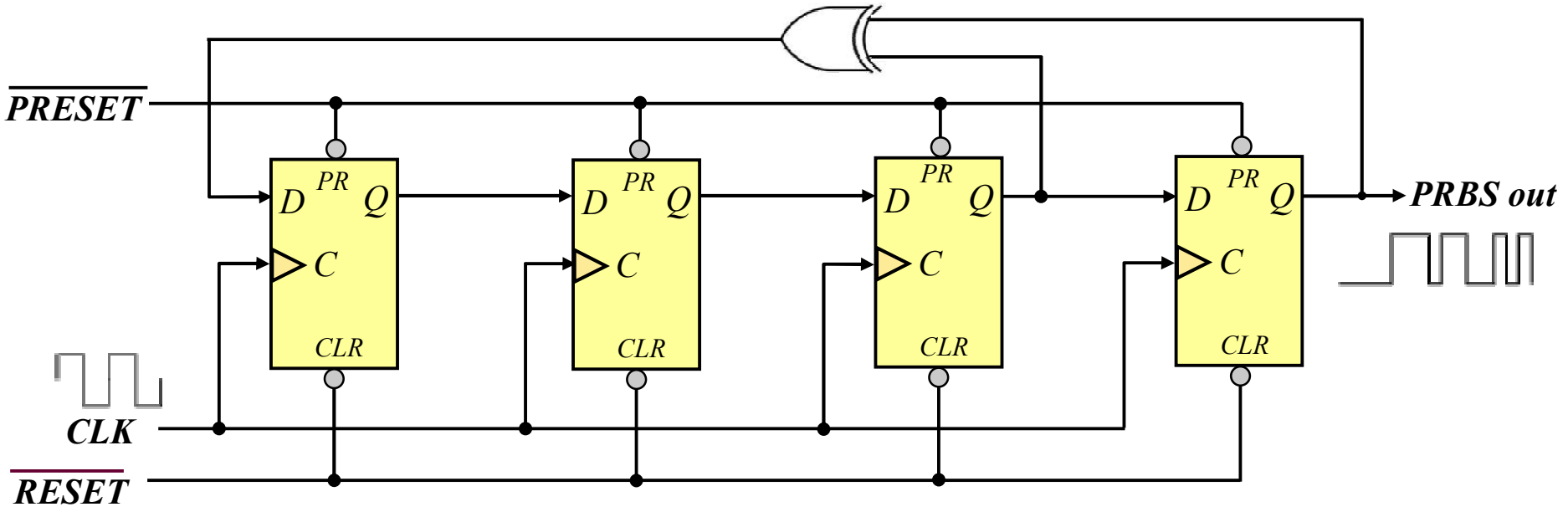


Arithmetic right shift ⇒ divide a signed number by two with rounding.



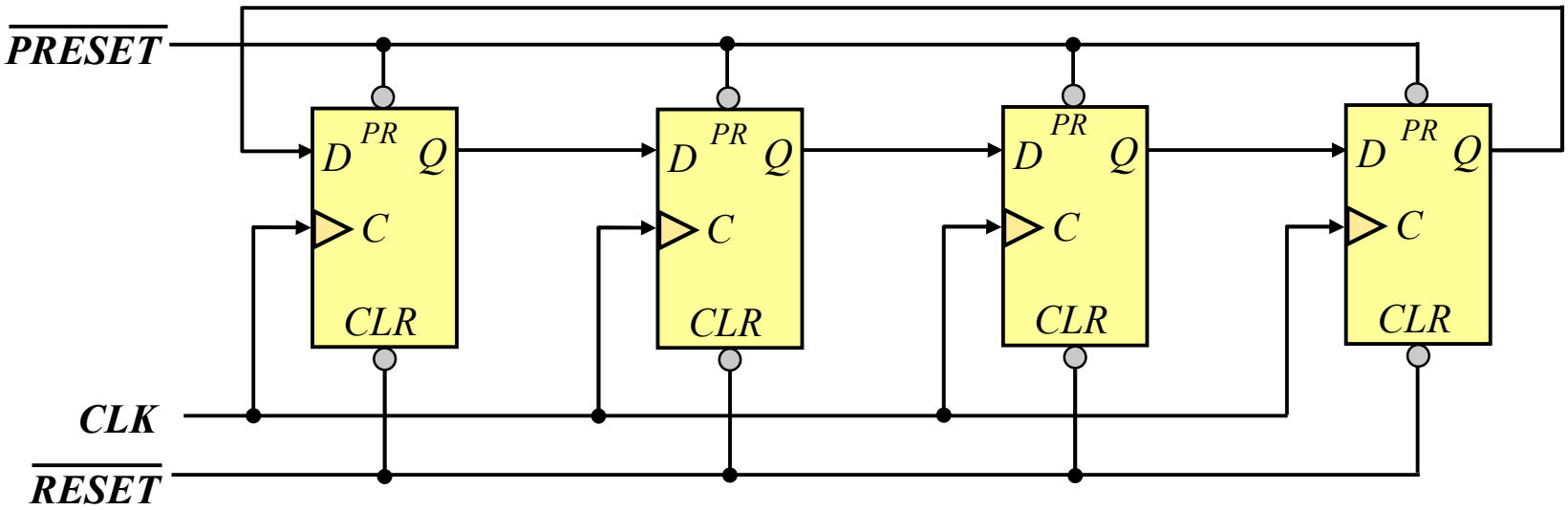
Shift Register Applications

- **Pseudo-random Pattern Generator:** the vacant bit is filled with the xor of the last two bits in the shift register.
- An *n*-bit shift register can generate $2^n - 1$ patterns.
- Useful in test pattern generation.

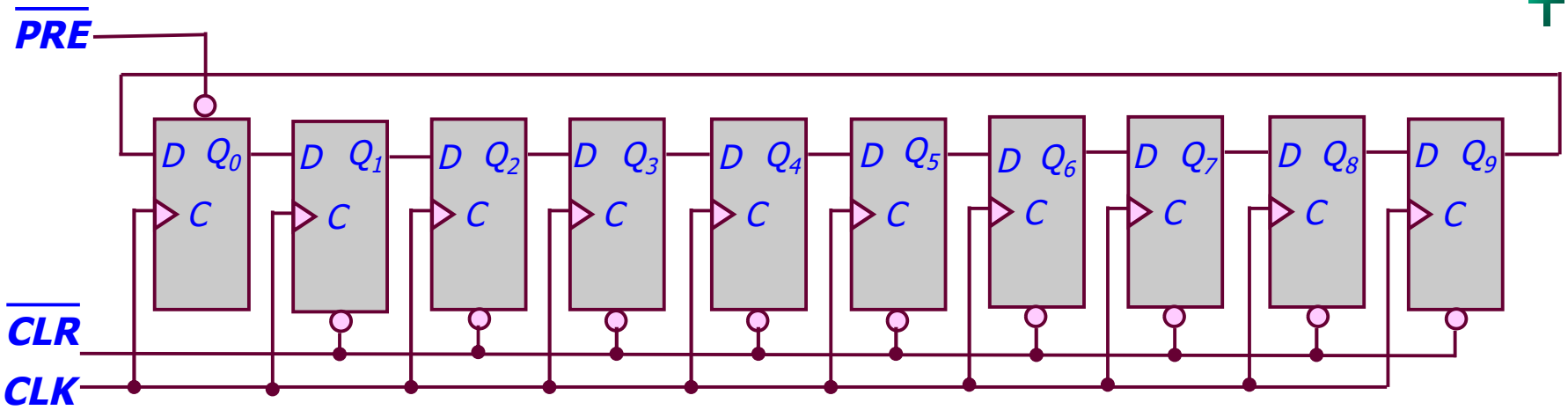


Shift Register Counters

- Counters can be constructed using shift registers with *feedback* to produce special sequences. These counters are commonly called *ring counters* or *twisted ring counters*.
- A **Ring counter** is a shift register in which the output of the last flip-flop is fed to the serial input.
- The data shifts into the next flip-flop for every clock cycle. Thus, an *n*-bit ring counter can only count *n* states.

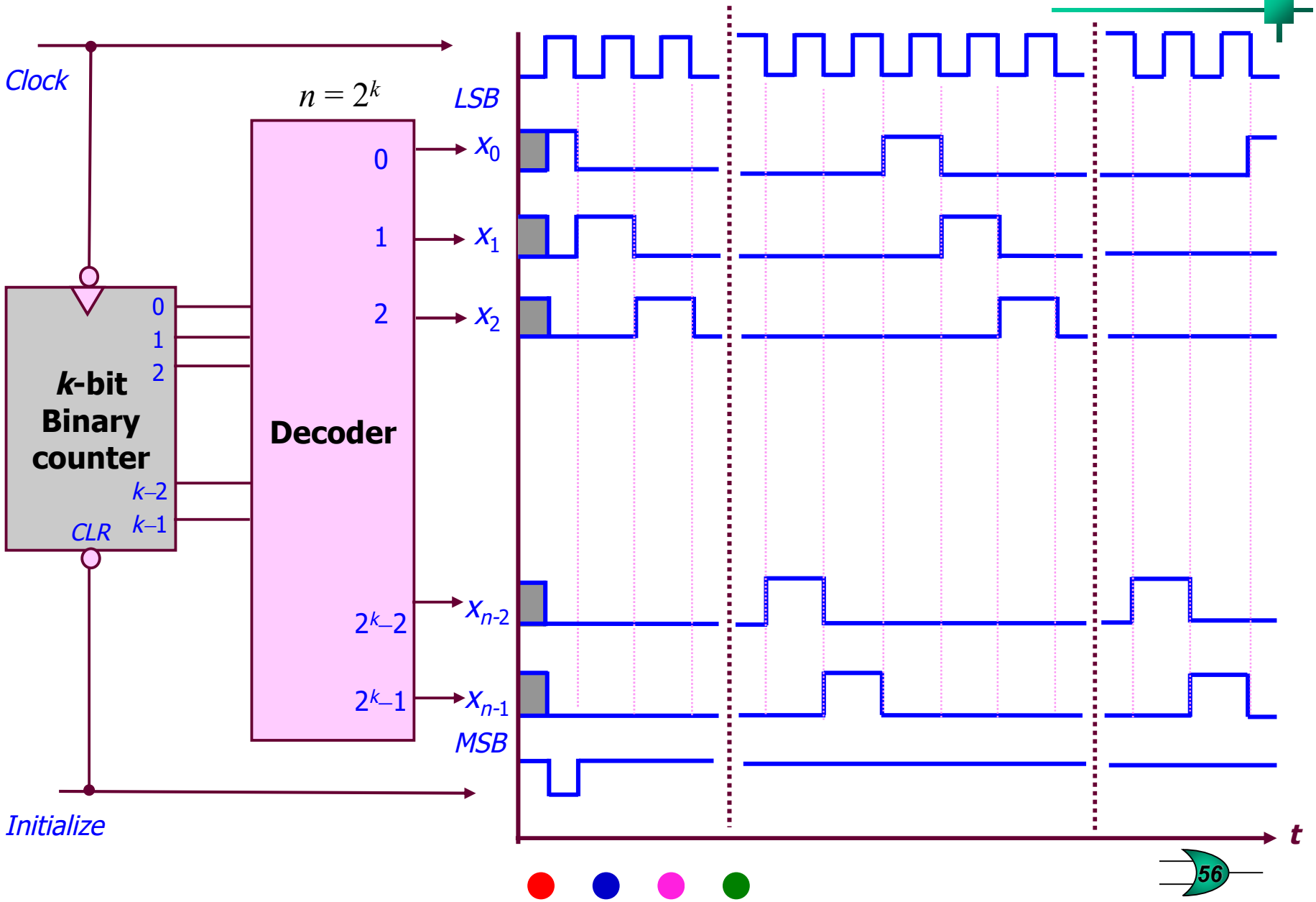


A 10-bit Ring Counter



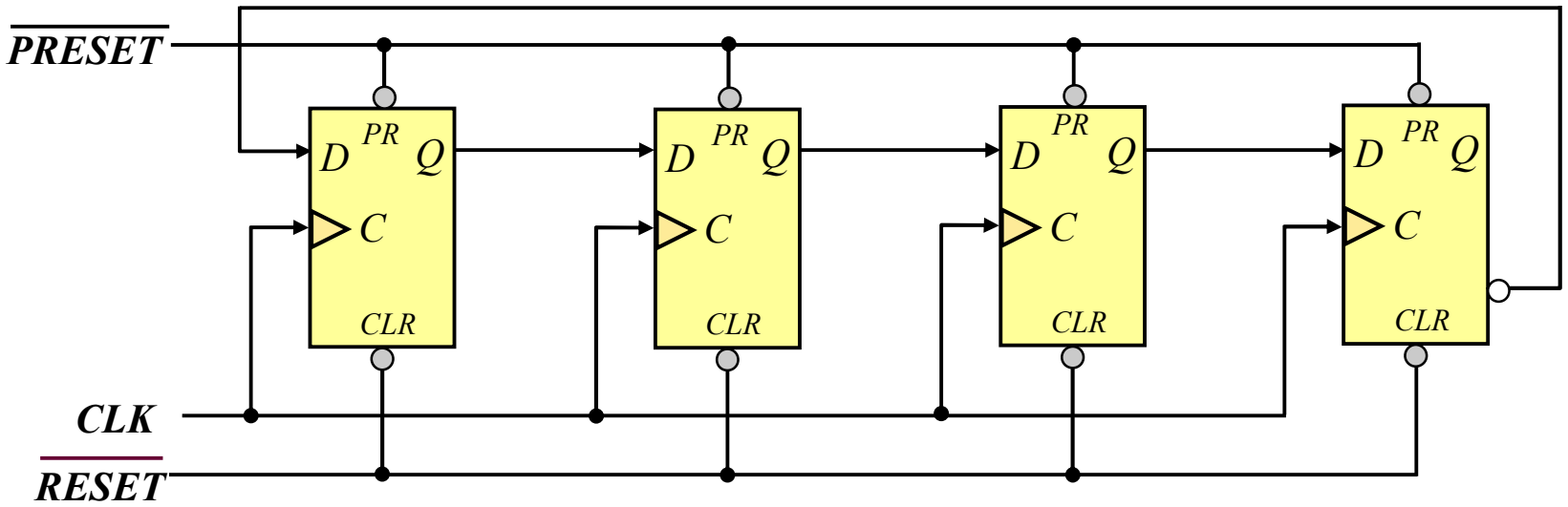
Clock pulse	Q_0	Q_1	Q_2	Q_3	Q_4	Q_5	Q_6	Q_7	Q_8	Q_9
0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0	0
3	0	0	0	1	0	0	0	0	0	0
4	0	0	0	0	1	0	0	0	0	0
5	0	0	0	0	0	1	0	0	0	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	0	0	0	0	0	1	0	0
8	0	0	0	0	0	0	0	0	1	0
9	0	0	0	0	0	0	0	0	0	1

Ring Counter from Binary Counter and Decoder

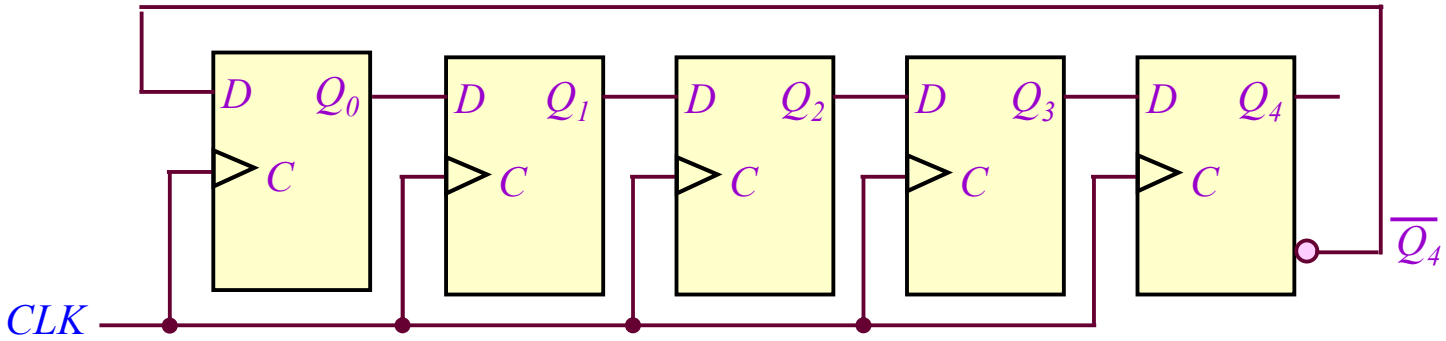


Shift Register Counters

- **A twisted-ring counter** *(a.k.a. Johnson counter or Moebius counter)* **:** the complement output of the last flip-flop is fed to the input of the first stage.
- An *n*-bit Johnson counter cycles through 2*n* states.
- Useful for the generation of multiphase clock, timing, and control signals.



A 5-bit Twisted Ring (Johnson) Counter

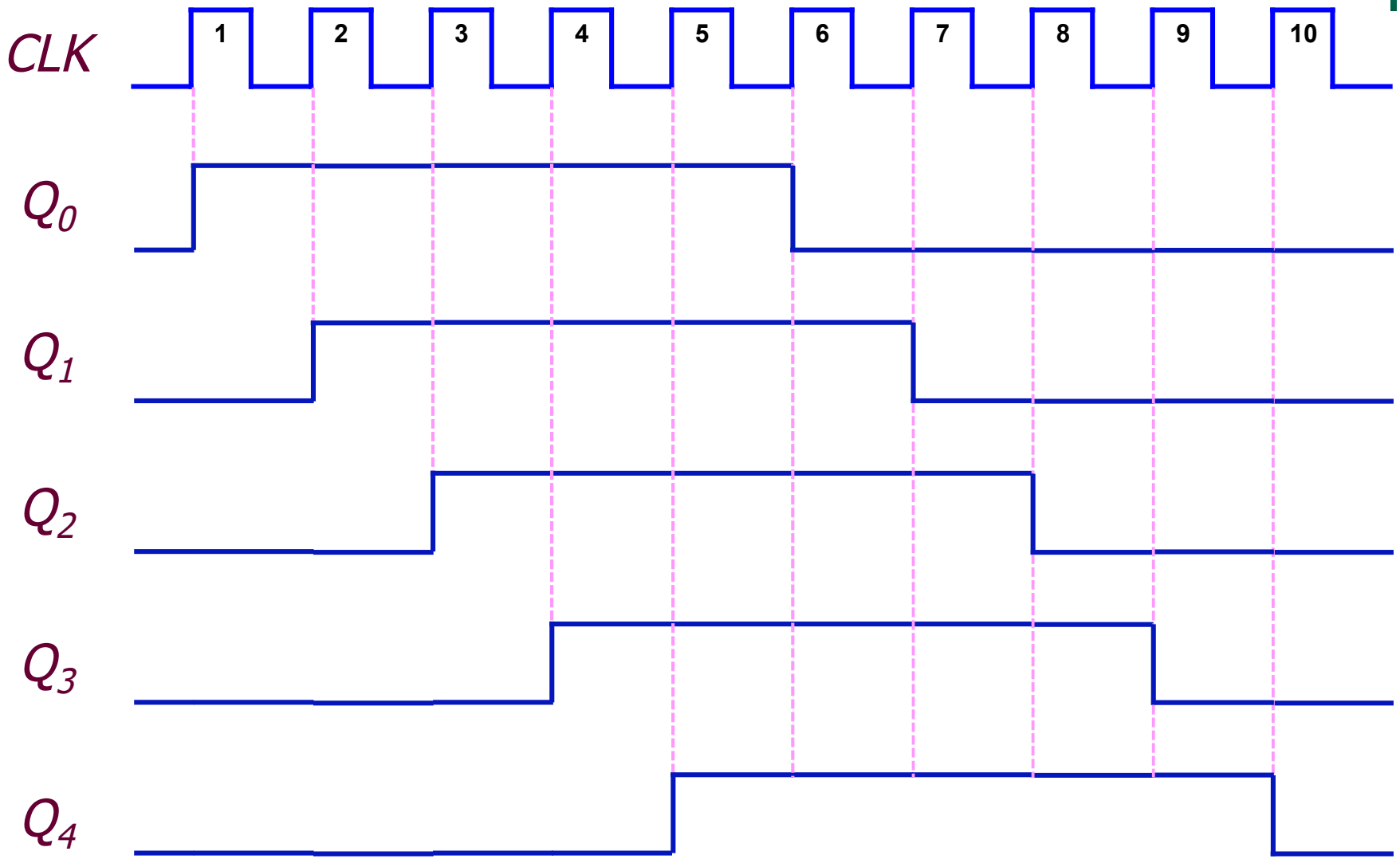


Clock Pulse	Q_0	Q_1	Q_2	Q_3	Q_4	State decoder
0	0	0	0	0	0	$Q_4'\,Q_0'$
1	1	0	0	0	0	$Q_1'\,Q_0$
2	1	1	0	0	0	$Q_2'\,Q_1$
3	1	1	1	0	0	$Q_3'\,Q_2$
4	1	1	1	1	0	$Q_4'\,Q_3$
5	1	1	1	1	1	$Q_4\,Q_0$
6	0	1	1	1	1	$Q_1\,Q_0'$
7	0	0	1	1	1	$Q_2\,Q_1'$
8	0	0	0	1	1	$Q_3\,Q_2'$
9	0	0	0	0	1	$Q_4\,Q_3'$

invert

Decimal counter

Timing Sequence for a 5-bit Johnson Counter

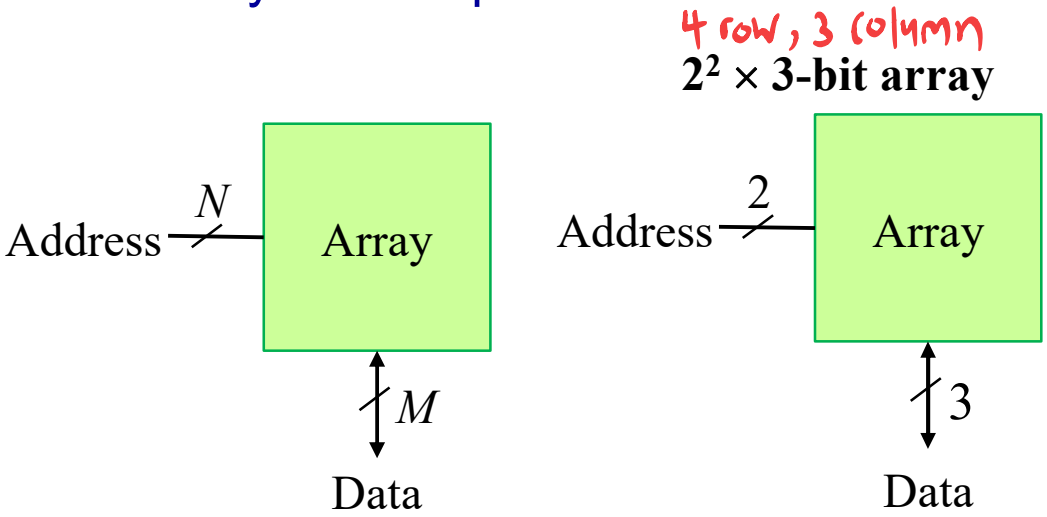


Topic : Sequential Circuits

- Latches & flip-flops
- Synchronous state machine analysis
- Synchronous state machine design
- Timing parameters
- Counters
- Shift registers
- Memory

Memory Arrays

- Two-dimensional array of bit cells.
- Each bit cell stores one bit.
- An array with *N* address bits and *M* data bits.
  - 2^N rows and *M* columns.
  - Depth: number of rows (number of words).
  - Width: number of columns (size of word).
  - Array size: depth × width = $2^N \times M$.



Address	Data		
00	0	1	0
01	1	0	0
10	1	1	0
11	0	0	1

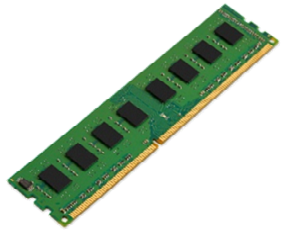
depth

width

Types of Memory Array

- Read Only Memory (ROM):
  - **Nonvolatile:** retains data when power is turned off.
  - Read quickly, but writing is impossible or slow.
  - Name for historical reasons, as ROMs were written at manufacturing time or by burning fuses. Once configured, it cannot be written again. This is no longer the case for flash memory and other types of ROMs.
  - Boot ROM for personal computers, firmware in digital devices for providing built-in functionalities, flash memory in cameras, thumb drives, televisions, and mobile phones are all ROMs.

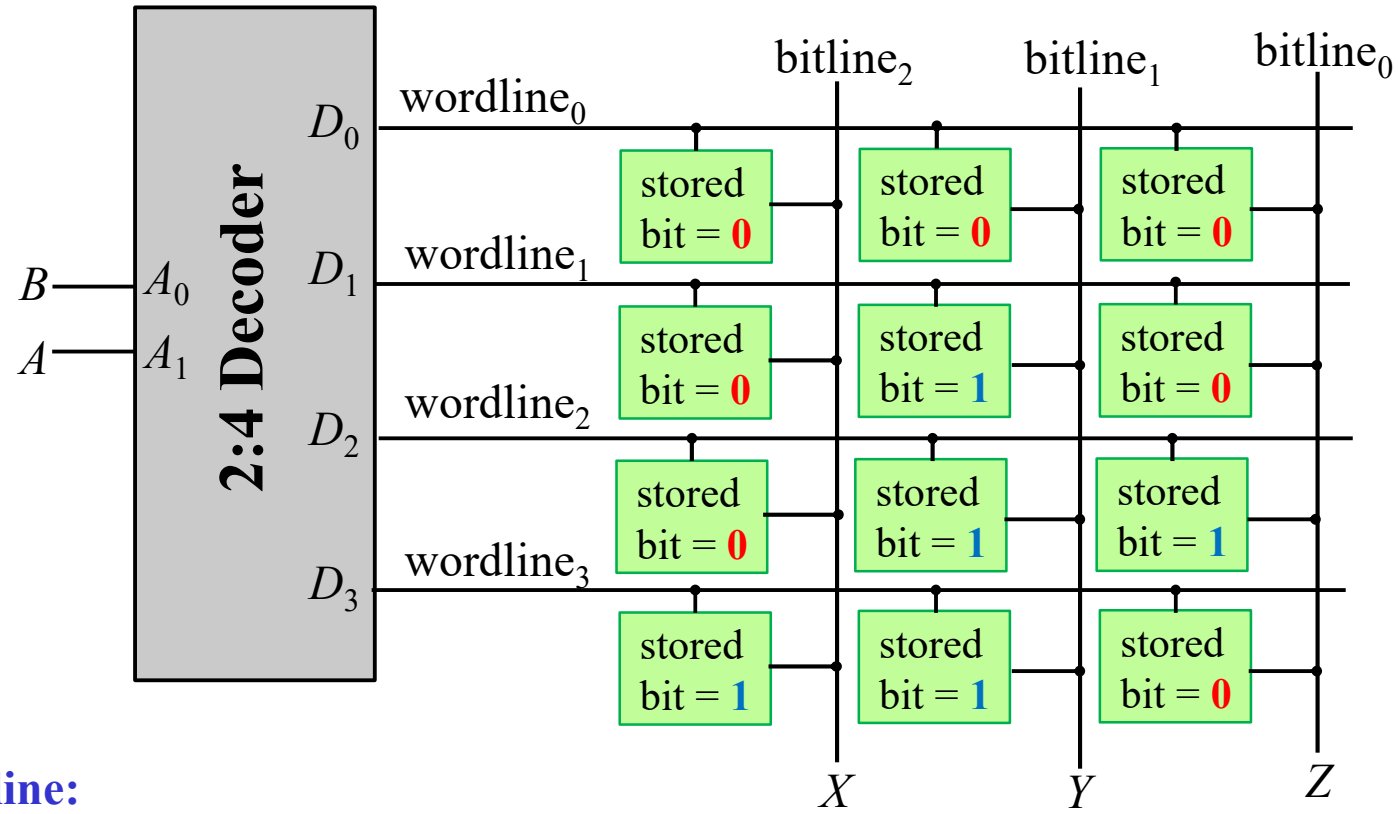




- Random Access Memory (RAM):
  - **Volatile:** loses data when power is turned off.
  - Read and write quickly.
  - Name for historical reasons, as any data word can be accessed as easily as others as opposed to sequential access memories such as magnetic tape.
  - Store data for run-time applications, operating systems, and other active processes.

Logic Implementation With ROM

Implement $X = AB$, $Y = A+B$ and $Z = A\bar{B}$ using a 4×3-bit memory array.



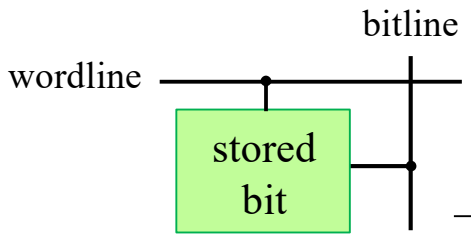
Wordline:

– similar to an enable

– allows a single row in the memory array to be read or written

– corresponds to a unique address

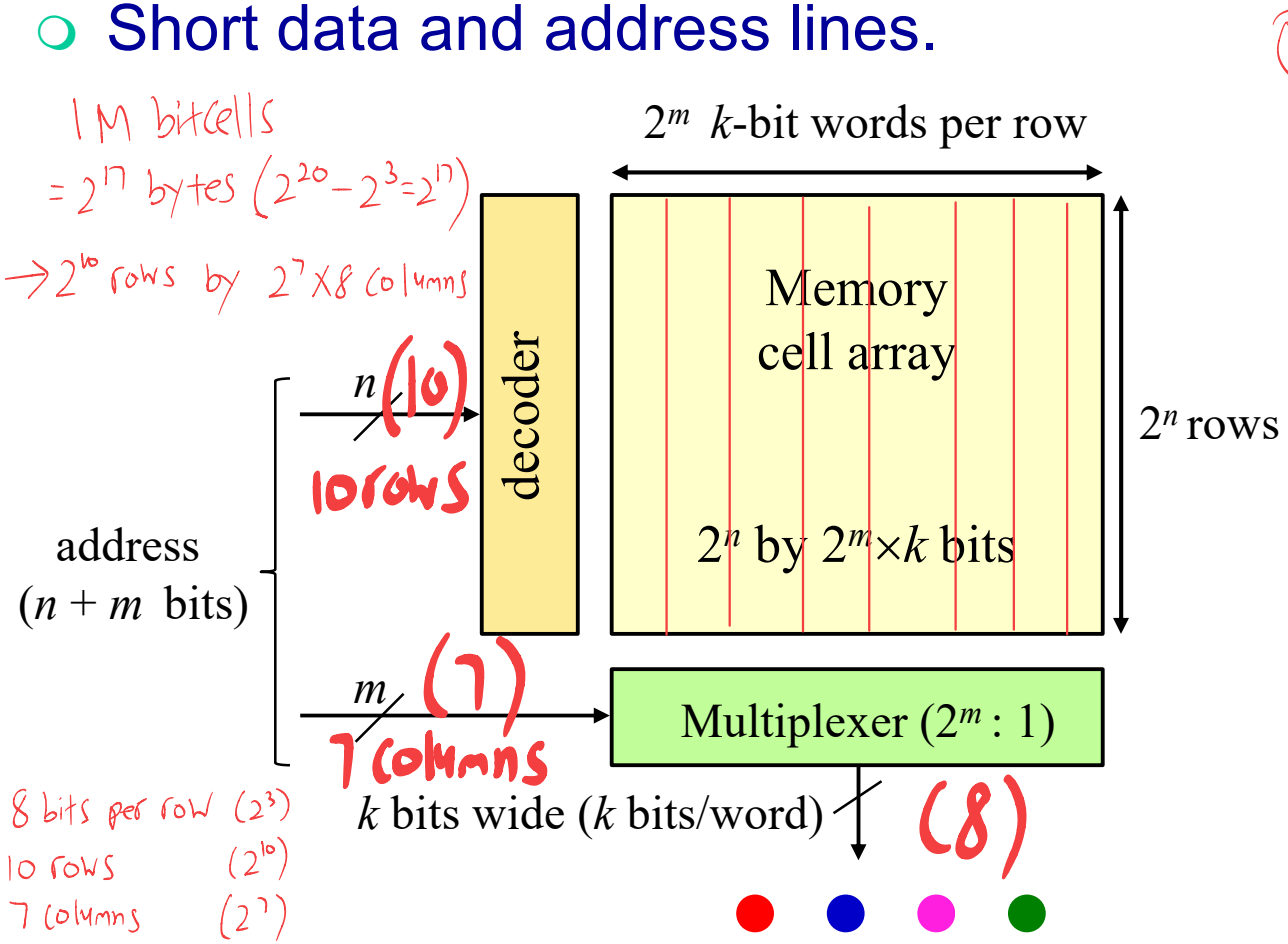
– only one wordline is HIGH at any given time

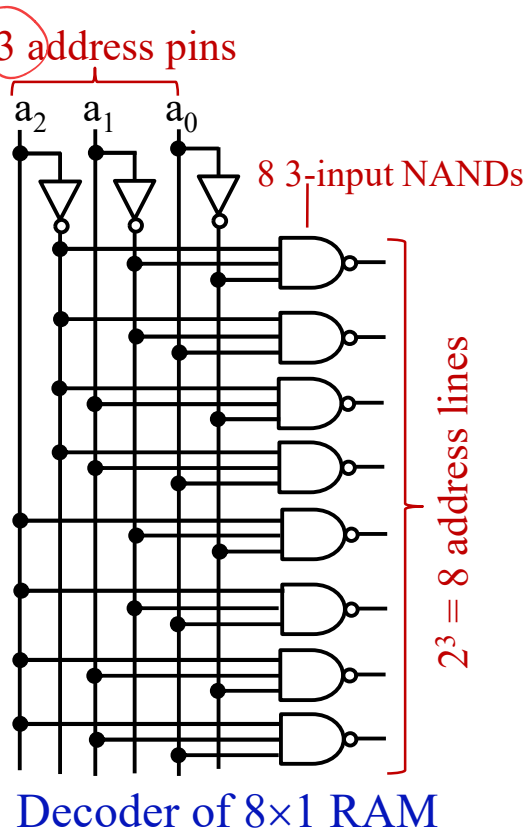


Addressing A Memory

Desirable to have

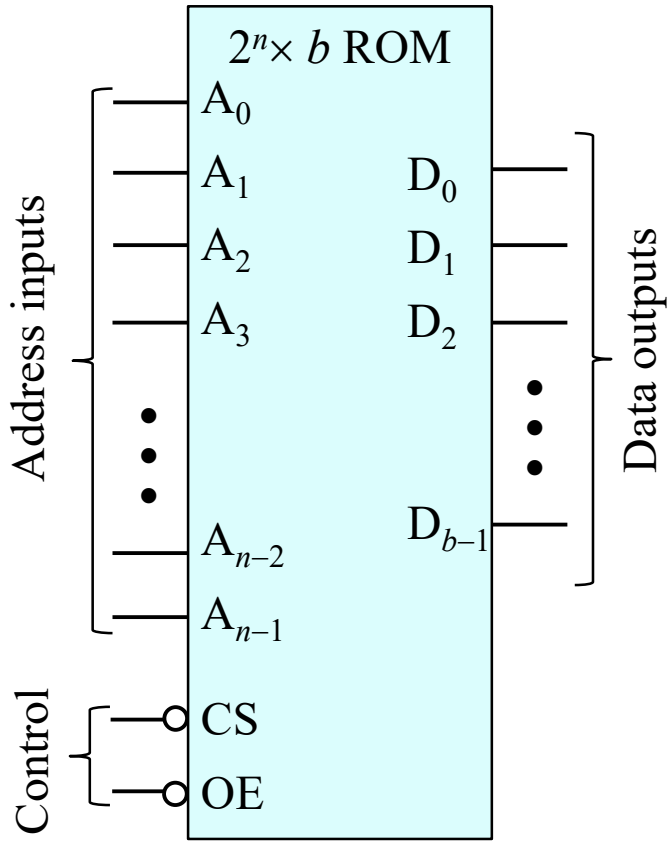
- Rectangular or square memory array.
- Simple decoding logic.
  - Problem: A 1M × 1 RAM uses 1,048,576 20-input NANDs.
- Short data and address lines.

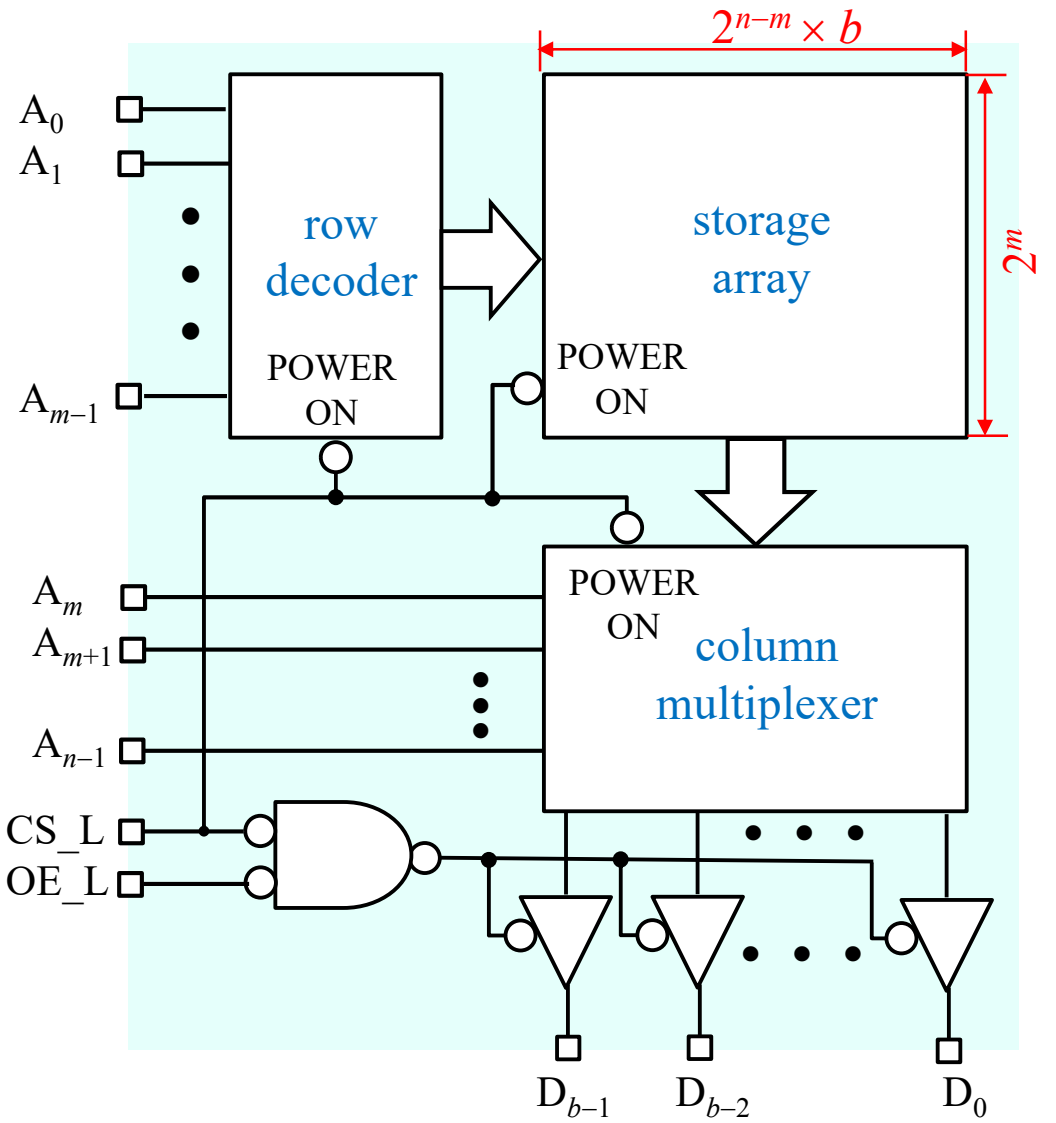




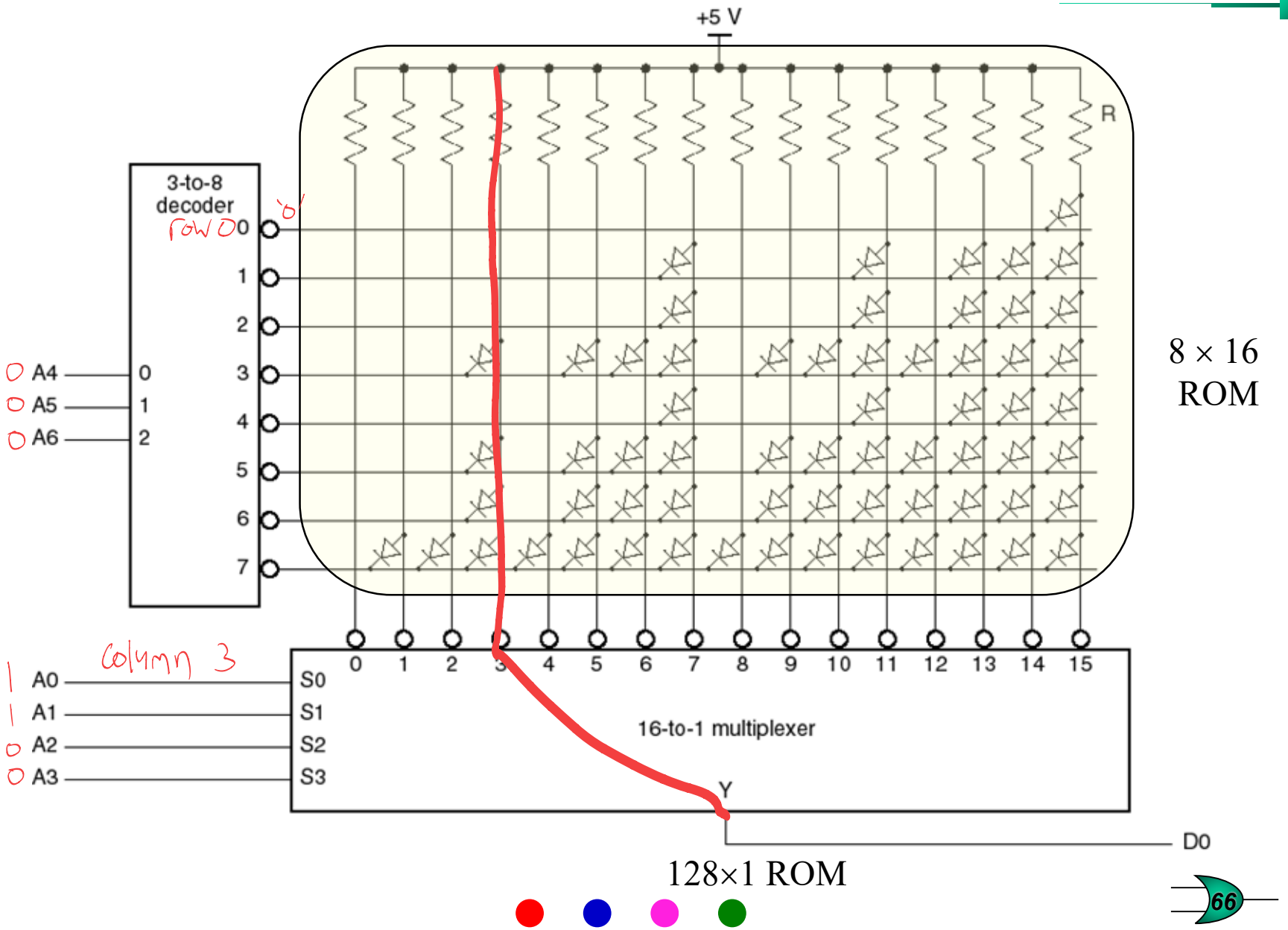
Decoder of 8×1 RAM

ROM Control and I/O Signals



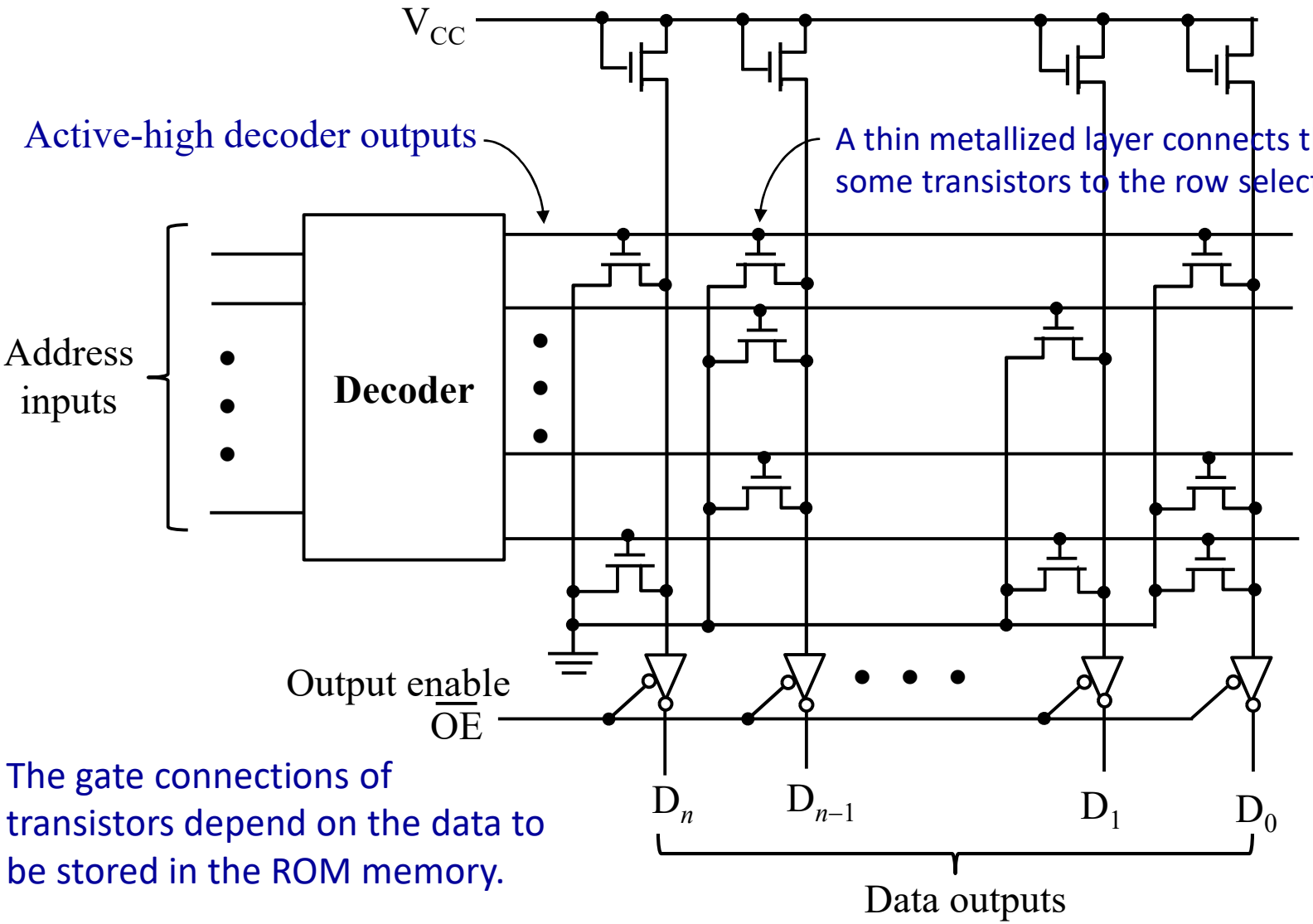


Two-dimensional Decoding



Modern ROM Design

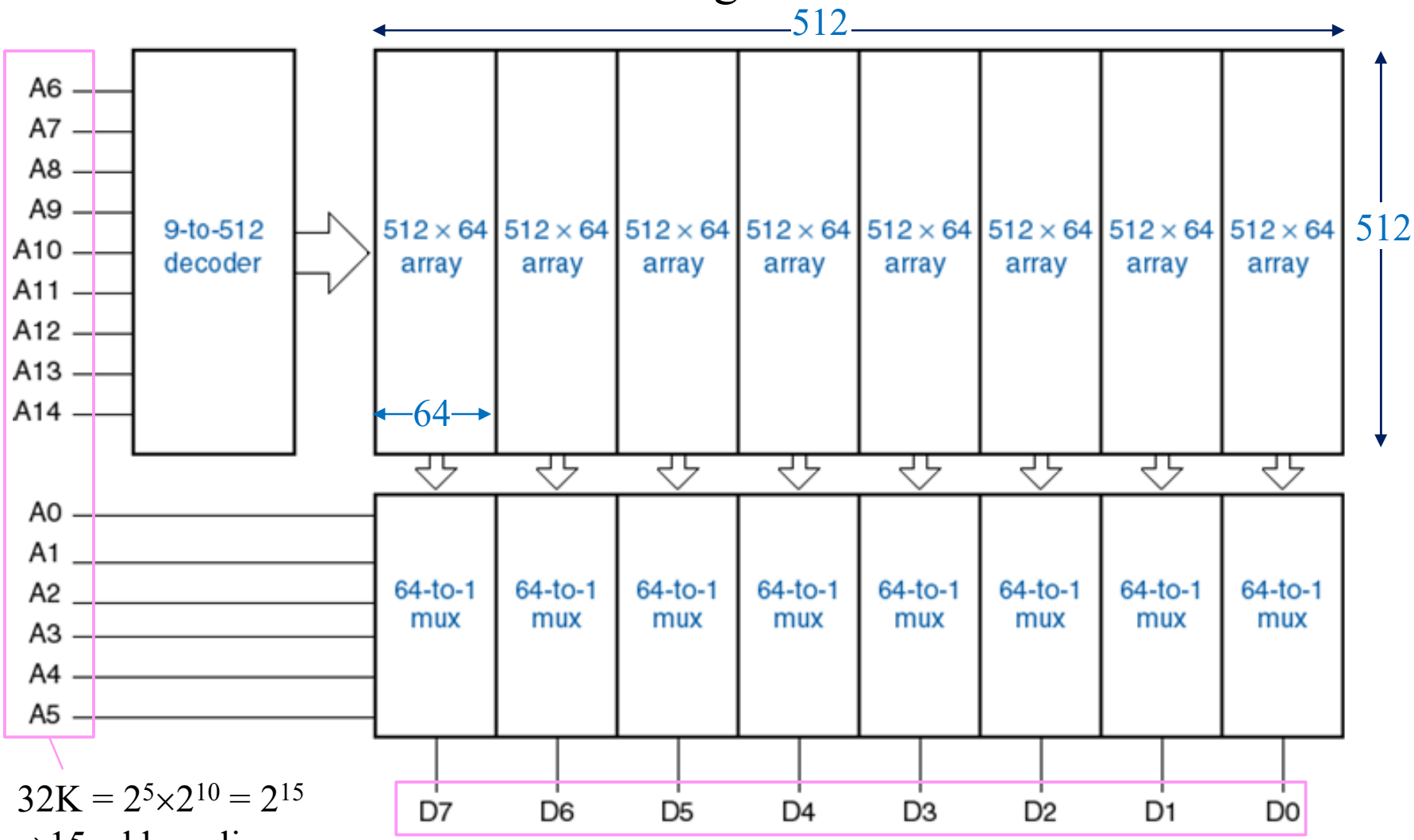
Diodes and pull up resistors are replaced by MOS transistors.



The gate connections of transistors depend on the data to be stored in the ROM memory.

Row Decoder & Column Multiplexer

Construct a 32K × 8 ROM from eight 512×64 ROMs

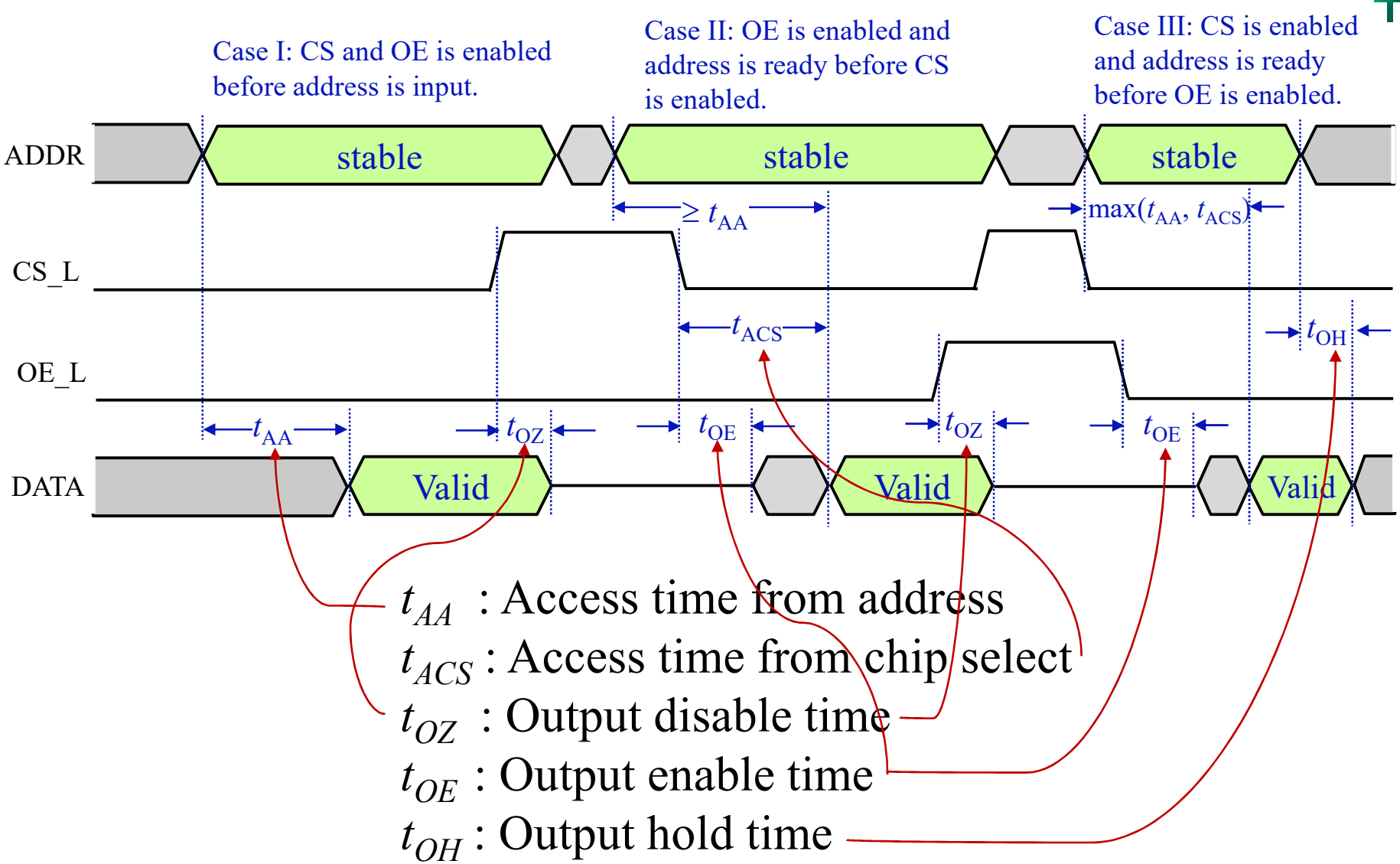


$32K = 2^5 \times 2^{10} = 2^{15}$

⇒15 address lines, A0 to A14

8 data lines

ROM Timing



t_{AA} : Access time from address

t_{ACS} : Access time from chip select

t_{OZ} : Output disable time

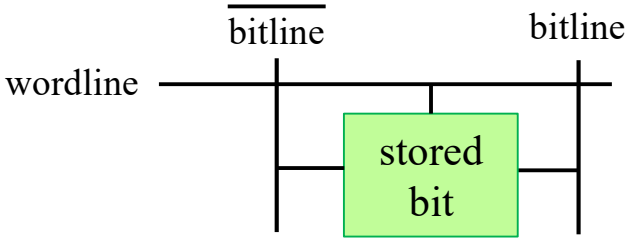
t_{OE} : Output enable time

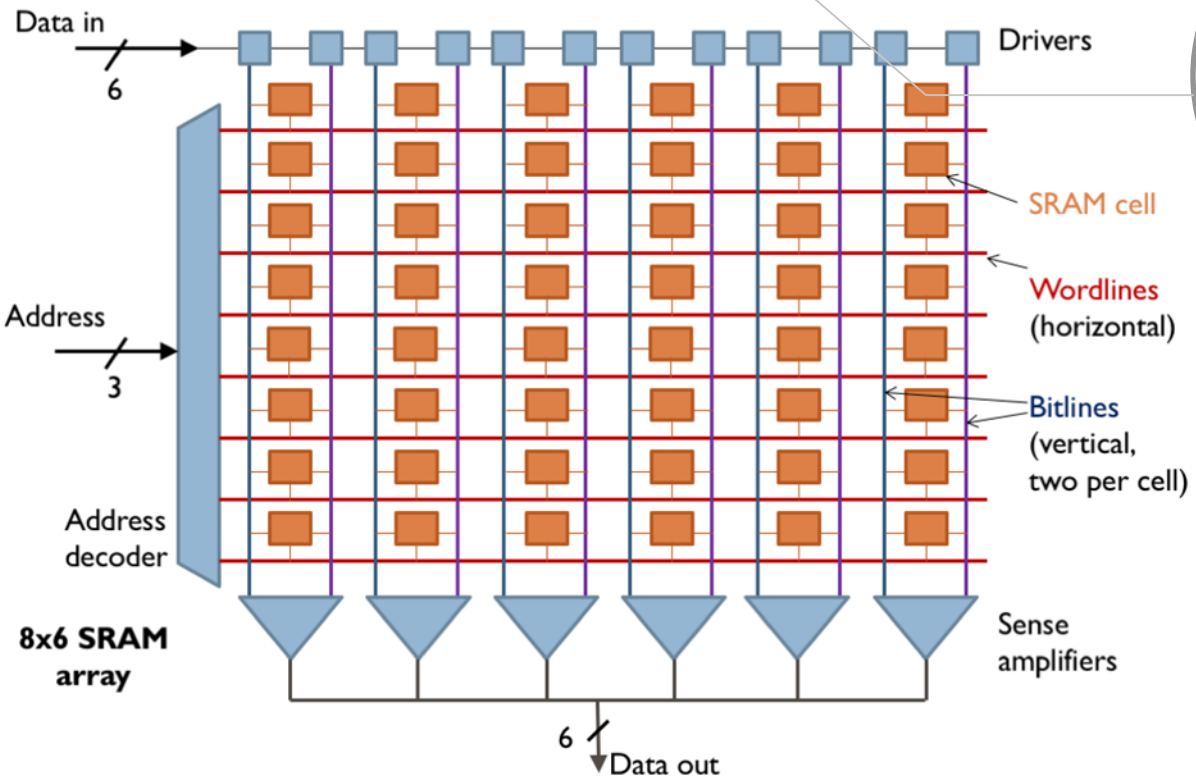
t_{OH} : Output hold time

Random Access Memories

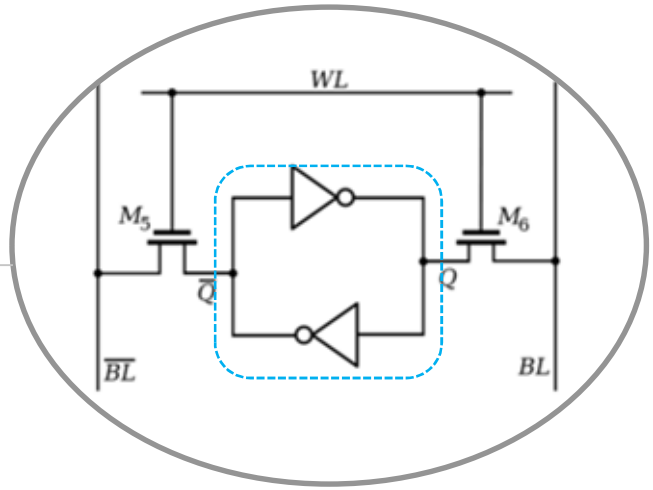
- SRAM (Static RAM)
  - Memory behaves like latches or flip-flops
- DRAM (Dynamic Memory)
  - Memory lasts only for a few milliseconds
  - Must “refresh” locations by reading or writing
- Non-volatile RAM (NVRAM) = RAM + battery

SRAM

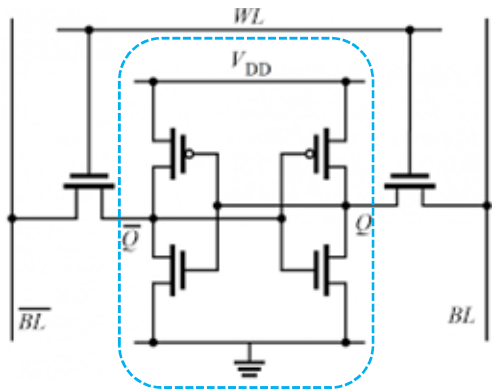




8×6 SRAM array



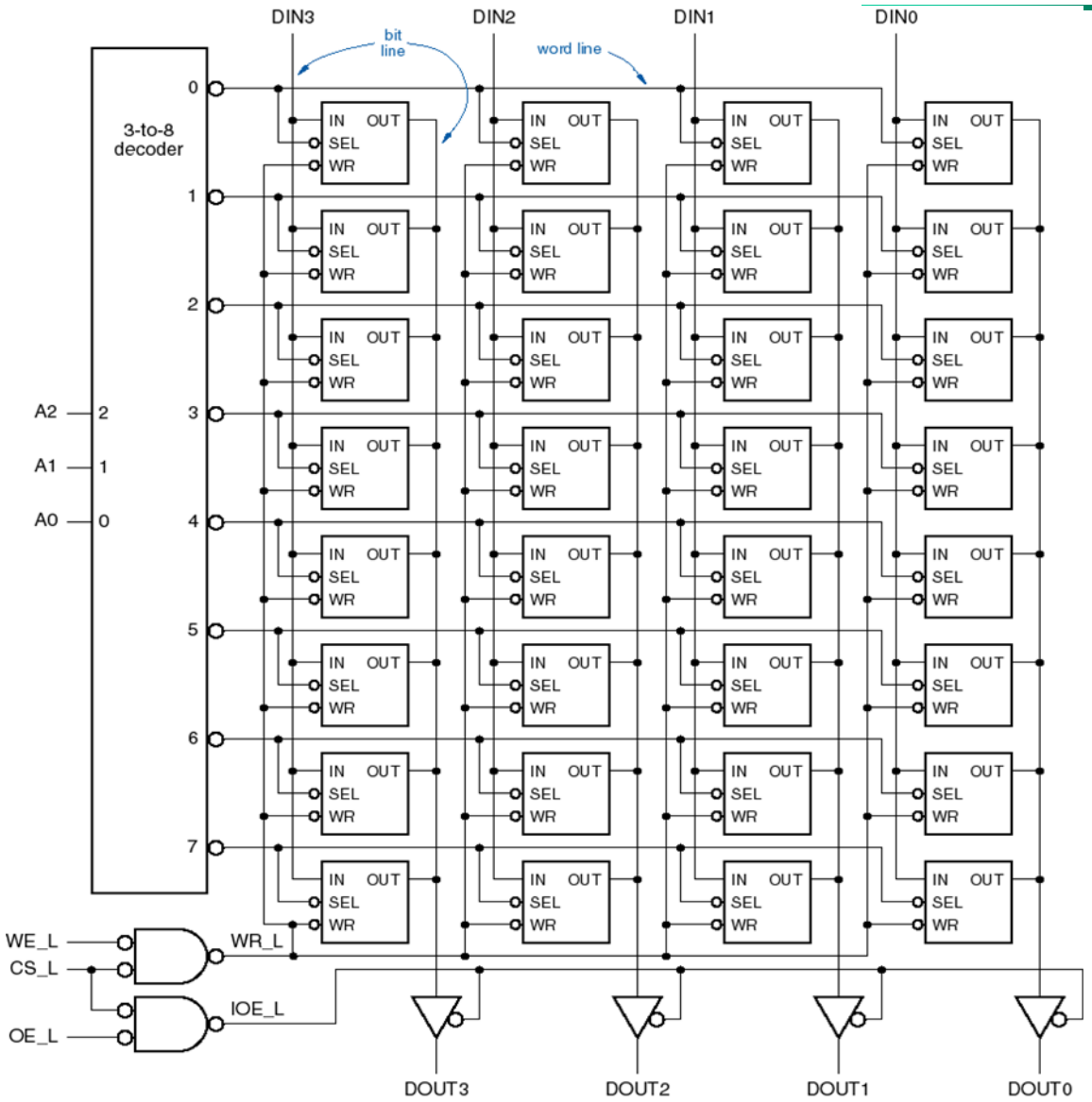
1-bit SRAM cell



SRAM Write Operation

Implications for write operation:

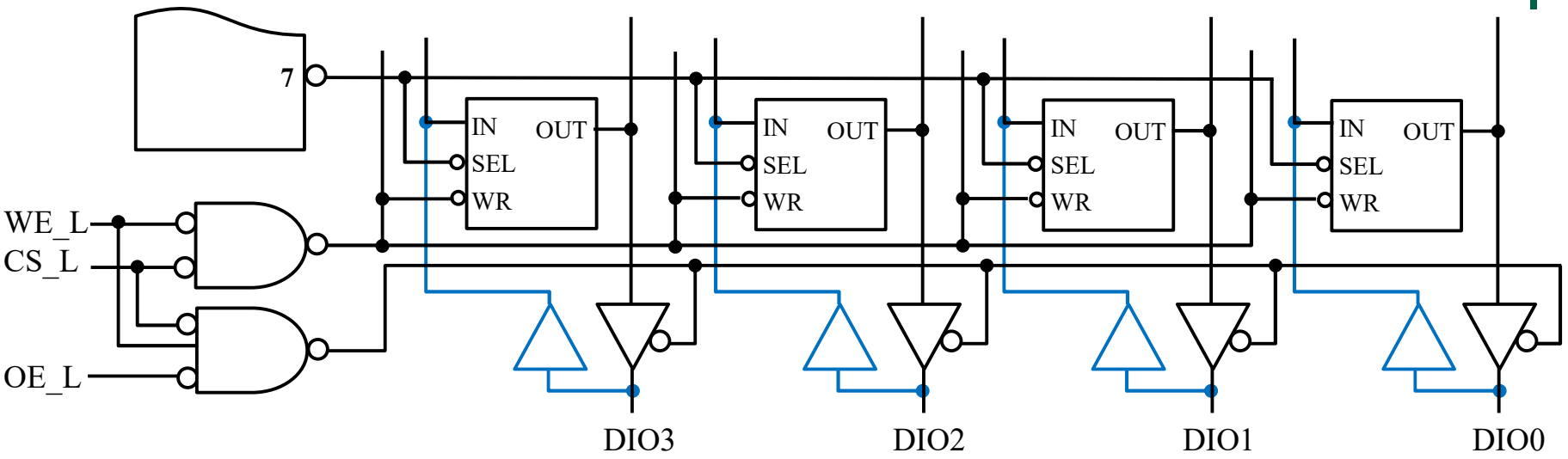
- Address must be stable before writing cell.
- Data must be stable before ending a write



Control lines:

- Write enable
- Chip select
- Output enable

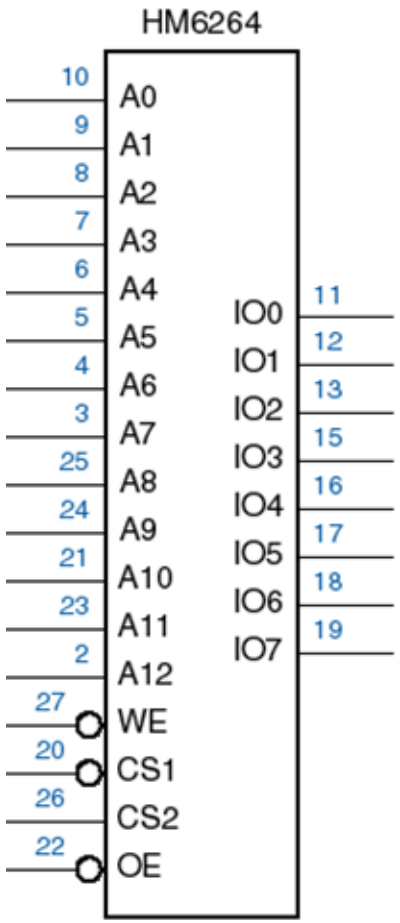
Bidirectional Data In and Out Pins



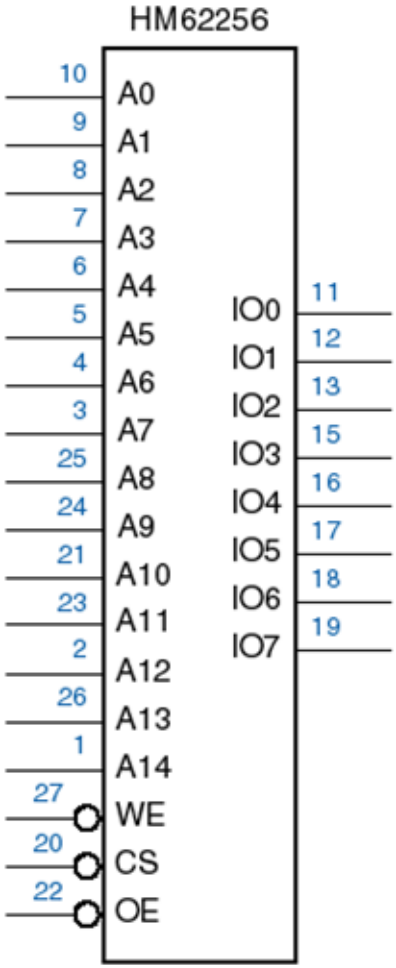
- Use the same data pins for reads and writes
  - Common on wide devices (≥ 32 data bits)
  - Makes sense when used with (bidirectional) microprocessor buses
  - Tristate buffers are required to use the same lines for data input and output.

SRAM Devices

- Similar to ROM packages

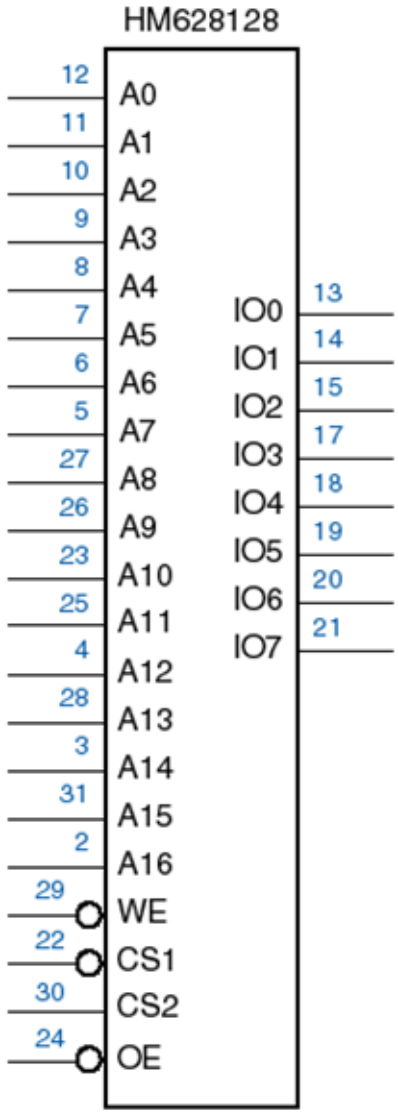


8Kbytes

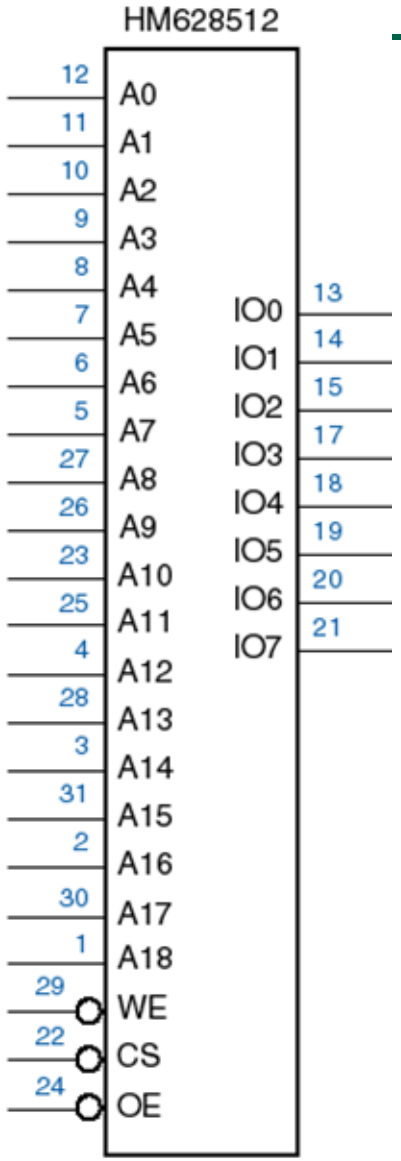


32Kbytes

28-pin DIPs



128Kbytes

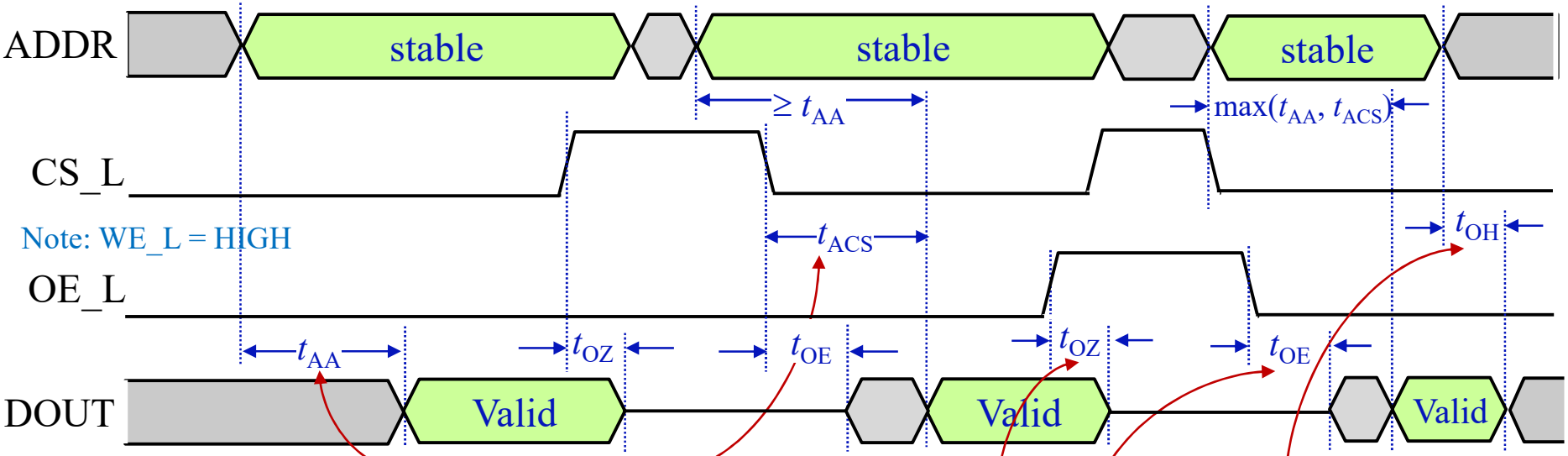


512Kbytes

32-pin DIPs

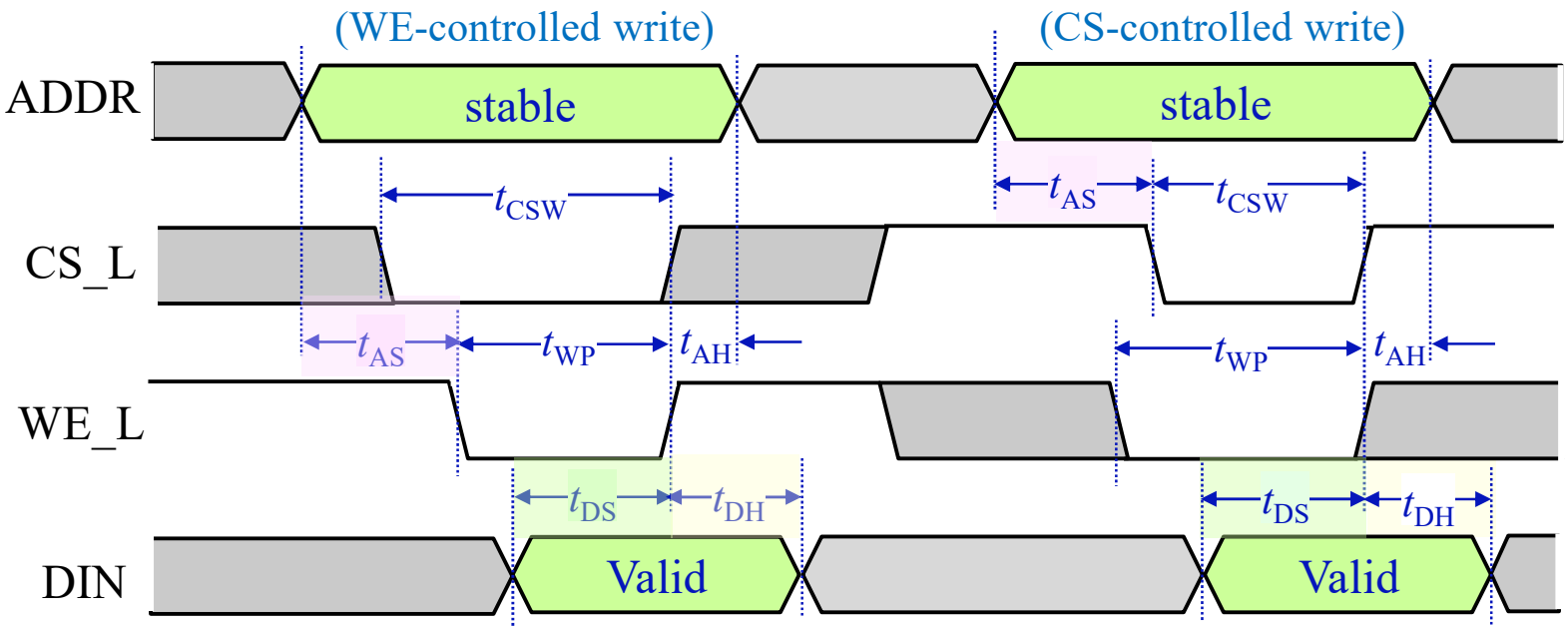
SRAM Read timing

- Similar to ROM read timing (Refer to Slide# 69)



t_{AA} : Access time from address
t_{ACS} : Access time from chip select
t_{OZ} : Output disable time
t_{OE} : Output enable time
t_{OH} : Output hold time

SRAM Write Timing

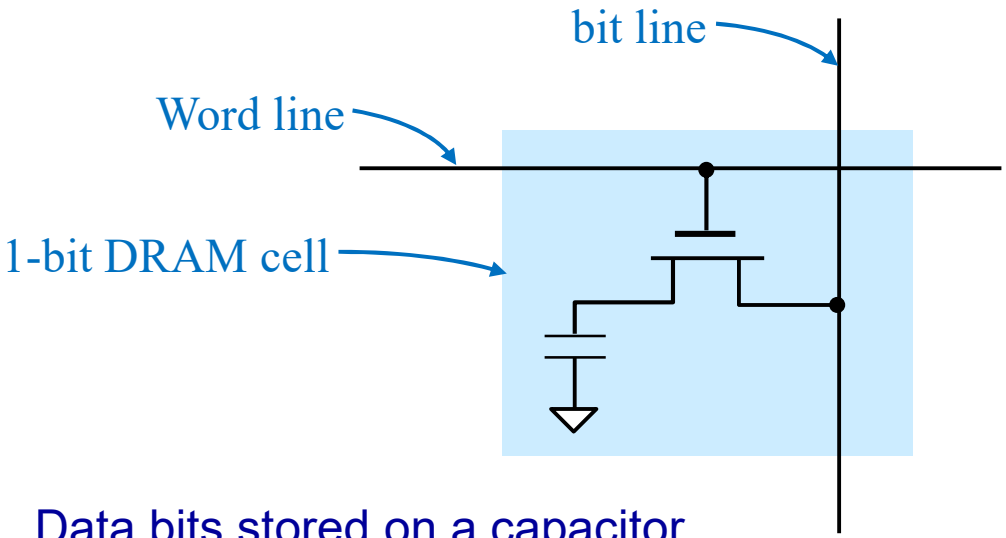


t_{AS} : Address setup time before write
t_{AH} : Address hold time after write
t_{CSW} : Chip select setup before end of write (to select a cell)
t_{WP} : Write pulse minimum width (to reliably latch data into a cell)
t_{DS} : Data setup time before end of write
t_{DH} : Data hold time after end of write

- The address must be stable before and after write-enable is asserted
- Data is latched on the trailing edge of WE (WE-controlled) or CS (CS-controlled)

Dynamic RAM (DRAM)

- SRAMs typically use six transistors per bit of storage.
- DRAMs use only one transistor per bit.
- 1/0 = capacitor charged/discharged.
- *Dynamic* because the value needs to be refreshed (rewritten) periodically and after being read.



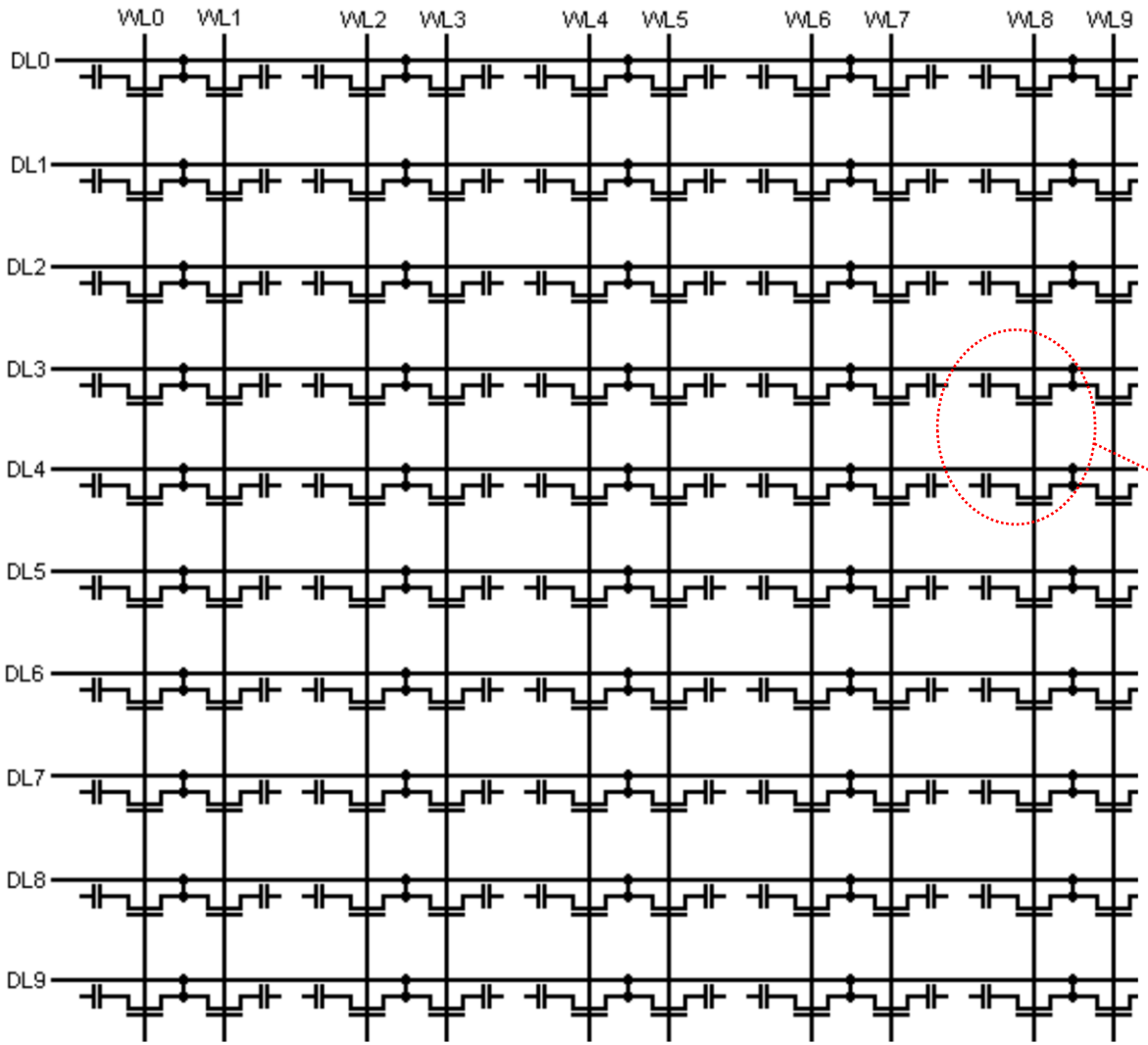
Data bits stored on a capacitor

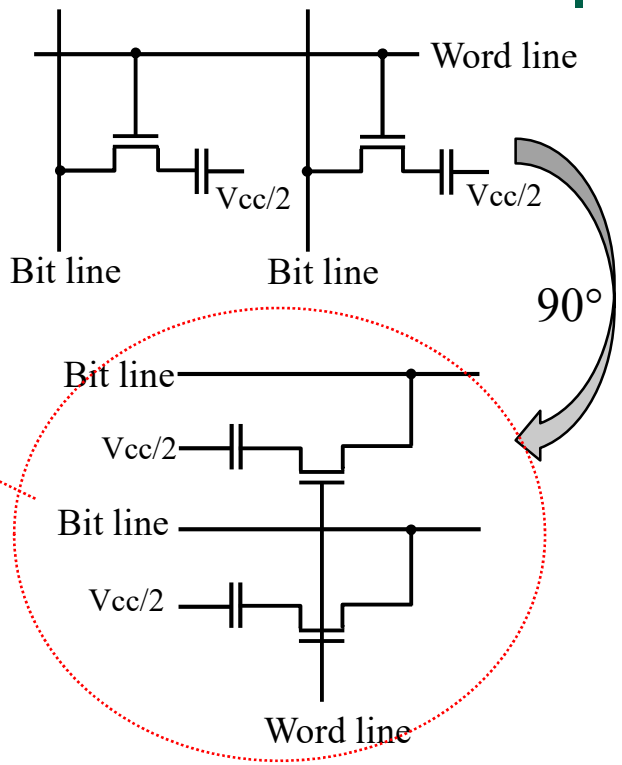
Robert Dennard, 1932 -

Invented DRAM in 1966 at IBM

- Others were skeptical that the idea would work.
- By the mid-1970's DRAM was in virtually all computers.

A Compact Layout of DRAM Array

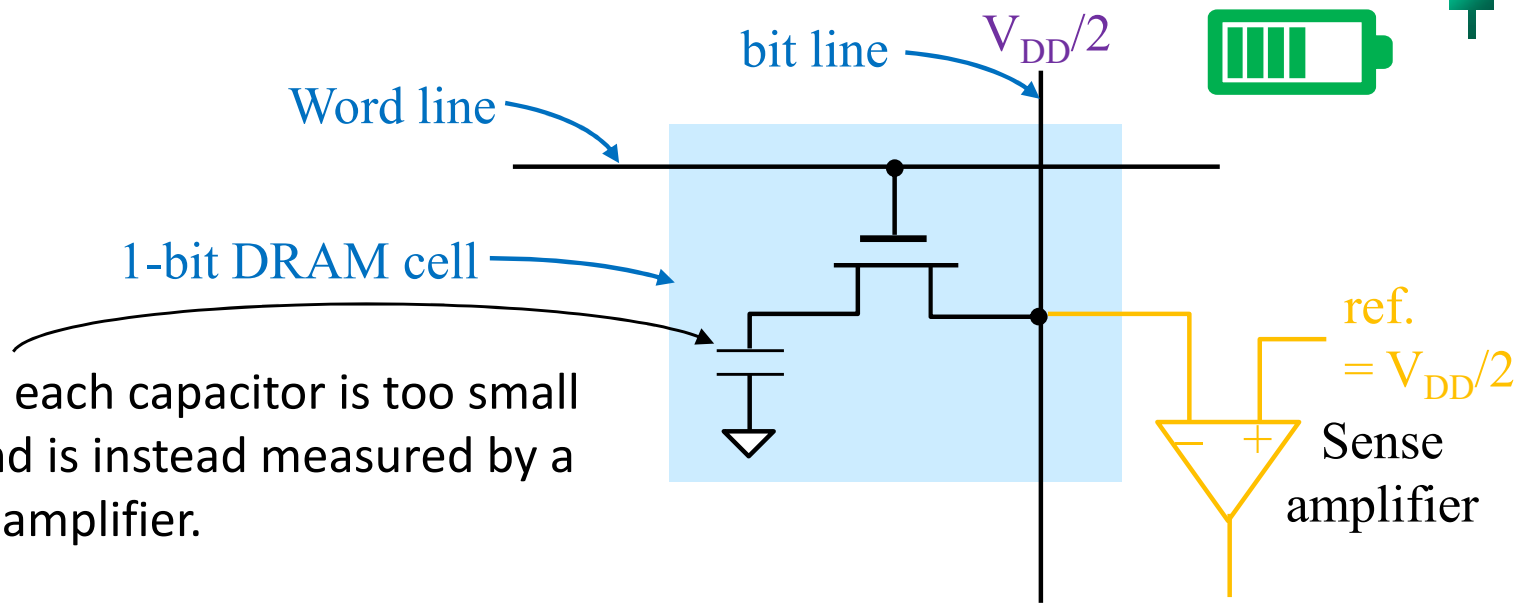




- Smaller cell size per bit.
- Lower cost.
- Lower power consumption.

- Slower.

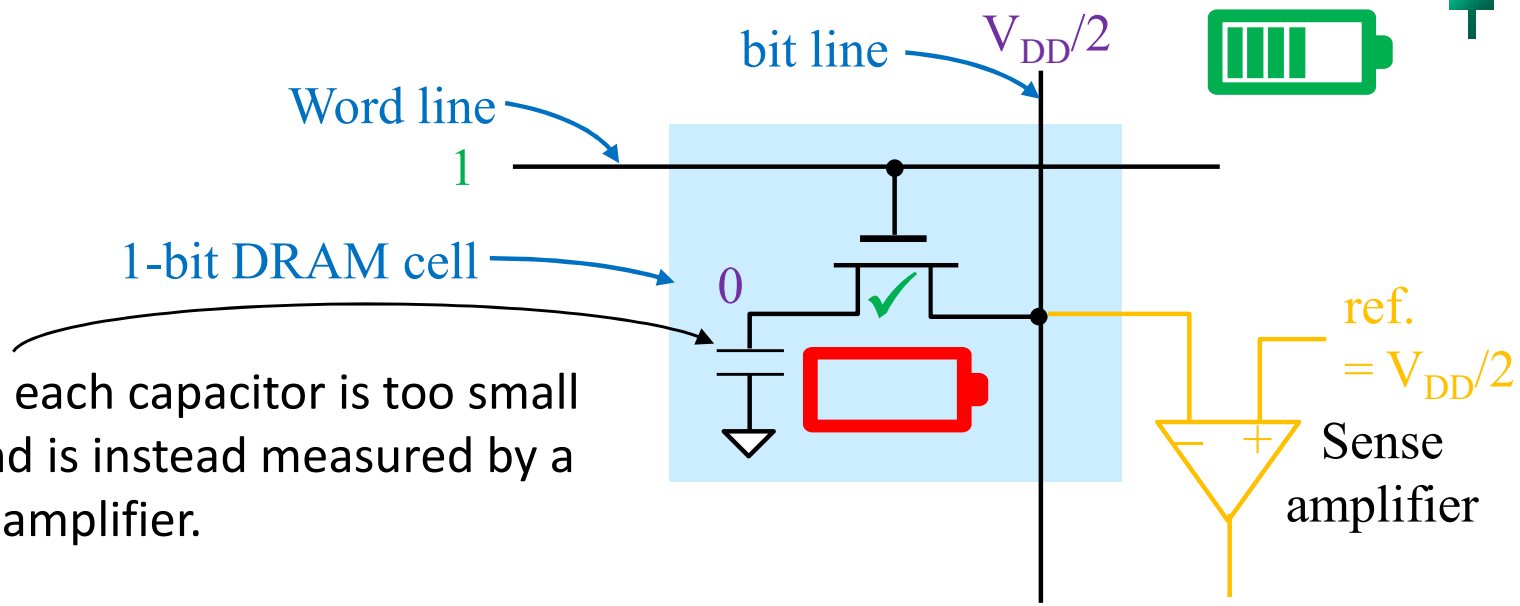
DRAM Read Operations



The charge stored on each capacitor is too small to be read directly and is instead measured by a circuit called a sense amplifier.

- Pre-charge bit line to $V_{DD}/2$.

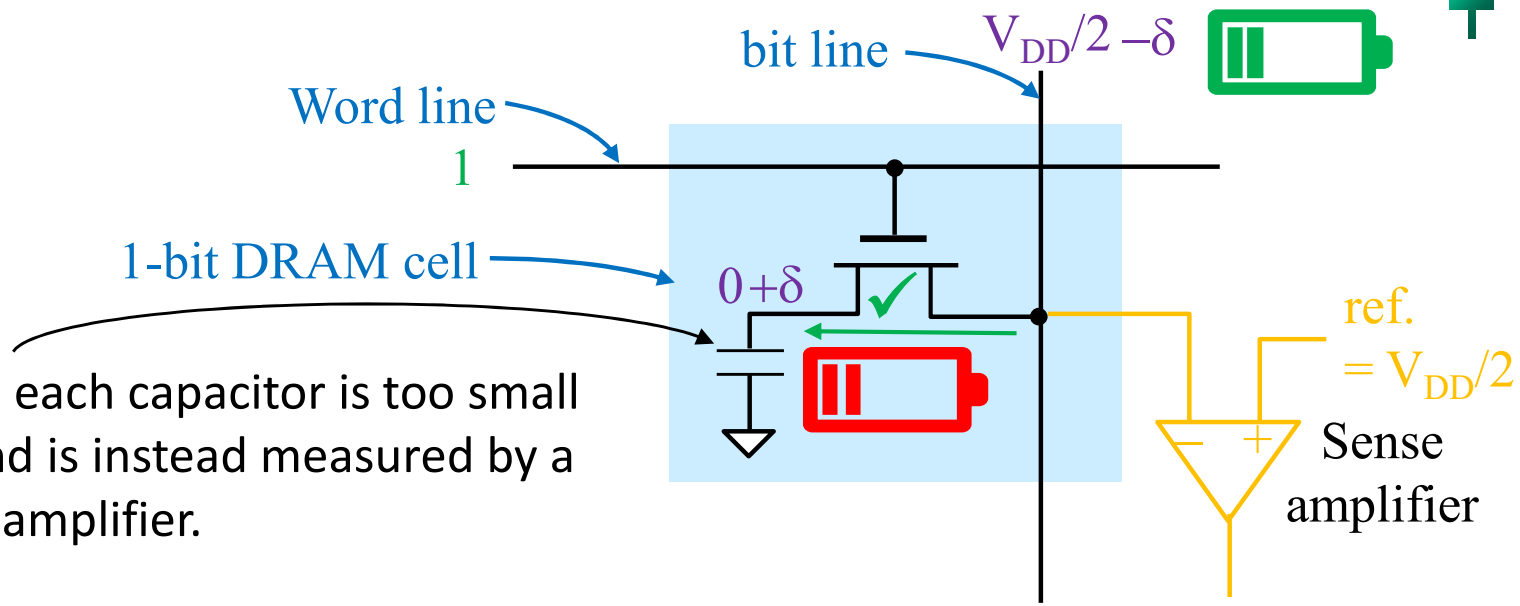
DRAM Read Operations



The charge stored on each capacitor is too small to be read directly and is instead measured by a circuit called a sense amplifier.

- Pre-charge bit line to $V_{DD}/2$.
- Take the word line HIGH.
- If the cell stores a logic 0 (0V),

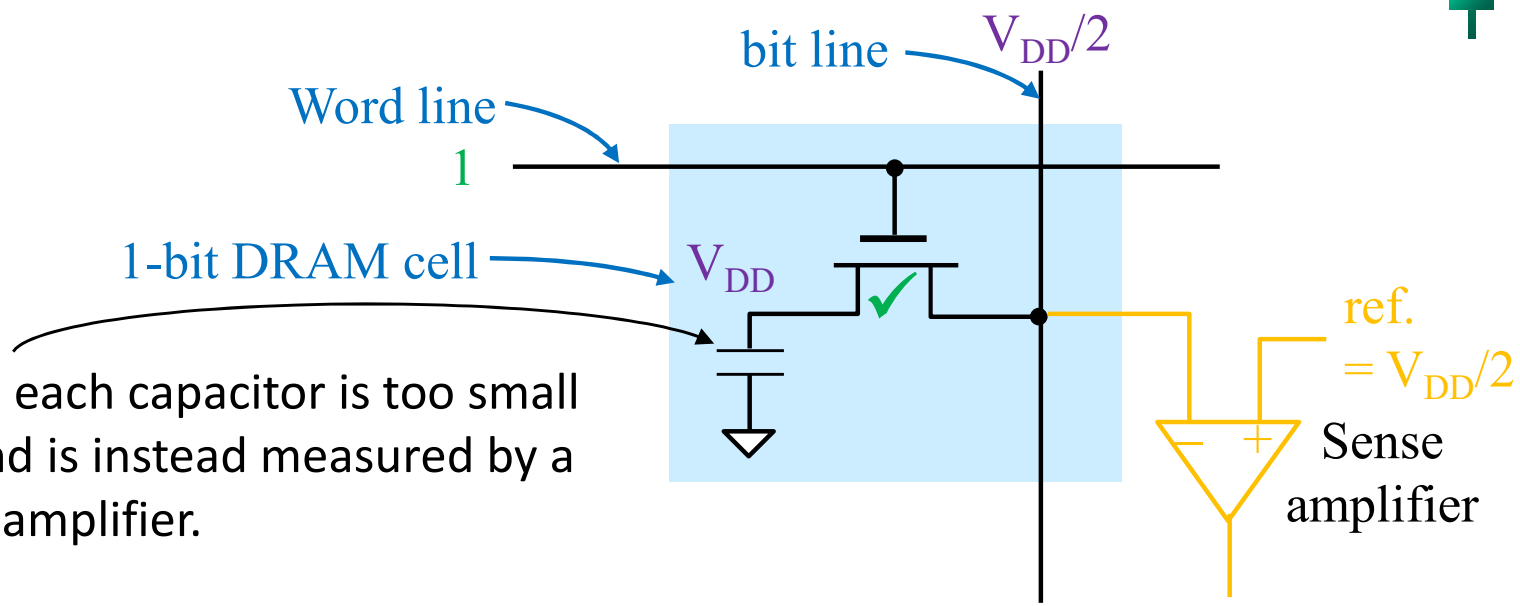
DRAM Read Operations



The charge stored on each capacitor is too small to be read directly and is instead measured by a circuit called a sense amplifier.

- Pre-charge bit line to $V_{DD}/2$.
- Take the word line HIGH.
- The cell (if at 0V) shares its charge with the bitline ⇒ produce small change in the voltage of the bitline ($V_{DD}/2 - \delta$).
- Sensor amplifier amplifies δ towards the original value of the cell.

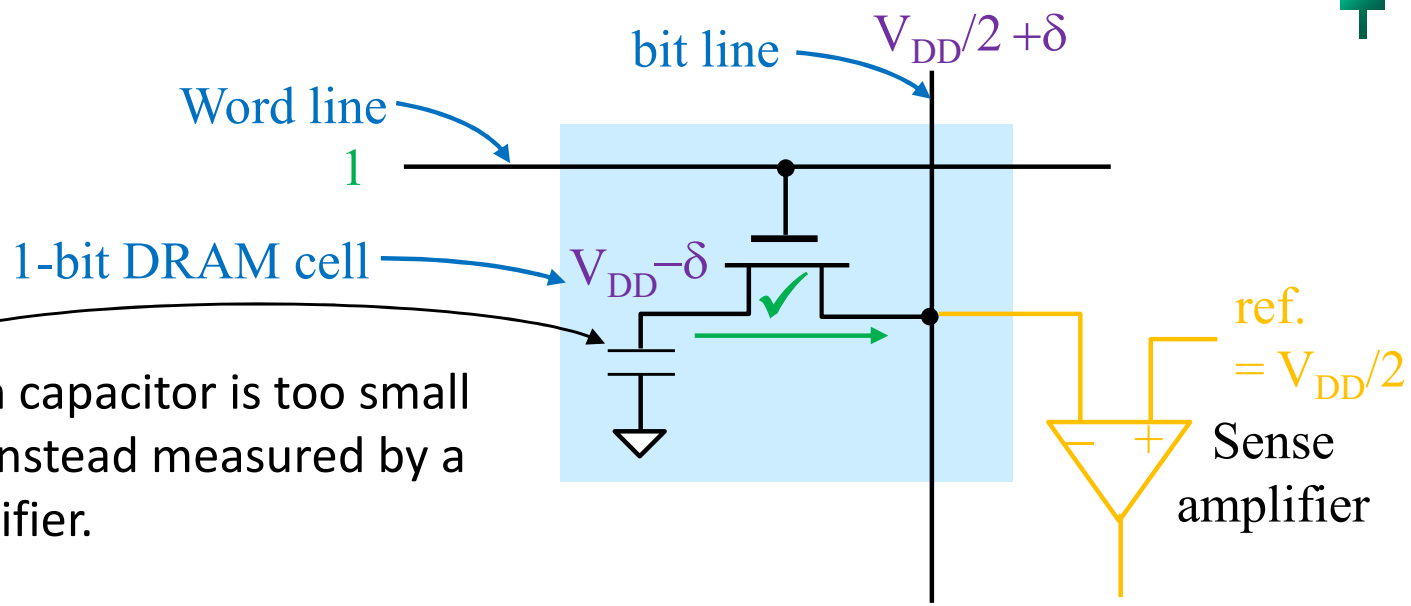
DRAM Read Operations



The charge stored on each capacitor is too small to be read directly and is instead measured by a circuit called a sense amplifier.

- Pre-charge bit line to $V_{DD}/2$.
- Take the word line HIGH.
- If the cell stores a logic 1 (V_{DD}),

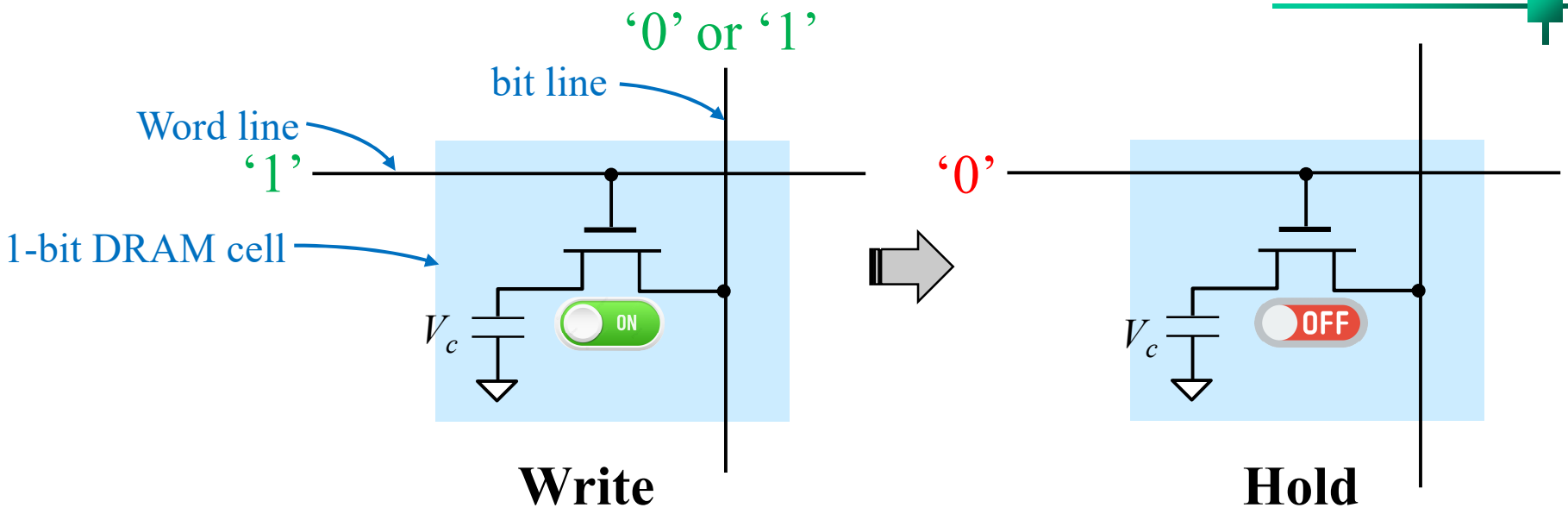
DRAM Read Operations



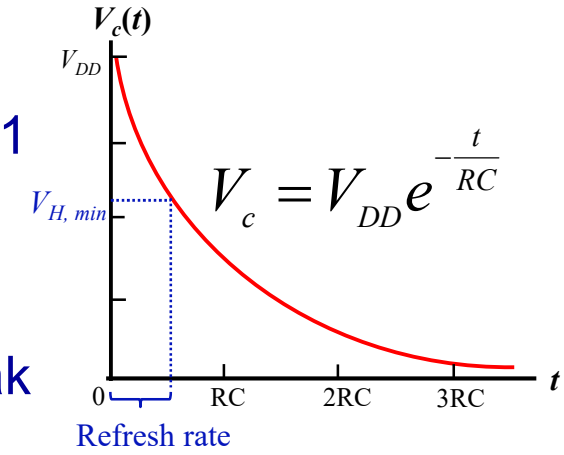
The charge stored on each capacitor is too small to be read directly and is instead measured by a circuit called a sense amplifier.

- Pre-charge bit line to $V_{DD}/2$.
- Take the word line HIGH.
- The cell (if at V_{DD}) shares its charge with the bitline ⇒ produce small change in the voltage of the bitline ($V_{DD}/2 + \delta$).
- Sensor amplifier amplifies δ towards the original value of the cell.
- Note: Reading from the bitline forces the charge to flow out of the capacitor ⇒ Cell contents are destroyed by the read!
- Must write the bit value back after reading.

DRAM Write Operations



- Take the word line HIGH
- Set the bit line LOW to store a 0 or HIGH to store a 1
- Take the word line LOW
- The stored charge for a 1 will eventually leak off
- Refresh rate is determined by the time *t* for V_c to leak from V_{DD} to minimum logic ‘1’ voltage.



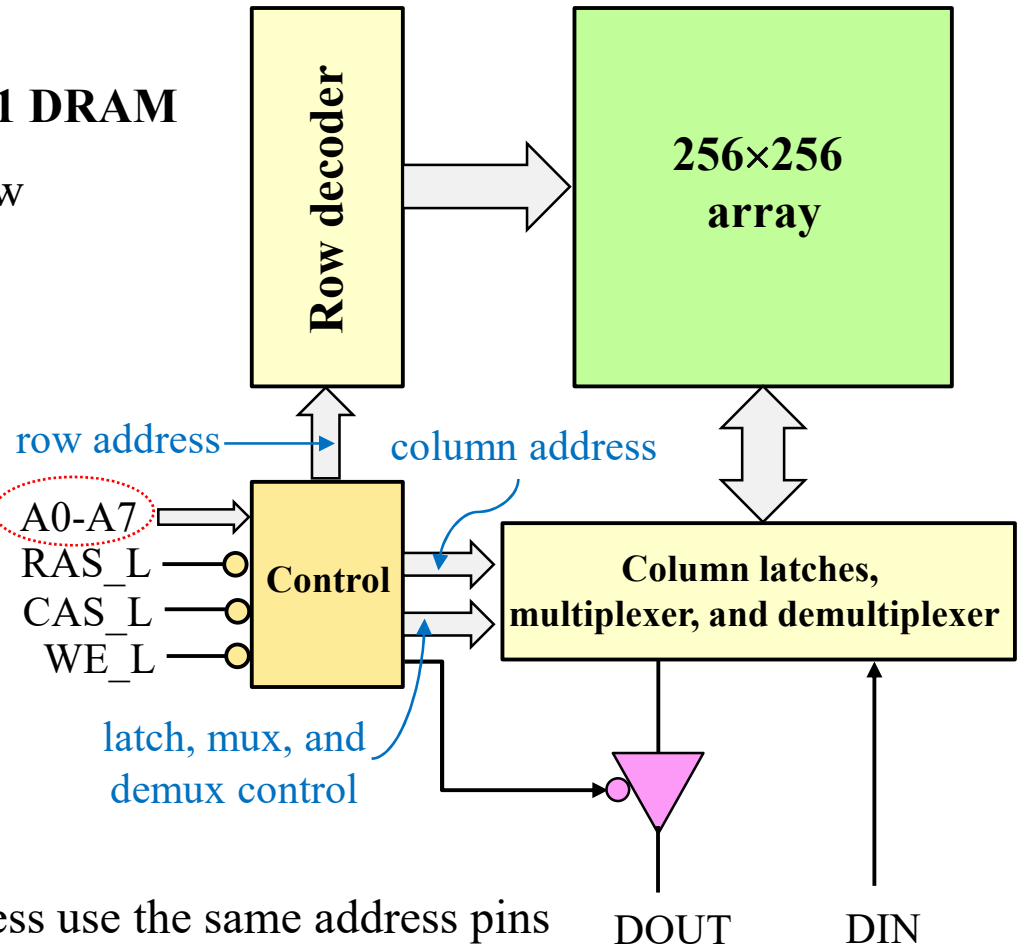
DRAM-chip Internal Organization

$64K = 2^6 \times 2^{10} = 2^{16}$

64K × 1 DRAM

multiplex 16-bit address as 8-bit row selector and 8-bit column selector

RAS: row-address strobe
CAS: column-address strobe



DRAMs share address pins
– Row address and column address use the same address pins
WE_L is used for read/write operation,
WE_L = 0, write; WE_L = 1, read

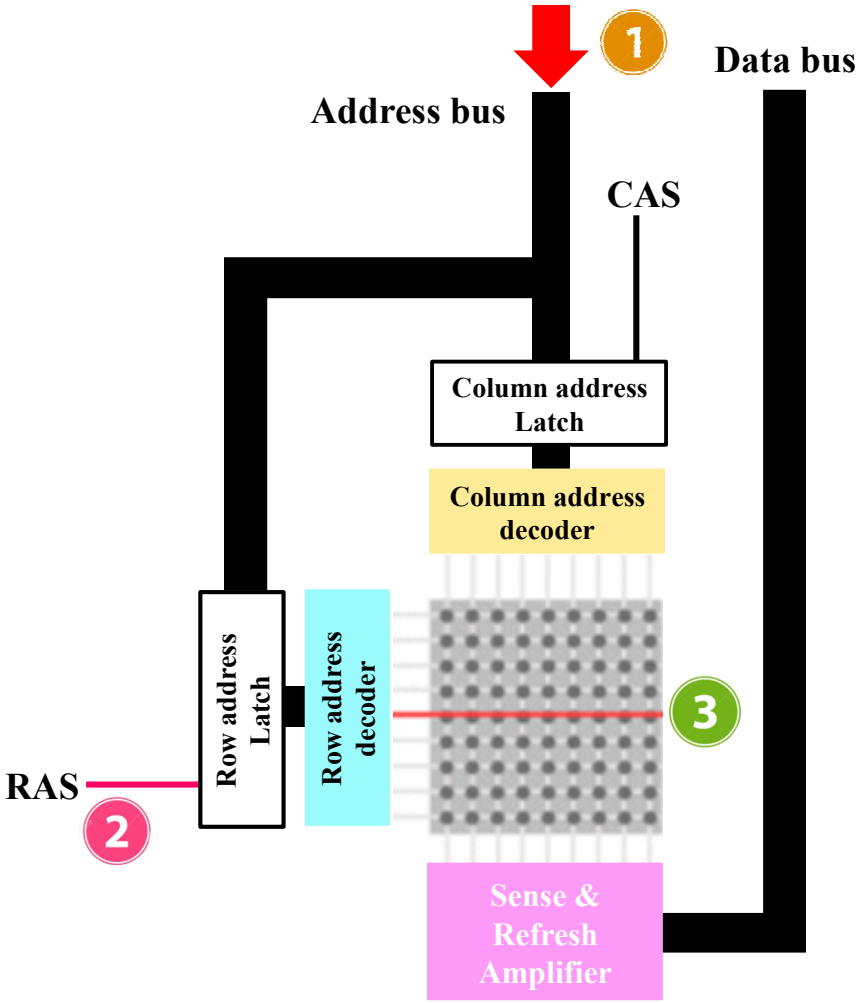
RAS/CAS Operation

- Row Address Strobe, Column Address Strobe.
  - *n* address bits are provided in two steps using *n*/2 pins, referenced to the falling edges of RAS_L and CAS_L.
  - *RAS and CAS* load row and column addresses.

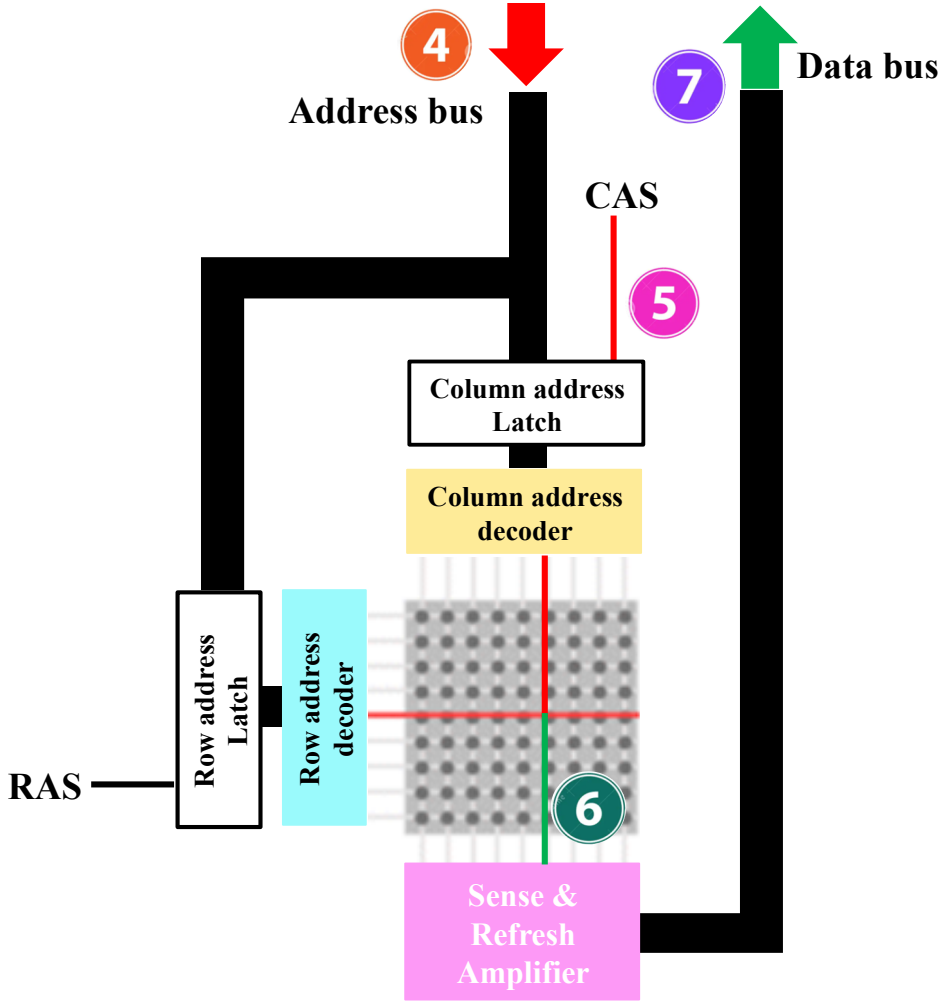
    Example: 1MB (2^{10} rows $\times$ 2^{10} columns) DRAM has 10 address pins:
    - RAS loads 10 row address bits
    - CAS loads 10 column address bits
  - Older DRAMs are *asynchronous.*

    Timing depends on falling edges of RAS and CAS.
  - Traditional method of DRAM operation for 20 years.
  - Now being supplanted by synchronous, clocked interfaces in SDRAM (synchronous DRAM).

RAS and CAS Operation

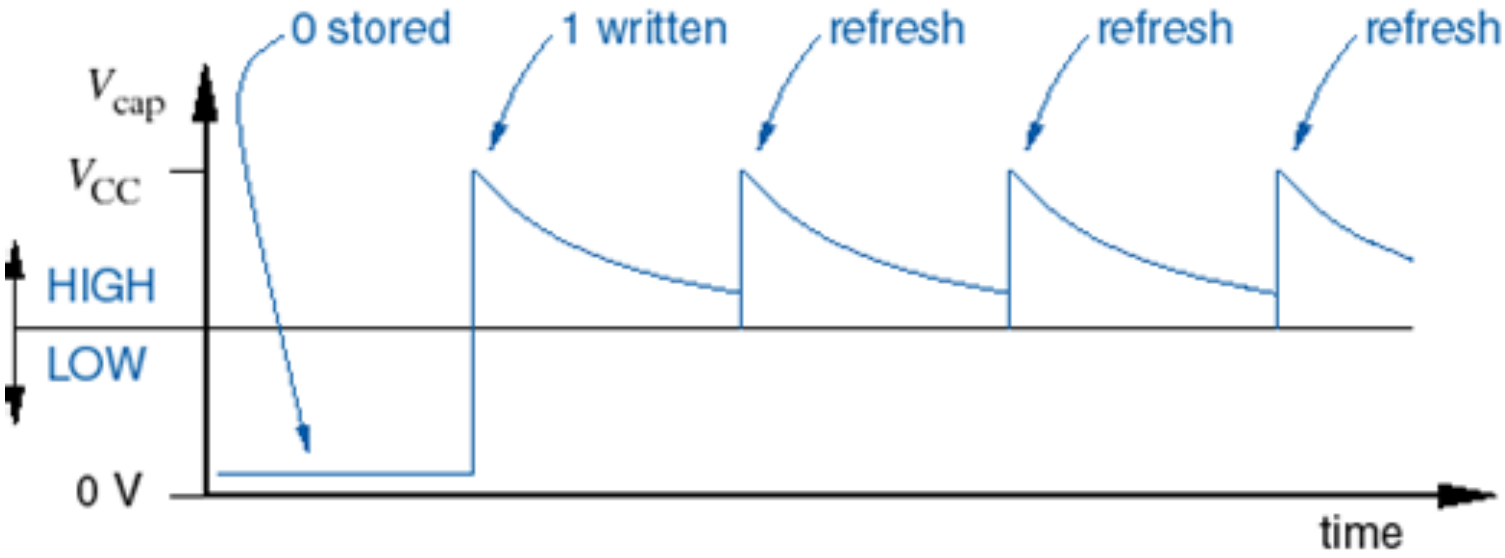


Row Address Latch controlled by RAS (Row Address Strobe) pin

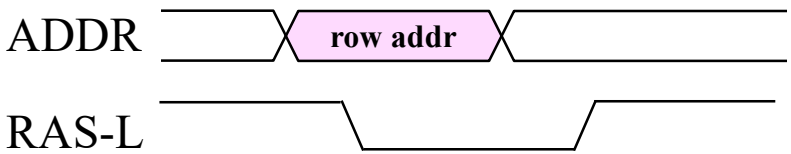


Column Address Latch controlled by CAS (Column Address Strobe) pin.

DRAM Charge Leakage



- Typical devices require each cell to be refreshed once every 4 to 64 *ms*.
- During "suspended" operation, notebook computers use power mainly for DRAM refresh.
- Assert RAS_L to refresh row.
  - Reads data and writes it back.
  - Takes ~1% of DRAM cycles.



DRAM Refresh Overhead

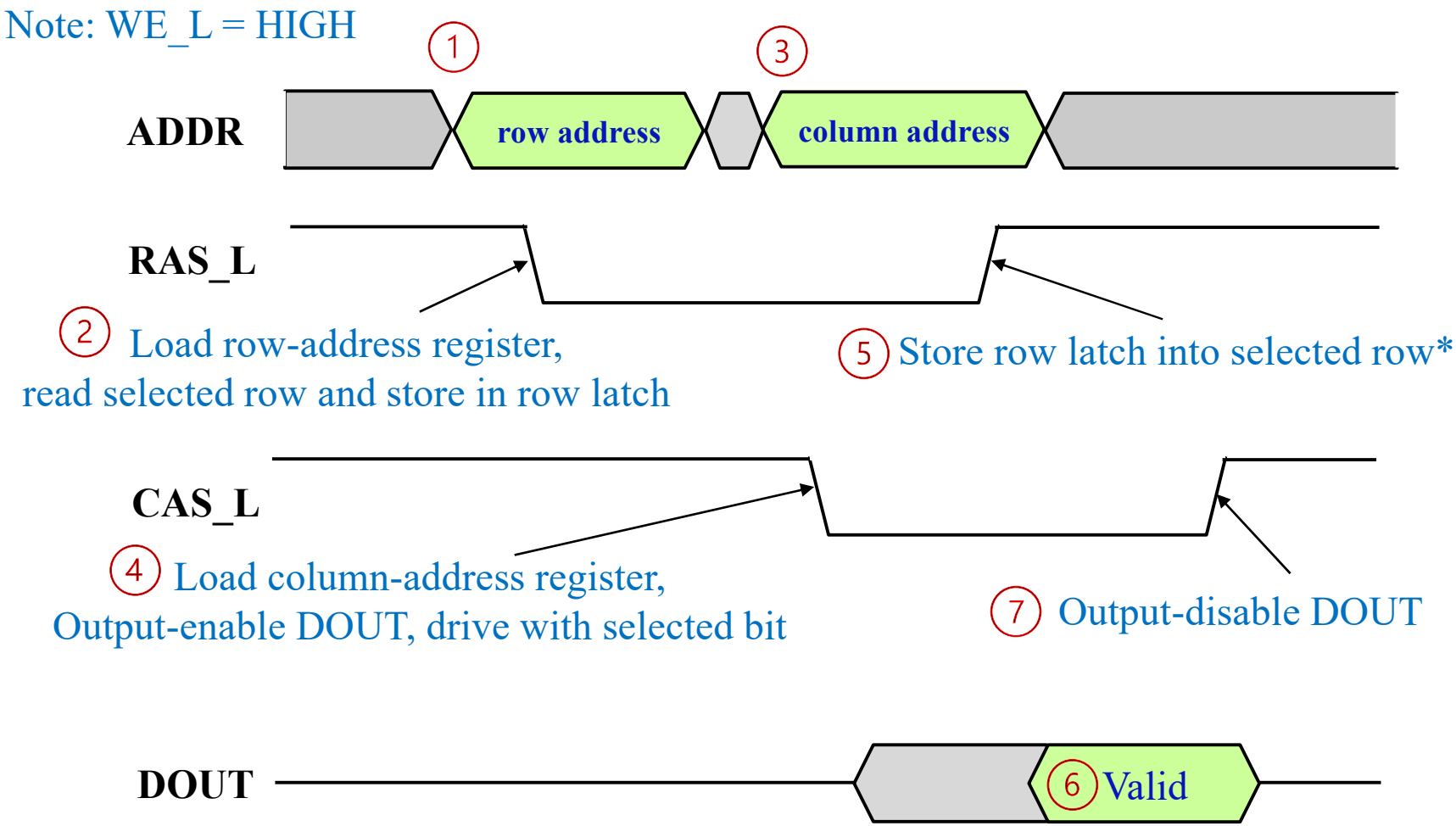
- The refresh circuitry consists of a refresh counter which contains the address of the row to be refreshed, and a timer that increments the counter to step through the rows.
- Two types of refresh schedule: burst or distributed refresh.
- Refresh interval (or refresh rate) = Data retention time of a DRAM cell.
- Number of refresh cycles = Number of rows in a DRAM array.
- Refresh cycle interval $= \frac{Refresh\ interval}{Number\ of\ refresh\ cycles}$

  - e.g., a DRAM with a refresh interval of 64 ms and 8,192 rows, the refresh cycle interval is 7.8 μs.
- Actual refresh time needed per row << refresh cycle interval

  $\therefore$ Refresh overhead = $\frac{actual\ refresh\ time\ required}{Refresh\ cycle\ interval}$
- The higher the size of the memory, the higher the refresh overhead.

  => memory of each chip is divided into banks, which are refreshed in parallel.

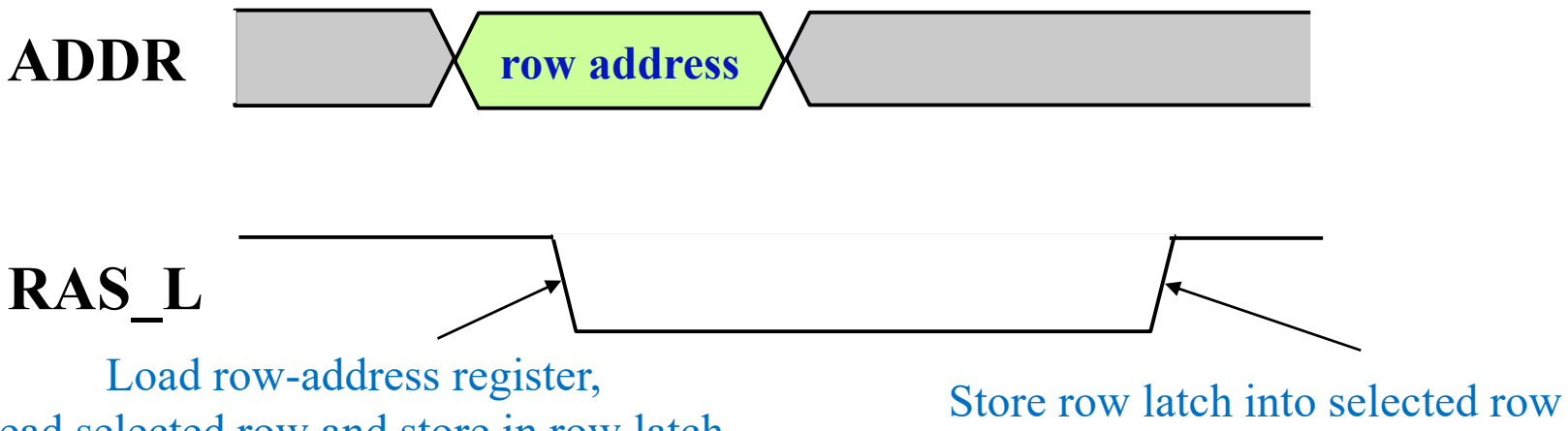
DRAM Read Timing



***Note:** DRAM use the row latch to store the content of the selected row address to refresh the selected row.

DRAM Refresh Timing

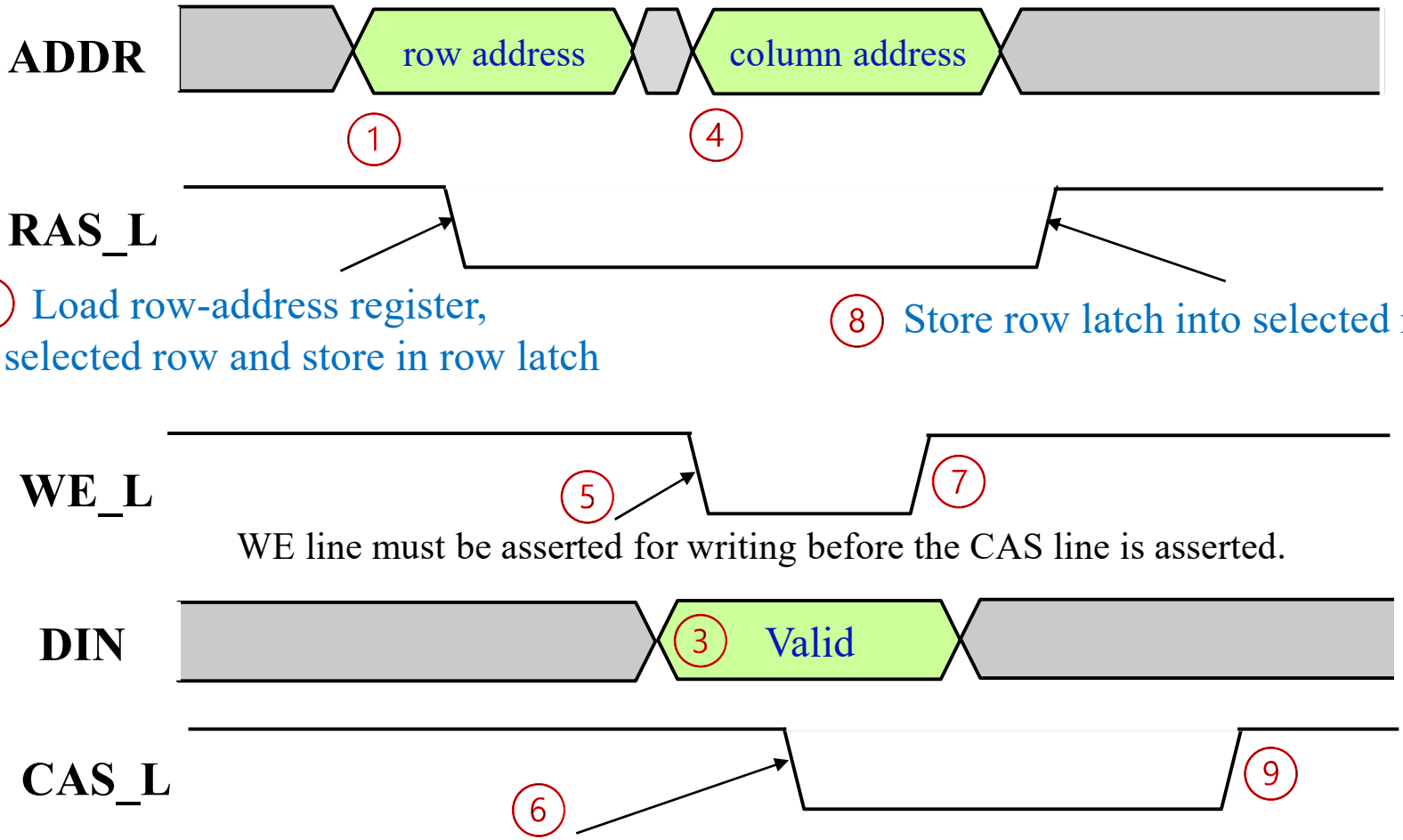
Note: WE_L = HIGH



- This cycle is used to refresh a row of memory without reading or writing any data at the external pins of the DRAM chip.
- The cycle begins when a row address is applied to the multiplexed address inputs and RAS_L is asserted.

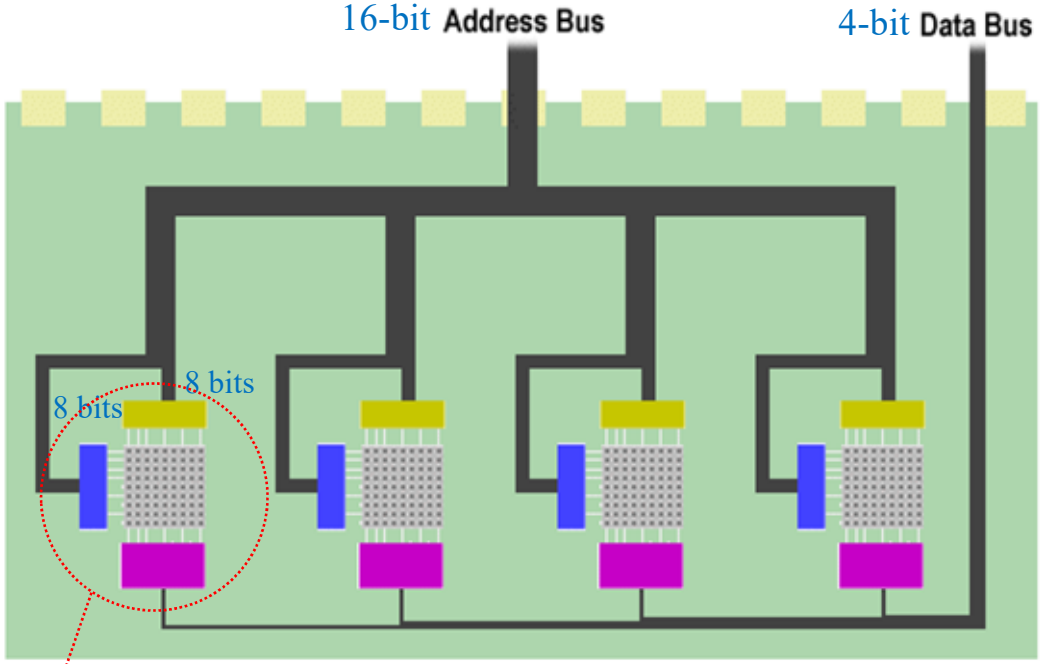
DRAM Write Timing

Note: DOUT = Hi-Z

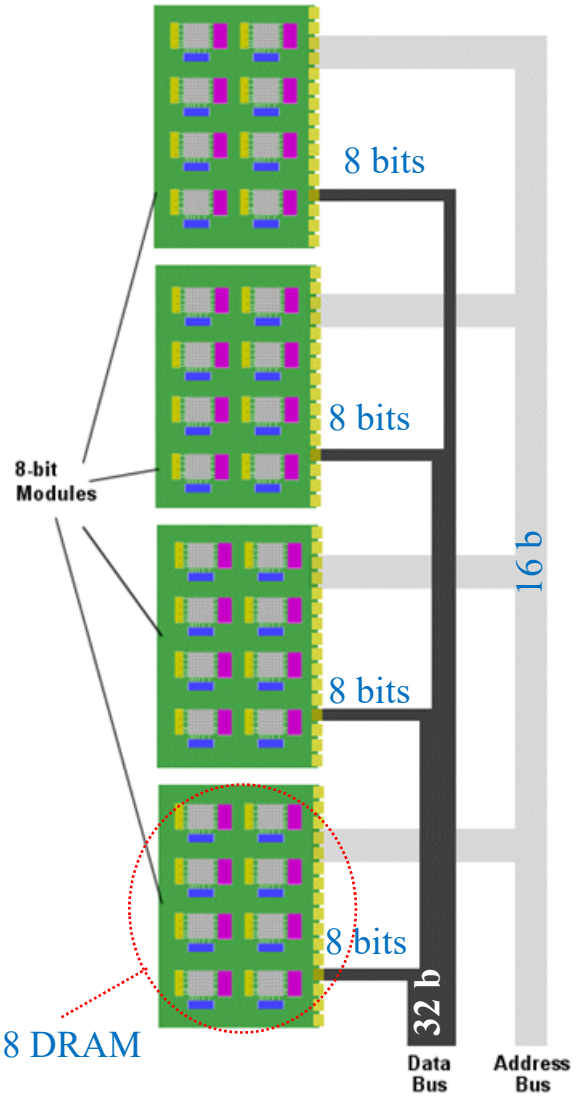


Word-Length Expansion

- Increasing data bit-width without increasing address lines.
  - ↓ Use four 64*k*×1 RAMs to form a 64*k*×4 RAM.
  - → Use four 64*k*×8 RAMs to form a 64*k*×32 RAM.



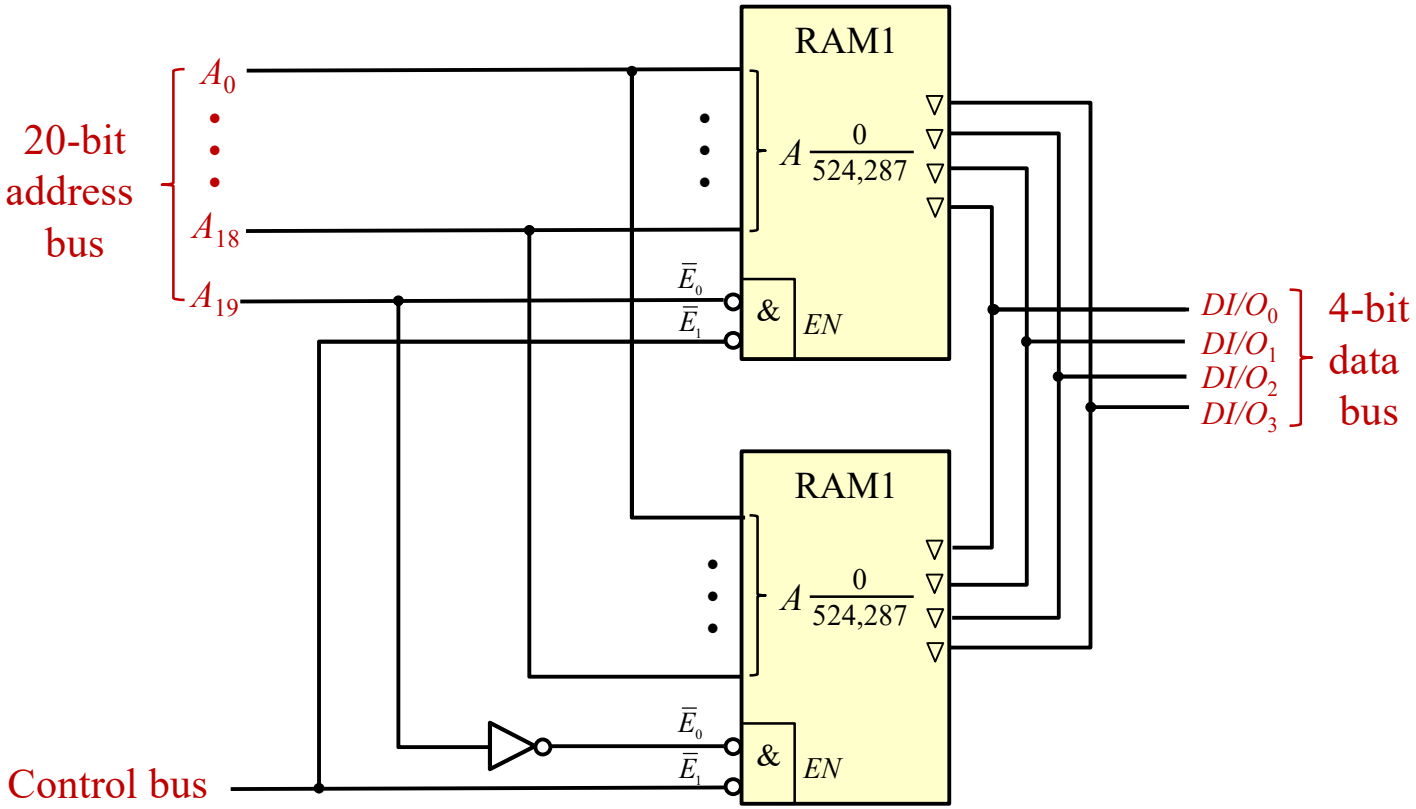
A 64*k* × 4 DRAM (4-bit module)



A 64k × 32 DRAM

Word-Capacity Expansion

- When the memory is expanded to increase word capacity, the number of addresses is increased .
  e.g., Use two 512k × 4 RAMs to implement a 1M×4 RAM.
  Note: 512k = 2^{19} and 1M = 2^{20}




The End
You are free